INTERVIEWHUB · INTERVIEW PREP

Ansible Automation — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**150 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/ansible_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

interviewhub.eknathalabs.co

certs.eknathalabs.com
InterviewHub · interviewhub.ekn

Q1 What is Ansible and how does its agentless architecture work?

Core concepts

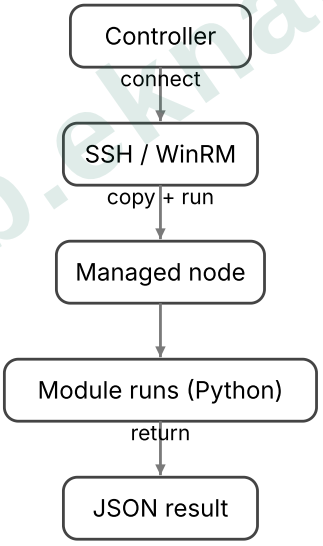
🎙 Spoken answer · ~59 s

Ansible is an automation tool for configuration management, application deployment and orchestration. A control node reads an inventory of target hosts and a playbook, which is a YAML description of desired state. For each task it generates a small module, copies it to the target over SSH or WinRM, runs it with the target's Python interpreter, collects the JSON result and removes the temporary files. Because nothing permanent is installed, there is no agent to patch, no extra port to open and no central server required, which makes onboarding fast and the attack surface small. The trade-offs are that SSH round trips cost time at large scale, the control node needs network reach to every host, and Python is needed on managed Linux hosts. Modules are designed to be idempotent, so running the same playbook twice should change nothing the second time.

Likely follow-ups

- What does a module run look like on the target?
- When would you choose an agent-based tool instead?

🗺 Agentless execution



A control node pushes small modules over SSH or WinRM, runs them, returns JSON and cleans up. No agent is installed, which keeps onboarding easy and the attack surface small, at the cost of SSH latency at very large scale.

```
ansible all -i inventory.ini -m ping
```

```
ansible-playbook -i inventory.ini site.yml
```

Interview tip: Say agentless plus idempotent modules, and name the scale trade-off.

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub · interviewhub.eknathalabs

📚 **Official docs**

- Ansible documentation

Q2 What is an inventory, and how do static and dynamic inventories differ?

Core concepts

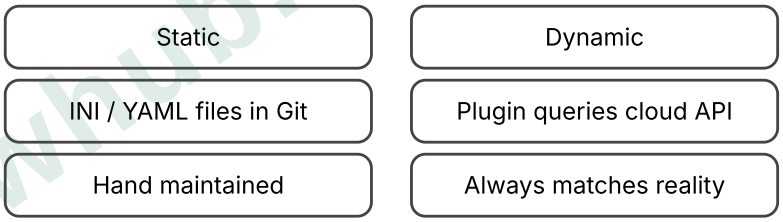
🎙 **Spoken answer · ~60 s**

An inventory tells Ansible which hosts exist and how they are grouped. A static inventory is an INI or YAML file listing hosts and groups, with variables attached to hosts or groups. It is simple, readable and ideal for stable environments. A dynamic inventory is produced at run time by an inventory plugin or script that queries a source of truth, such as AWS EC2, Azure, GCP, VMware, OpenStack or a CMDB, so the host list follows reality as instances come and go. Plugins usually build groups from tags, regions or other metadata with keyed groups and compose variables. In practice I combine them: dynamic sources for cloud fleets, static files for fixed infrastructure, and a directory containing both. I keep group_vars and host_vars beside the inventory, and use ansible-inventory with the graph or list options to verify what Ansible sees before running anything.

Likely follow-ups

- How do keyed groups work in a cloud inventory plugin?
- How do you verify what an inventory resolves to?

🗺 **Static vs dynamic inventory**



Static inventories are files listing hosts and groups; dynamic inventories use plugins to query AWS, Azure, VMware or a CMDB at run time. Combine both in a directory and check results with ansible-inventory.

```
ansible-inventory -i inventory/ --graph
```

```
ansible-inventory -i aws_ec2.yml --list | jq '."_meta".hostvars | keys'
```

Interview tip: Say you verify with ansible-inventory before any run.

📚 **Official docs**

- Ansible: Inventory
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q3 When do you use ad-hoc commands rather than playbooks?

Core concepts

🎙 Spoken answer · ~62 s

Ad-hoc commands run a single module against a set of hosts from the command line, with no file to write. They are ideal for quick, one-off operational tasks: checking connectivity with the ping module, gathering facts, checking disk space, restarting a service on a few hosts during an incident, copying a file, or testing that a new module and credentials work. Anything that should be repeated, reviewed, versioned or run by someone else belongs in a playbook, because playbooks document intent, can be code reviewed, are repeatable across environments and support handlers, loops, conditionals and variables. My rule is that an ad-hoc command is fine for investigation and for emergencies, but if I run the same command twice I turn it into a playbook or role. I also take care with the command and shell modules ad hoc, since they are not idempotent and I prefer purpose-built modules.

Likely follow-ups

- Why avoid shell in ad-hoc commands?
- How would you restart a service on 5 hosts quickly and safely?

Ad-hoc commands suit quick investigation or incident actions with one module. Repeatable or shared work goes into a playbook, which is reviewable, versioned and supports variables, handlers and conditionals.

```
ansible web -m service -a 'name=nginx state=restarted' -b
```

```
ansible all -m setup -a 'filter=ansible_distribution*'
```

Interview tip: Say if you run it twice, it becomes a playbook.

📚 Official docs

- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q4 What does idempotency mean in Ansible and how do you keep tasks idempotent?

Core concepts

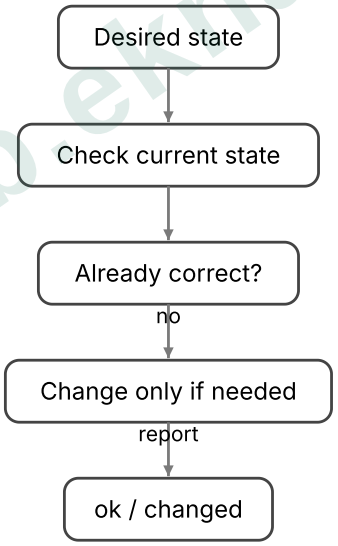
🎙 Spoken answer · ~62 s

Idempotency means running a task repeatedly leads to the same end state, and a second run reports no changes. Most core modules achieve this by describing desired state, for example the package module with state present, the file module with a mode, or the service module with state started. Each checks current state first and acts only if different. It matters because it makes playbooks safe to rerun, supports drift correction, and lets you trust the changed count as a signal. Problems arise with the command, shell and raw modules, which always report changed unless told otherwise. I fix that with creates or removes arguments, changed_when and failed_when conditions, or by replacing them with proper modules. I also verify by running twice in CI and expecting zero changes on the second run, and use check mode with diff to preview. Non-idempotent tasks hide real drift and cause noisy handlers.

Likely follow-ups

- How do you make a shell task report changed correctly?
- How would you test idempotency in CI?

🗺 Idempotent task



Idempotent tasks converge to the same state on every run, so a second run reports zero changes. Prefer state-based modules, use creates, changed_when and failed_when for command tasks, and test by running twice.

```
ansible-playbook site.yml --check --diff
```

```
ansible-playbook site.yml | grep -E 'changed=[1-9]' && echo 'not idempotent'
```

Interview tip: Mention the run-twice test in CI.

interviewhub.eknathalabs.com

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Ansible: Playbook guide
- Ansible documentation

Q5 How does Ansible compare with Puppet, Chef and Terraform, and where does each fit?

Core concepts

🎙 **Spoken answer · ~63 s**

Ansible is agentless and push-based, uses YAML and is easy to start with, so it is strong for orchestration, ad-hoc operations and configuration of existing machines. Puppet and Chef use agents that pull configuration from a server on a schedule, with a declarative language in Puppet and Ruby-based recipes in Chef, which gives continuous enforcement and drift correction at large scale but needs infrastructure to run. Terraform is different in kind: it provisions and manages the lifecycle of infrastructure resources such as networks, instances and managed services, tracking them in state, while Ansible is better at what happens inside the machine afterwards. A common pattern is Terraform to create infrastructure and Ansible to configure it, or to bake images with Packer and Ansible. There is overlap, since Ansible has cloud modules and Terraform has provisioners, so I choose by the lifecycle: create and destroy with Terraform, configure and operate with Ansible.

Likely follow-ups

- Why not use Ansible for all provisioning?
- What is the drawback of a pull-based agent model?

Ansible is agentless, push-based and strong for orchestration and configuration. Puppet and Chef are agent-based pull tools with continuous enforcement. Terraform manages infrastructure lifecycle with state; use it to create, and Ansible to configure.

```
terraform output -json instance_ips > ips.json
```

```
ansible-playbook -i inventory/from_tf.py configure.yml
```

Interview tip: Say Terraform creates, Ansible configures.

📚 **Official docs**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q6 How does Ansible choose configuration, and what is the precedence of ansible.cfg?

Core concepts

🎙 **Spoken answer · ~60 s**

Ansible reads settings from several places, and the first match wins rather than merging. The order is the ANSIBLE_CONFIG environment variable, then ansible.cfg in the current directory, then a hidden ansible.cfg in the user's home, then the system file in etc. Individual settings can also be overridden by environment variables such as ANSIBLE_FORKS and by command line flags, which take priority. Because the current directory file is picked up, a project-level ansible.cfg checked into the repository makes behaviour reproducible for the team and in CI, including the inventory path, roles path, remote user, forks, pipelining, callbacks and the collections path. One security detail: Ansible ignores a configuration file in a world-writable directory, to prevent hijacking. I use ansible-config dump with the only-changed option to see which values are in effect and where they came from, which is the fastest way to debug surprising behaviour.

Likely follow-ups

- Why is a world-writable ansible.cfg ignored?
- How do you see the effective configuration?

The first config found wins: ANSIBLE_CONFIG, then ansible.cfg in the current directory, home, then system. Environment variables and flags override. Keep a project ansible.cfg in the repo and inspect with ansible-config dump.

```
ansible-config dump --only-changed
```

```
ansible --version
```

Interview tip: Mention a repo-level ansible.cfg for reproducibility.

📚 **Official docs**

- Ansible: Variables and precedence
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q7 What are facts, and when would you disable or customise fact gathering? Core concepts

🎙 Spoken answer · ~60 s

Facts are system information that Ansible collects from each host at the start of a play using the setup module, such as operating system, distribution, network interfaces, memory and mounts. They are available as variables, for example ansible_facts distribution or ansible_default_ipv4, and let playbooks adapt to different platforms. Gathering costs a round trip and a second or more per host, which is significant at scale. I disable it with gather_facts false when a play needs none, or limit it with gather_subset to only what I use, such as min or network. Fact caching with a backend like Redis or JSON files avoids recomputing across runs. I can also write custom facts as files on the host in the facts.d directory, which Ansible exposes under ansible_local, useful for exposing application or compliance data. I prefer the ansible_facts dictionary over injected top-level variables for clarity.

Likely follow-ups

- How does fact caching speed up large runs?
- What are local facts?

Facts are host details gathered by the setup module at play start and used as variables. Disable them or limit with gather_subset to save time, cache them for large fleets, and add custom facts under facts.d.

```
ansible web -m setup -a 'gather_subset=min'
```

```
- hosts: all
  gather_facts: false
```

Interview tip: Say gather_subset and caching are the scale levers.

📚 Official docs

- Ansible: Variables and precedence

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q8 How do connection methods and privilege escalation (become) work?

Core concepts

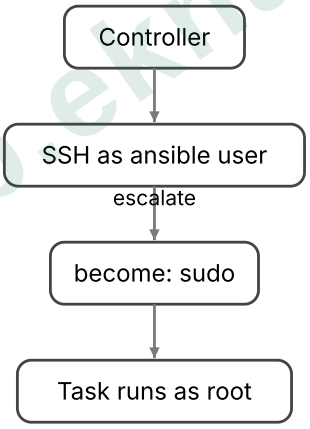
🎙 Spoken answer · ~59 s

Ansible connects through a connection plugin: SSH by default for Linux and network devices using paramiko or OpenSSH, WinRM or SSH for Windows, local for the control node, and docker or kubectl style plugins for containers. Authentication is normally by SSH keys through an agent, with the remote user set in inventory, config or play. Privilege escalation is controlled by become: when true, tasks run through sudo by default, or su, doas and others, as the become user, root unless specified, and the become password can be supplied or sudo can be passwordless for the automation account. I set become at play level only where needed rather than globally, and use become_user for application accounts. Good practice is a dedicated automation user, key-based authentication, restricted sudoers entries where feasible, and a bastion through ProxyCommand or ansible_ssh_common_args when hosts are in private networks.

Likely follow-ups

- How would you reach hosts behind a bastion?
- What is the risk of passwordless sudo for all commands?

🗺 Connection and become



Connection plugins such as SSH or WinRM reach hosts; become escalates with sudo or su. Use a dedicated automation user with keys, apply become only where needed, and use a bastion through ProxyCommand for private networks.

```
ansible_ssh_common_args='-o ProxyCommand="ssh -W %h:%p -q bastion"'
```

```
- hosts: db
  become: true
  become_user: postgres
```

Interview tip: Say least privilege: become only on tasks that need it.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Kubernetes: RBAC good practices

Q9 What is the difference between modules, plugins and collections?

Core concepts

🎙 **Spoken answer · ~59 s**

A module is a unit of work that runs on the target host, such as copy, apt, user or service, and returns structured results. A plugin extends Ansible itself and runs on the control node: connection plugins, lookup plugins, filter plugins, callback plugins, inventory plugins and strategy plugins, among others. So modules do the managed work, while plugins change how Ansible connects, reads data, formats output or processes variables. A collection is a distribution format that bundles modules, plugins, roles, playbooks and documentation under a namespace, for example amazon.aws or kubernetes.core, versioned and installed from Galaxy, Automation Hub or a private source. Since ansible-core 2.10 most content lives in collections, so the core package stays small and content releases independently. In playbooks I use fully qualified collection names like ansible.builtin.copy to avoid ambiguity, and pin collection versions in requirements.yml for reproducible runs.

Likely follow-ups

- Where do lookup plugins run?
- Why pin collection versions?

Modules run on targets and do the work. Plugins run on the control node and extend connections, lookups, filters and callbacks. Collections package modules, plugins and roles under a namespace and are versioned and pinned in requirements.yml.

```
ansible-doc -t module ansible.builtin.copy
```

```
ansible-galaxy collection install -r requirements.yml
```

Interview tip: Say modules run on targets, plugins on the controller.

📚 **Official docs**

- Ansible: Roles
- Ansible: Developing modules
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q10 What is the difference between ansible-core and the Ansible community package, and how do you manage versions?

Core concepts

🎙 Spoken answer · ~60 s

Ansible-core is the engine: the command line tools, the language, the built-in collection and the plugin framework. The community Ansible package, sometimes called the batteries-included distribution, bundles ansible-core with a curated set of collections, so a single install gives you many modules. Red Hat's Ansible Automation Platform is a supported product with certified collections, execution environments and a controller. Version management matters because modules and syntax change: each ansible-core release supports a range of Python versions on the control node and on targets, and old targets may need an older interpreter. I pin ansible-core and collection versions in a requirements file, ideally packaged in an execution environment container image, so laptops, CI and the controller all run the same versions. Upgrades are tested in a branch with ansible-lint and Molecule, and I read the porting guides before moving to a new major release.

Likely follow-ups

- What breaks when upgrading ansible-core?
- Why use an execution environment image?

Ansible-core is the engine and built-ins; the community package adds many collections. Pin ansible-core and collection versions, ideally in an execution environment image, and test upgrades with lint and Molecule before rollout.

```
pip install 'ansible-core==2.17.*'
```

```
ansible-galaxy collection list
```

Interview tip: Say pin versions and package them in an execution environment.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q11 Describe the structure of a playbook, plays and tasks.

Playbooks & modules

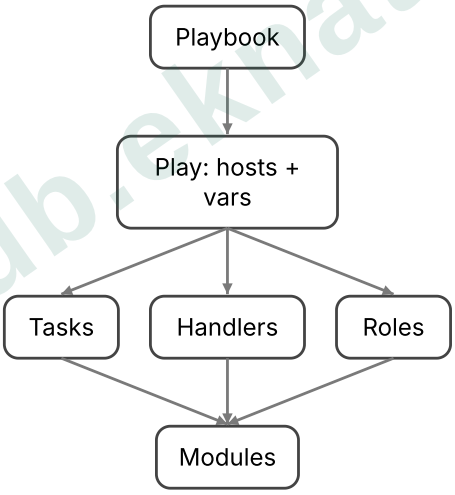
🎙 Spoken answer · ~63 s

A playbook is a YAML file containing a list of plays. Each play maps a group of hosts to a set of tasks and defines settings such as the remote user, become, variables, gather_facts and tags. Within a play, execution is ordered: pre_tasks, then roles, then tasks, then post_tasks, with handlers notified along the way and run at defined points. Each task calls one module with arguments, may have a name, conditions, loops, register, tags, delegation, error handling and notify. Tasks run in order against all hosts in the play in parallel, up to the forks limit, using the linear strategy by default, so every host completes a task before the next starts. Good style is to give every task a descriptive name, keep plays short by moving logic into roles, use fully qualified module names, keep YAML valid and indent consistently, and put variables in group_vars rather than inside the play.

Likely follow-ups

- In what order do pre_tasks, roles and tasks run?
- What does the linear strategy mean?

🗺 Playbook anatomy



A playbook is a list of plays; each play targets hosts, sets options and runs pre_tasks, roles, tasks and post_tasks in order. Tasks call modules, run across hosts in parallel, and notify handlers. Keep plays short and named.

```
- name: Configure web
  hosts: web
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
```

certs.eknathalabs.com

InterviewHub - interviewhub.ekn

Interview tip: Say name every task and push logic into roles.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

Q12 How do handlers work and what are the pitfalls?

Playbooks & modules

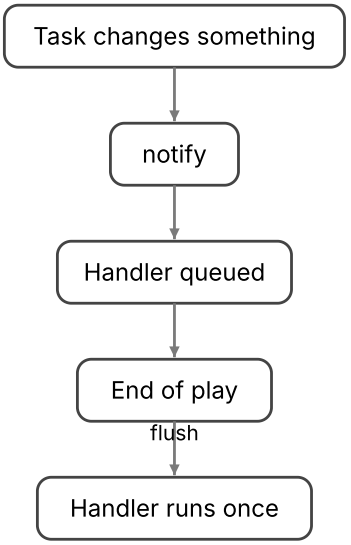
🎙 **Spoken answer · ~65 s**

Handlers are tasks that run only when notified by another task that reported a change, typically to restart or reload a service after its configuration file changed. Multiple notifications of the same handler result in a single run, and handlers run at the end of the play by default, in the order they are defined, not the order they were notified. This design avoids restarting a service several times. Pitfalls include a handler not running because the notifying task failed before the end of the play, since by default a failure stops the flow and skips handlers unless force_handlers is set. Another is needing the restart earlier, in the middle of a play, which you handle with the meta flush_handlers task. Handlers also only fire on change, so a first run that fails halfway can leave a changed file with no restart on the next run, and listen topics allow one notification to trigger several handlers.

Likely follow-ups

- Why might a handler not run after a failure?
- How do you force a handler to run mid-play?

🗺 **Handlers**



Handlers run once at the end of a play when notified by changed tasks, in definition order. Use meta flush_handlers to run earlier, force_handlers to run after failures, and listen to group handlers.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- name: Update config
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app.conf
  notify: Restart app
```

```
- ansible.builtin.meta: flush_handlers
```

Interview tip: Mention flush_handlers and the failed-play pitfall.

📚 **Official docs**

- Ansible: Playbook guide

Q13 How do loops work in Ansible, and when should you avoid them?

Playbooks & modules

🎙 **Spoken answer · ~62 s**

The loop keyword repeats a task over a list, exposing each element as the item variable, which can be renamed with loop_control and loop_var. Loops can iterate over simple lists, lists of dictionaries, dictionaries converted with the dict2items filter, or results from lookups and filters such as subelements and product. You can add labels to keep output readable, a pause for rate limits, and an index variable. Older with_ keywords still work but loop is preferred. I avoid loops when the module already accepts a list, for example the package module accepts many names in one call, which is much faster since it runs once rather than once per item. Another problem is loops with register, which return a results list, and large loops over big data in Jinja, which are slow. Also nested loops in included tasks need loop_var set to avoid clashes with the outer item.

Likely follow-ups

- Why does loop_var matter in nested includes?
- How do you loop over a dictionary?

Use loop with item, loop_control for labels and renamed variables, dict2items for dictionaries. Avoid loops where a module accepts a list, such as package, since one call is faster than many.

```
- name: Create users
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
  loop: "{{ users }}"
  loop_control:
    label: "{{ item.name }}"
```

Interview tip: Say pass a list to the module instead of looping where possible.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

Q14 How do conditionals with when and failed_when and changed_when work?

Playbooks & modules

🎙 **Spoken answer · ~65 s**

The when keyword runs a task only if a Jinja2 expression is true, written without double curly braces, for example when ansible_os_family equals Debian, or when a registered variable is defined or contains a string. Conditions can combine with and, or and not, and a list of conditions means all must be true. On loops, when is evaluated per item. The failed_when keyword overrides what counts as a failure, useful when a command returns a nonzero code that is acceptable or prints an error message with a zero exit code. Similarly changed_when controls whether the task reports a change, which is essential for making command and shell tasks idempotent and for accurate handler notification. Common mistakes include comparing strings and booleans loosely, forgetting that variables from the command line are strings, and using when with an undefined variable without the default filter. I also use the is defined, is changed and is failed tests for clarity.

Likely follow-ups

- How do you treat a registered result as failed on certain output?
- Why do command-line booleans cause problems?

when runs a task on a true expression; failed_when and changed_when redefine failure and change for tasks such as command. Use defaults for undefined variables and watch for string versus boolean types.

```
- ansible.builtin.command: /usr/local/bin/check_app
  register: r
  failed_when: "'CRITICAL' in r.stdout"
  changed_when: false
```

Interview tip: Say changed_when false for read-only commands.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Error handling

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q15 How do you handle errors with block, rescue and always, and ignore_errors?

Playbooks & modules

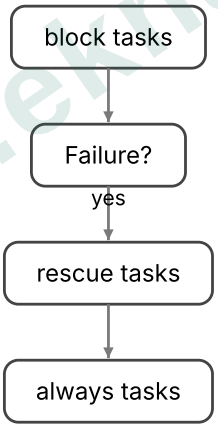
🎙 Spoken answer · ~59 s

A block groups tasks so they share conditions, privilege settings and error handling. The rescue section runs if any task in the block fails, similar to catch, and the always section runs regardless, similar to finally. The pattern lets you attempt a risky operation, such as a deployment, roll back in rescue, and always send a notification or clean up in always. After a successful rescue the play continues for that host and the host is not marked failed, though the failure is shown in the output. The ignore_errors keyword simply continues past a failing task but hides real problems, so I use it sparingly and prefer targeted failed_when conditions. Related controls are any_errors_fatal to stop all hosts on a failure, max_fail_percentage for rolling deployments, and ignore_unreachable. Inside rescue, the ansible_failed_task and ansible_failed_result variables describe what failed, which helps with diagnostics.

Likely follow-ups

- What variables are available inside rescue?
- When would you use any_errors_fatal?

🗺 block / rescue / always



block groups tasks; rescue handles failures and always runs regardless, like try, catch and finally. Prefer this and targeted failed_when over ignore_errors, and use any_errors_fatal or max_fail_percentage for rolling changes.

```
- block:
    - ansible.builtin.include_role: {name: deploy}
  rescue:
    - ansible.builtin.include_role: {name: rollback}
  always:
    - ansible.builtin.debug: {msg: "done"}
```

Interview tip: Say rescue for rollback, always for cleanup.

certs.eknathalabs.com

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Error handling

Q16 How are tags used, and what are the best practices?

Playbooks & modules

🎙 **Spoken answer · ~65 s**

Tags label tasks, blocks, roles and plays so you can run or skip a subset of a playbook with the tags and skip-tags options, for example only configuration changes or only the deployment step. Special tags include always, which runs unless skipped explicitly, and never, plus tagged, untagged and all. Tags applied to a role or an include are inherited by the contained tasks differently depending on whether it is a static import or dynamic include, which is a frequent source of confusion: with include_role you need the apply keyword to pass tags to the inner tasks. Best practice is a small, documented vocabulary such as packages, config, deploy and security, rather than ad-hoc tags, and using list-tags to document them. I do not rely on tags for safety, because a partial run may leave things inconsistent, and full runs in CI remain the source of truth. Tags are great for fast iteration during development.

Likely follow-ups

- How do tags behave with include_role?
- What does the always tag do?

Tags let you run or skip subsets with --tags and --skip-tags. Imports inherit tags, dynamic includes need apply. Keep a small documented tag vocabulary and rely on full runs in CI.

```
ansible-playbook site.yml --tags config --skip-tags deploy
```

```
ansible-playbook site.yml --list-tags
```

Interview tip: Mention the import versus include difference.

📚 **Official docs**

- Ansible: Playbook guide
- ansible-lint

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q17 What do delegate_to, run_once and local_action do, and when do you use them?

Playbooks & modules

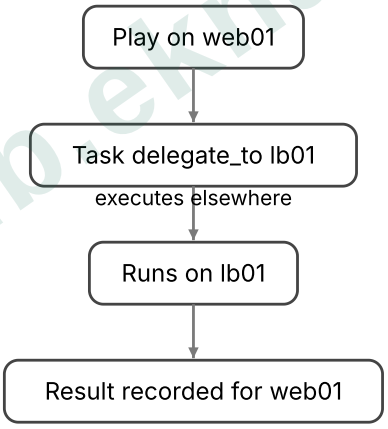
🎙 Spoken answer · ~62 s

The delegate_to keyword runs a task on a different host than the one being iterated, while still using the original host's variables. It is used for actions that must happen elsewhere: removing a host from a load balancer from the control node before updating it, updating a DNS record through an API, or registering with a monitoring server. With delegate_to localhost, tasks run on the control node, and local_action is a shorthand for that. The run_once keyword executes a task a single time for the whole play, on the first host in the batch, for example one database migration or sending one notification, and the result is shared with all hosts. Cautions are that run_once with serial applies per batch, that delegated tasks use facts of the original host unless delegate_facts is set, and that connection and become settings apply to the delegate target, so credentials must work there.

Likely follow-ups

- How does run_once behave with serial?
- What is delegate_facts?

🗺 Delegation



delegate_to runs a task on another host while keeping the current host's variables, for load balancer, DNS or monitoring calls. run_once executes a task once per play or batch. Watch serial interactions and delegated credentials.

```
- name: Drain from LB
  ansible.builtin.uri:
    url: "https://lb/api/drain/{{ inventory_hostname }}"
    method: POST
  delegate_to: localhost
```

Interview tip: Say run_once applies per serial batch.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Start here (official docs)**

- Ansible documentation

Q18 How do check mode and diff mode work, and what are their limits?

Playbooks & modules

🎙 **Spoken answer · ~64 s**

Check mode, enabled with the check option, simulates a run: modules that support it report what would change without changing anything. Diff mode, enabled with the diff option, shows a before and after comparison for files and templates. Together they give a dry run for review, useful in change approval and in pull request pipelines. The limits are important: modules that do not support check mode are skipped or report nothing, command and shell tasks are skipped by default unless a check_mode override is set, and later tasks that depend on earlier changes or registered output may fail or show misleading results because the earlier change never happened. You can mark a task with check_mode true or false to force behaviour, for example to run a read-only command even in check mode. So check mode is a strong signal rather than a guarantee, and I still verify in a staging environment before production.

Likely follow-ups

- Why do some tasks fail only in check mode?
- How do you force a command to run in check mode?

Check mode simulates changes and diff mode shows file differences, giving a reviewable dry run. Limits include modules without support, skipped command tasks and dependent tasks, so treat it as a signal, not a guarantee.

```
ansible-playbook site.yml --check --diff -l staging
```

```
- ansible.builtin.command: cat /etc/app.conf
  check_mode: false
  changed_when: false
```

Interview tip: Say dry-run is advisory; staging remains the proof.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q19 What is the difference between command, shell, raw and script modules?

Playbooks & modules

🎙 **Spoken answer · ~62 s**

The command module runs a command without a shell, so pipes, redirects, environment expansion and operators do not work, which also makes it safer against injection. The shell module runs through a shell, supporting pipes and redirects, with more risk if variables contain untrusted input. The raw module sends the command straight over SSH without needing Python on the target, useful for bootstrapping a host that lacks Python or for network devices. The script module copies a local script to the target and runs it there. None of them is idempotent by default, so each always reports changed unless you use creates or removes, changed_when or failed_when. My order of preference is: a dedicated module first, then command, then shell only when a shell feature is truly required, and then I quote variables carefully with the quote filter. Raw is used only for bootstrap, such as installing Python.

Likely follow-ups

- When is raw appropriate?
- How do you protect shell tasks from injection?

command runs without a shell and is safer; shell allows pipes but risks injection; raw needs no Python and suits bootstrapping; script copies and runs a local script. Prefer proper modules and make these idempotent with creates, removes and changed_when.

```
- ansible.builtin.command: /opt/app/migrate
  args:
    creates: /var/lib/app/.migrated
```

```
- ansible.builtin.shell: "ps aux | grep {{ name | quote }}"
```

Interview tip: Say a module first, then command, shell last.

📚 **Official docs**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q20 What is the difference between include and import for tasks and roles?

Playbooks & modules

🎙 **Spoken answer · ~63 s**

Imports are static: the import_tasks and import_role statements are processed when the playbook is parsed, so the content is pre-loaded, tags and conditionals are applied to every contained task, and you can list all tasks with list-tasks. Includes are dynamic: include_tasks and include_role are processed when execution reaches them, so file names and loops can use runtime variables, and the included content is not visible to list-tasks. When a condition is attached, with an import it is copied to each inner task, with an include it is evaluated once and gates the whole include. Tags behave differently too: tags on an include apply to the include itself unless you use apply. I use imports for fixed structure because they are predictable, and includes when the file or role depends on facts, loops or conditions known only at run time. Mixing them without understanding this causes skipped tasks and tag surprises.

Likely follow-ups

- Which one supports looping?
- Why might tags seem to be ignored on an include?

Import is static, processed at parse time, applying tags and conditions to inner tasks. Include is dynamic, evaluated at run time, so it supports loops and variable file names. Use imports for fixed structure and includes for runtime decisions.

```
- ansible.builtin.import_tasks: common.yml
```

```
- ansible.builtin.include_tasks: "{{ ansible_os_family }}.yml"
```

Interview tip: Say static for structure, dynamic for runtime choices.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Roles

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q21 What is the structure of an Ansible role and why use roles?

Roles & collections

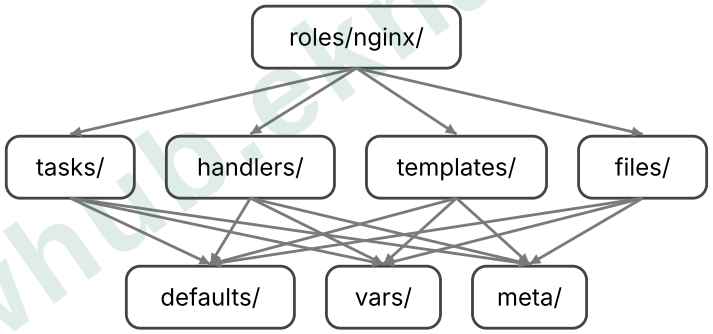
🎙 Spoken answer · ~57 s

A role is a standard directory layout for reusable automation. Key directories are tasks with main.yml as the entry point, handlers, defaults for low-precedence variables meant to be overridden, vars for high-precedence internal values, templates for Jinja2 files, files for static files, meta for dependencies and metadata, and optionally library, module_utils and tests. Roles let you package one capability, such as an nginx setup or a hardening baseline, and reuse it across playbooks, teams and environments with different variables. They improve readability because playbooks become short lists of roles, enable testing in isolation with Molecule, and can be shared through collections or Galaxy. Good practice is one role per clear responsibility, a documented README with variables, defaults for everything configurable, prefixing variable names with the role name to avoid collisions, no hidden dependencies, and idempotent, platform-aware tasks.

Likely follow-ups

- Why prefix variables with the role name?
- What belongs in defaults versus vars?

🗺 Role layout



A role has tasks, handlers, defaults, vars, templates, files and meta. It packages one capability for reuse and isolated testing. Keep one responsibility, document variables, prefix names and put overridable values in defaults.

```
ansible-galaxy role init nginx
```

```
tree roles/nginx
```

Interview tip: Say prefix variables to avoid collisions.

📚 Official docs

- Ansible: Roles
- Ansible documentation

certs.eknathalabs.com

Q22 How do role dependencies work and what are their pitfalls?

Roles & collections

🎙 Spoken answer · ~61 s

Dependencies are declared in the role's meta main file as a list of other roles that must run first, optionally with parameters. When the playbook applies the role, Ansible executes the dependencies before it, and by default each dependency runs only once even if several roles list it, unless allow_duplicates is set. They are useful for shared prerequisites such as a common baseline or a repository setup role. Pitfalls include hidden coupling that makes a role harder to understand and test alone, ordering surprises when dependencies pull in tasks the playbook author did not expect, variable precedence changes because dependency parameters are applied as role variables, and difficulty when a dependency comes from another collection. Many teams prefer explicit composition: the playbook lists roles in order, or a higher-level role uses include_role, so the flow is visible. I keep dependencies minimal, documented and version-pinned in requirements.yml.

Likely follow-ups

- What does allow_duplicates do?
- Why prefer explicit role ordering in the playbook?

Dependencies in meta main run before a role and execute once by default unless allow_duplicates is set. They hide coupling and affect variable precedence, so keep them minimal and often compose roles explicitly in the playbook.

```
# roles/app/meta/main.yml
dependencies:
  - role: common
  - role: nginx
    vars:
      nginx_port: 8080
```

Interview tip: Say explicit composition is clearer than hidden dependencies.

📚 Official docs

- Ansible: Roles
- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q23 What are Ansible collections and how do you use and publish them?

Roles & collections

🎙 **Spoken answer · ~60 s**

A collection is a versioned package that can contain modules, plugins, roles, playbooks and docs under a namespace dot name identifier. Users install them with ansible-galaxy collection install, from Galaxy, Red Hat Automation Hub, a private Automation Hub or a Git repository, and list them in requirements.yml with version constraints. In playbooks you call content by its fully qualified collection name, or declare a collections search path. To publish, you create the skeleton with ansible-galaxy collection init, write a galaxy.yml with the namespace, name, version and dependencies, build with ansible-galaxy collection build, and publish the tarball with an API token. Teams often host internal collections in a private registry to share company roles, modules and filter plugins. I use semantic versioning, changelogs, sanity tests with ansible-test, and CI to build and release, and I pin consumers to a version range to avoid surprise breakages.

Likely follow-ups

- What goes in galaxy.yml?
- How do you host a private collection?

A collection bundles modules, plugins and roles under a namespace and version. Install from Galaxy or a private hub via requirements.yml, call by fully qualified name, and publish with galaxy.yml, ansible-galaxy build and publish, with ansible-test in CI.

```
ansible-galaxy collection init myorg.platform
```

```
ansible-galaxy collection build && ansible-galaxy collection publish myorg-platform-1.2.0.tar.gz
```

Interview tip: Say semantic versions and a pinned requirements file.

📚 **Official docs**

- Ansible: Roles
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q24 How do you use Galaxy and requirements.yml to manage dependencies reproducibly?

Roles & collections

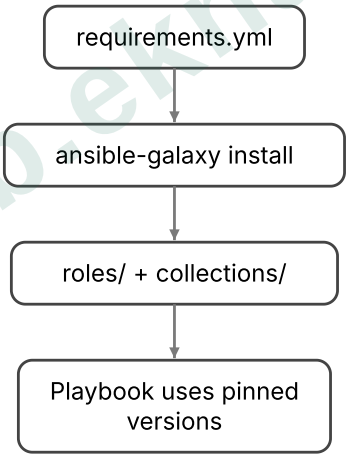
🎙 Spoken answer · ~63 s

The requirements.yml file lists the roles and collections a project needs, with sources and versions, so anyone can install exactly the same content. For collections you give a name and a version constraint, and optionally a source or server, and for roles you can reference Galaxy names or Git URLs with a tag or commit. Installing is one command, and I set the collections and roles paths in ansible.cfg to a project-local directory, so nothing depends on what happens to be installed globally. For reproducibility I pin exact versions or narrow ranges, avoid tracking a moving branch in production, and use a private Automation Hub or mirror so builds do not depend on public availability. In CI, I install from the file in a clean environment before linting and testing. For enterprise setups, the same requirements file feeds ansible-builder to create an execution environment image, so the controller uses identical content.

Likely follow-ups

- Why avoid pinning to a branch?
- How does requirements.yml feed an execution environment?

🗺 Reproducible dependencies



requirements.yml pins the roles and collections a project needs. Install into a project-local path, pin versions or commits, use a private mirror, and reuse the file to build execution environment images so every environment matches.

```
ansible-galaxy install -r requirements.yml -p ./roles
```

```
collections:
  - name: kubernetes.core
    version: ">=3.0,<4.0"
```

Interview tip: Say pin versions and mirror privately for reproducibility.

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible: Roles
- Molecule documentation

Q25 When should logic be a role, a collection or a custom module?

Roles & collections

🎙 **Spoken answer · ~63 s**

I choose by what the reuse looks like. A role is the right unit to reuse a sequence of tasks that configures something, such as installing and configuring a service. A collection is the packaging and distribution layer: when I have several related roles, plugins or modules, or content shared across teams, I put them in a collection with versioning. A custom module is appropriate when the work needs real programming logic or an API that existing modules do not cover, or when a complex task in YAML becomes unreadable and not idempotent. Modules written in Python give structured arguments, check mode support, proper change detection and documentation, while a long chain of shell commands does not. Before writing anything new, I search existing collections, because maintained modules beat custom code. If I do write one, I test with ansible-test and unit tests and keep it small and well documented.

Likely follow-ups

- What makes a custom module better than a shell chain?
- What does a module need to support check mode?

Use a role for reusable task sequences, a collection to package and version related content, and a custom module when you need real logic, an uncovered API, or proper idempotency and check mode. Search existing collections first.

```
ansible-doc -l | grep -i grafana
```

```
ansible-test sanity --docker default
```

Interview tip: Say look for an existing module before writing one.

📚 **Official docs**

- Ansible: Roles
- Ansible: Developing modules
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q26 Why use fully qualified collection names, and how do you migrate old playbooks?

Roles & collections

🎙 **Spoken answer · ~63 s**

Fully qualified collection names, such as ansible.builtin.copy or amazon.aws.ec2_instance, remove ambiguity about which module runs. Without them, Ansible searches namespaces, and a collection added later could shadow a module of the same short name, changing behaviour silently. They also make it obvious where documentation and fixes live and which collection must be installed. Migration is mostly mechanical: ansible-lint has a fqcn rule that detects short names and can auto-fix many of them, and tools like the ansible-core migration notes list the mapping of old module names to collections. I update playbooks and roles, add the needed collections to requirements.yml with versions, run lint and Molecule, and test on a staging inventory. The effort is worth it because pinned, qualified content makes runs reproducible and upgrades of ansible-core much less risky. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- What can happen without FQCN when a new collection is installed?
- How does ansible-lint help migrate?

FQCNs such as ansible.builtin.copy remove ambiguity and shadowing risk and show which collection is required. Migrate with ansible-lint fqcn rule, add collections to requirements.yml and test in staging.

```
ansible-lint --fix=fqcn site.yml
```

```
ansible-doc ansible.builtin.service
```

Interview tip: Say lint can auto-fix most short names.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Roles
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q27 How do you write a custom Ansible module, and what makes it good?

Roles & collections

🎙 **Spoken answer · ~63 s**

A custom module is usually a Python file placed in a library directory next to a playbook or role, or inside a collection. It uses the AnsibleModule helper to declare an argument specification with types, required flags, choices and defaults, reads the parameters, does the work, and exits with exit_json including changed and any result data, or fail_json with a message. A good module is idempotent: it checks the current state, compares to the desired state and changes only if needed. It supports check mode by declaring that and not applying changes, it never prints secrets, marks sensitive parameters with no_log, handles errors with clear messages, and includes documentation in the DOCUMENTATION, EXAMPLES and RETURN strings so ansible-doc works. I test with ansible-test sanity and units, and mock external calls. Module code runs on the target, so dependencies must be available there, otherwise consider an action plugin or a lookup running locally.

Likely follow-ups

- How do you support check mode?
- How do you hide secret parameters?

Write a Python file using AnsibleModule with an argument spec, compute current versus desired state, return exit_json with changed, support check mode, use no_log for secrets, and document with DOCUMENTATION, EXAMPLES and RETURN. Test with ansible-test.

```
module = AnsibleModule(argument_spec=dict(name=dict(type='str', required=True)),
supports_check_mode=True)
```

```
module.exit_json(changed=changed, msg='ok')
```

Interview tip: Say idempotent, check-mode aware and documented.

📚 **Official docs**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q28 What is the difference between role defaults and role vars, and how should you design role variables?

Roles & collections

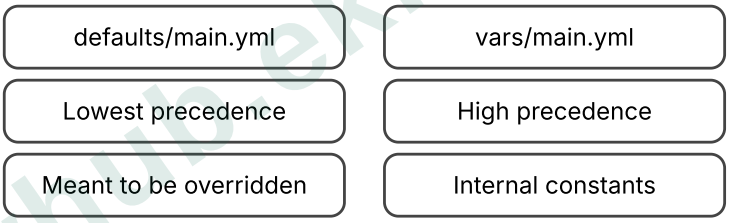
🎙 Spoken answer · ~67 s

Role defaults, in defaults main, have the lowest precedence of all variables, so anything else can override them, inventory, play vars or the command line. They are meant for the public interface of the role: the knobs that users are expected to change, with sensible values. Role vars, in vars main, have high precedence and are hard to override, so they suit internal constants such as package names per operating system, which the user should not change. A good design puts every configurable setting in defaults, documents each in the README, prefixes names with the role name, such as nginx_worker_processes, and uses vars only for internal data or for values loaded per platform with include_vars. I avoid using vars for things users may need to override, because it frustrates them and leads to extra vars hacks. I also validate inputs with the assert module or an argument specification in the role meta, so mistakes fail early with a clear message.

Likely follow-ups

- What is an argument spec in role meta?
- Why is overriding a role var difficult?

🗺 defaults vs vars



Defaults are lowest precedence and form the role's public, overridable interface. Vars are high precedence internals such as per-OS package names. Prefix names, document defaults and validate inputs with assert or an argument spec.

```
# roles/nginx/defaults/main.yml
nginx_worker_processes: auto
nginx_port: 80
```

```
- ansible.builtin.assert:
    that: nginx_port | int > 0
```

Interview tip: Say defaults are the API, vars are the internals.

📚 Official docs

- Ansible: Roles
- Ansible: Variables and precedence

InterviewHub · interviewhub.eknathalabs.com

Q29 How do you write custom filter and lookup plugins, and when are they worthwhile?

Roles & collections

🎙 **Spoken answer · ~61 s**

Plugins extend what you can do in expressions and data lookups. A filter plugin is a Python file in a filter_plugins directory, or inside a collection, with a FilterModule class that returns a dictionary mapping filter names to functions, which you then use in Jinja2 like any built-in filter. A lookup plugin is a LookupModule class with a run method that receives terms and returns a list, running on the control node, so it can read local files, call APIs or query a secrets store. They are worthwhile when the same transformation or data fetch appears in many places and doing it in Jinja2 makes templates unreadable, for example normalising a CIDR list, parsing a vendor format, or retrieving a secret. Keep them small, pure where possible, documented and unit tested. Remember lookups run on the controller, not on the target, which affects file paths and credentials.

Likely follow-ups

- Where do lookup plugins execute?
- Why keep filters pure?

A filter plugin is a Python class mapping names to functions used in Jinja2; a lookup plugin returns data from the control node. Use them to replace repeated or unreadable template logic, keep them small, tested and documented.

```
class FilterModule(object):
    def filters(self):
        return {'cidr_hosts': cidr_hosts}
```

```
{{ lookup('ansible.builtin.env', 'HOME') }}
```

Interview tip: Say lookups run on the controller, not on the target.

📚 **Official docs**

- Ansible: Templating (Jinja2)
- Ansible: Developing modules

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q30 What are the ways to apply a role, and how do roles, import_role and include_role differ?

Roles & collections

🎙 **Spoken answer · ~62 s**

There are three ways. The roles keyword in a play applies roles statically before the play's tasks, in order, after pre_tasks. The import_role task statement is also static but lets you place the role anywhere among tasks, with tags and conditions applied to inner tasks at parse time. The include_role task statement is dynamic: it is evaluated when execution reaches it, so you can loop over it, pass variables computed at run time, or choose the role name from a variable, but tags need apply and the role contents are not visible in list-tasks. I use the roles keyword for the common simple case, import_role when I need ordering mixed with tasks, and include_role when the decision depends on facts or loops. I also use public true on include_role if I need its variables exposed to later tasks, since by default a dynamically included role's variables stay private.

Likely follow-ups

- When would you loop over include_role?
- What does public true do?

The roles keyword applies roles statically before tasks; import_role is static anywhere in tasks; include_role is dynamic with loops and runtime names. Use public true to expose its variables.

```
- ansible.builtin.include_role:
    name: "{{ item }}"
  loop: "{{ enabled_roles }}"
```

```
- hosts: web
  roles: [common, nginx]
```

Interview tip: Say choose static for predictability, dynamic for runtime decisions.

📚 **Official docs**

- Ansible: Roles

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q31 Explain Ansible variable precedence and how to avoid surprises.

Variables & Jinja2

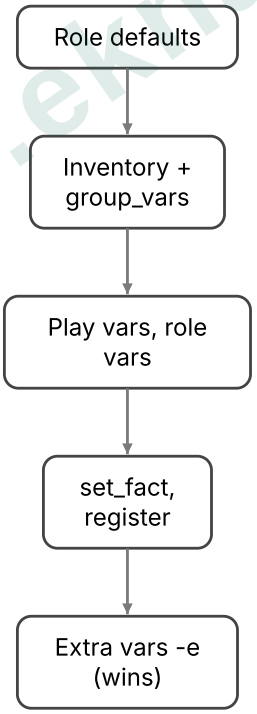
🎙 Spoken answer · ~65 s

Ansible merges variables from many sources, and a defined order decides which wins. From lowest to highest, roughly: role defaults, inventory group variables, then host variables, playbook group_vars and host_vars, facts and registered variables, play vars and vars files, role vars, block and task vars, include parameters, set_fact, and finally extra vars passed with the e option, which beat everything. The documentation lists about twenty-two levels, and the key mental model is that more specific and more explicit sources win. To avoid surprises I keep variables in few places: role defaults for knobs, group_vars and host_vars for environment data, and extra vars only for exceptional overrides or CI parameters. I prefix names with the role, avoid reusing names across layers, and avoid set_fact for configuration because it overrides most things. When debugging, the debug module with var set to the name, or the vars magic variable and the ansible-inventory host output, shows the final value.

Likely follow-ups

- Why do extra vars beat everything?
- How do you find where a variable value came from?

🗺 Precedence (low to high)



Variables merge from defaults, inventory, group and host vars, facts, play vars, role vars, task vars, set_fact and extra vars, with more specific and explicit sources winning. Keep variables in few places, prefix names and debug with ansible-inventory and the debug module.

```
ansible-inventory -i inventory --host web01
```

InterviewHub - interviewhub.eknathalabs.com

```
ansible web01 -m debug -a 'var=nginx_port' -i inventory
```

Interview tip: Say defaults for knobs, group_vars for environments, extra vars for exceptions.

📚 **Official docs**

- Terraform: Input variables (precedence)
- Ansible: Variables and precedence
- Ansible documentation

Q32 How do you organise group_vars and host_vars for multiple environments?

Variables & Jinja2

🎙 **Spoken answer · ~65 s**

I lay out the inventory so environments are separate and shared data is not duplicated. A common structure is one inventory directory per environment, such as production and staging, each with its own hosts file or dynamic source and its own group_vars and host_vars directories, plus an all group file for global values. Within group_vars I create a directory per group containing several small files, for example vars.yml for ordinary values and vault.yml for encrypted ones, which keeps secrets separate and diffs readable. Roles hold defaults, groups such as webservers and databases hold role-specific settings, and environment directories hold environment-specific settings like endpoints and sizes. I avoid duplicating the same value across environments by placing it in a shared group, and I avoid very deep group hierarchies since precedence between sibling groups at the same level is by alphabetical order, which is fragile. A prefix convention helps with the vault variables and their plain references.

Likely follow-ups

- Why split vars.yml and vault.yml?
- What decides precedence between two groups at the same depth?

Use one inventory directory per environment with group_vars and host_vars, a group directory with vars.yml and vault.yml, shared values in an all group, and avoid deep hierarchies where precedence is fragile.

```
inventories/
  production/
    hosts.yml
    group_vars/all/vars.yml
    group_vars/db/vault.yml
  staging/
```

```
ansible-playbook -i inventories/staging site.yml
```

Interview tip: Say separate plain and vault files per group.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible: Inventory

Q33 How do Jinja2 templates work in Ansible and what are good practices? Variables & Jinja2

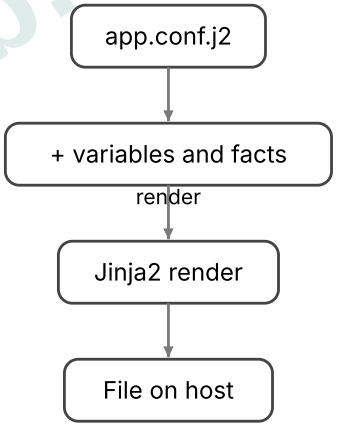
🎙 **Spoken answer · ~63 s**

The template module renders a Jinja2 file on the control node using the host's variables and copies the result to the target. Jinja2 provides expressions in double curly braces, statements such as for loops and if conditions in curly percent tags, filters, tests and comments. Templates are used to generate configuration files such as nginx, systemd units or application properties. Good practices: keep logic simple in templates and compute complex data in tasks or filters; add a managed-by header with the ansible_managed variable so people know not to edit by hand; use validate to check syntax with the service's own tool before replacing the file; set owner, group and mode; use trim_blocks and lstrip_blocks to control whitespace; use the default filter and the mandatory filter to handle missing values deliberately; and never put secrets in templated files that are logged or diffed, using no_log. Test rendering with check and diff.

Likely follow-ups

- What does the validate parameter do?
- How do you control whitespace in templates?

🗺 **Template rendering**



The template module renders Jinja2 on the controller with host variables. Keep logic simple, add an ansible_managed header, use validate before replacing, set mode and owner, use defaults and mandatory deliberately, and avoid exposing secrets.

```
- ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: '0644'
    validate: 'nginx -t -c %s'
```

certs.eknathalabs.com

InterviewHub - interviewhub.ekn

Interview tip: Say validate and ansible_managed.

📚 Official docs

- Ansible: Templating (Jinja2)
- Ansible documentation

Q34 Which Jinja2 filters do you use most in Ansible, and for what?

Variables & Jinja2

🎙 Spoken answer · ~64 s

A few filters cover most needs. The default filter provides a fallback when a variable is undefined, and with true as a second argument also for empty values. The mandatory filter fails loudly if a value is missing. For data shaping, I use map and select with attribute to extract fields from lists of dictionaries, selectattr and rejectattr to filter, join, split, unique, sort, flatten and zip. The combine filter merges dictionaries, optionally recursively, dict2items and items2dict convert between formats, and json_query uses JMESPath for nested queries. For strings and security, regex_replace and regex_search, b64encode, password_hash and the quote filter for shell safety. For networking, the ansible.utils ipaddr and ipaddr style filters handle addresses and subnets. Type filters like int, bool and list handle strings coming from the command line. I keep expressions short, and when a filter chain becomes long I compute it in a set_fact or a custom filter for readability.

Likely follow-ups

- How do you merge two dictionaries?
- What does selectattr do?

Key filters: default and mandatory, map, select, selectattr, join, unique, combine, dict2items, json_query, regex_replace, quote and type conversions. Keep chains short and move complex logic into set_fact or a custom filter.

```
{{ users | selectattr('active') | map(attribute='name') | list }}
```

```
{{ defaults | combine(overrides, recursive=True) }}
```

Interview tip: Say default, mandatory, combine and selectattr are the workhorses.

📚 Official docs

- Ansible: Templating (Jinja2)
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q35 How do register, set_fact and magic variables like hostvars work together?

Variables & Jinja2

🎙 Spoken answer · ~65 s

The register keyword saves a task's result into a variable, containing fields such as stdout, rc, changed and, for loops, a results list. The set_fact module creates or updates a variable for the current host at run time, which persists for the rest of the play, and can be cached across plays if fact caching is enabled. Magic variables are provided by Ansible: inventory_hostname, group_names, groups, play_hosts, ansible_play_batch, and hostvars, a dictionary giving access to variables and facts of any other host in the inventory. Together they let one host's play use data from others, for example a web server reading the database host's address from hostvars of the first host in the db group. Cautions: hostvars facts exist only if that host has already been contacted in the run, set_fact has high precedence, which can mask configuration, and registered variables from skipped tasks still exist with a skipped flag, so check before using the output.

Likely follow-ups

- When does hostvars not contain facts for another host?
- Why is set_fact risky for configuration?

register captures task output, set_fact defines run-time variables per host, and magic variables like hostvars, groups and inventory_hostname expose inventory and other hosts. Beware hostvars before facts are gathered, set_fact precedence and skipped registered results.

```
- ansible.builtin.command: hostname -f
  register: fqdn
- ansible.builtin.set_fact:
    db_host: "{{ hostvars[groups['db'][0]].ansible_default_ipv4.address }}"
```

Interview tip: Say hostvars needs facts already gathered for that host.

📚 Official docs

- Ansible: Variables and precedence

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q36 What are lookup plugins and how do you use them safely?

Variables & Jinja2

🎙 **Spoken answer · ~68 s**

Lookups retrieve data from external sources during templating and run on the control node, not on the target. Common ones read files, environment variables, URLs, DNS records, CSV or INI files, password generation, and secret stores such as HashiCorp Vault, AWS Secrets Manager and 1Password through collections. They are called with the lookup function, or the query function, which always returns a list. Safe use means remembering the controller location, for example a file lookup reads from the machine running Ansible, not the target. For secrets, I prefer a lookup against a dedicated secrets manager with short-lived credentials over storing values in vars files, and I set no_log on tasks that use them so the value is not printed. Because lookups run at each evaluation of a variable, an expensive one used in many places can slow down a play, so I register the result once or wrap it in set_fact. I also handle missing items with the default or errors ignore options.

Likely follow-ups

- What is the difference between lookup and query?
- Why might a lookup be slow in a template?

Lookups fetch data on the control node from files, env, URLs, DNS or secret stores. Use query for lists, protect output with no_log, and evaluate expensive lookups once with set_fact. Remember paths are controller paths.

```
{{ lookup('ansible.builtin.file', 'files/key.pub') }}
```

```
{{ lookup('community.hashi_vault.hashi_vault', 'secret/data/app:password') }}
```

Interview tip: Say lookups run on the controller and should be no_log when secret.

📚 **Official docs**

- Ansible: Developing modules

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q37 How do you generate configuration from structured data using loops in templates?

Variables & Jinja2

🎙 Spoken answer · ~66 s

I model the data as variables in YAML, for example a list of virtual hosts or a dictionary of upstreams, then render with a for loop in the template. Inside Jinja2 a for loop can iterate lists and dictionary items, with loop.index, loop.first and loop.last helpers for commas or separators, and conditional blocks for optional settings. For cleaner output I use trim_blocks and lstrip_blocks, and I avoid putting complicated logic in the template by preparing data with filters in the task, for example sorting, grouping or merging defaults with overrides using combine. For files that are one resource per file, I loop at the task level with the template module and set the destination from the item, which makes handlers and permissions simpler. Testing is by rendering with check and diff, and by validating the output with the service's own syntax checker. Keeping the data model separate from the template means non-experts can add a host by editing YAML.

Likely follow-ups

- When would you loop in the task instead of the template?
- How do you avoid trailing commas in generated files?

Model data as YAML, loop in Jinja2 with loop.first and loop.last helpers, prepare data with filters or combine, and use task-level loops for one-file-per-item cases. Validate generated files with the service's checker.

```
{% for v in vhosts %}
server_name {{ v.name }};
{% endfor %}
```

```
- ansible.builtin.template:
    src: vhost.j2
    dest: "/etc/nginx/conf.d/{{ item.name }}.conf"
  loop: "{{ vhosts }}"
```

Interview tip: Say separate the data model from the template.

📚 Official docs

- Ansible: Playbook guide
- Ansible: Templating (Jinja2)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q38 How do you merge default and override dictionaries, and what are the pitfalls?

Variables & Jinja2

🎙 **Spoken answer · ~60 s**

The combine filter merges dictionaries, with the right side winning on key conflicts. By default it is shallow, so a nested dictionary on the right replaces the entire nested dictionary on the left. With recursive true it merges nested keys, and the list_merge option controls what happens with lists: replace, append, prepend or keep. A typical pattern is a role default dictionary, such as app_config_defaults, combined with a user-supplied app_config_overrides, into a final value in a task or template. Pitfalls include accidental shallow merges that drop nested defaults, ordering mistakes where the wrong side wins, lists that are replaced rather than extended, and variable precedence, since a dictionary defined at multiple levels is replaced, not merged, unless hash_behaviour merge is enabled, which is discouraged and deprecated. I therefore use explicit combine in code with documented variable names, rather than relying on global merge behaviour.

Likely follow-ups

- Why is hash_behaviour merge discouraged?
- How do you append lists when merging?

Use combine, with recursive true for nested dictionaries and list_merge to control lists. Keep defaults and overrides as separate variables, and avoid global hash_behaviour merge since precedence otherwise replaces whole dictionaries.

```
{{ app_defaults | combine(app_overrides | default({}), recursive=True, list_merge='append')
}}
```

Interview tip: Say variables replace by default, so combine explicitly.

📚 **Start here (official docs)**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q39 What is Ansible Vault and how do you use it?

Vault & security

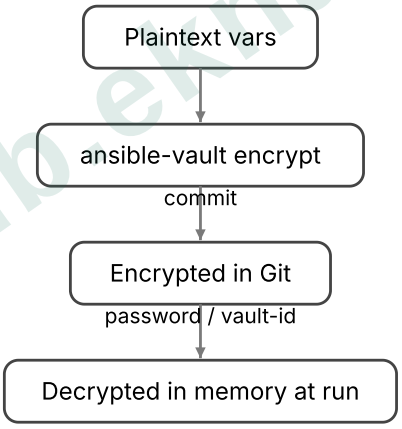
🎙 Spoken answer · ~64 s

Ansible Vault encrypts sensitive data, such as passwords, keys and tokens, so it can be stored in version control. You can encrypt a whole file with ansible-vault encrypt, create one with create, edit with edit, and view with view. Alternatively, you can encrypt a single value with encrypt_string, which produces a vault tag inline in a YAML file, so the rest of the file stays readable in diffs. Playbooks decrypt transparently at run time when you provide a password through a prompt, a file, or a script with the vault-password-file option or the ANSIBLE_VAULT_PASSWORD_FILE variable. It uses AES-256 with a key derived from the password. Practices: keep encrypted variables in a separate vault.yml and reference them from plain variables named with a vault prefix so people can grep names; never commit the password file; rotate with rekey; and for larger organisations use an external secrets manager so that rotation and auditing are handled centrally.

Likely follow-ups

- Whole-file versus inline encryption: which do you prefer?
- How do you rotate the vault password?

🗺 Vault lifecycle



Vault encrypts files or single strings with AES-256 so secrets can live in git. Supply the password through a file or script, keep encrypted variables in a separate vault.yml referenced by plain names, rekey to rotate, and consider an external secrets manager at scale.

```
ansible-vault encrypt group_vars/prod/vault.yml
```

```
ansible-vault encrypt_string 's3cr3t' --name 'db_password'
```

```
ansible-playbook site.yml --vault-password-file ~/.vault_pass
```

Interview tip: Say never commit the password file.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible Vault
- Ansible documentation
- HashiCorp Vault documentation

Q40 How do vault IDs work and how do you use multiple vault passwords?

Vault & security

🎙 **Spoken answer · ~66 s**

Vault IDs label encrypted content so you can use different passwords for different environments or teams, for example dev, prod and shared. When encrypting, you pass vault-id with a label and a source, such as prod at a prompt, a file, or a script. The label is stored in the vault header. At run time you can supply several vault-id options, each with its own source, and Ansible tries the matching label first, then others. This allows developers to hold the dev password while only the pipeline or a few engineers hold the production one, so a developer can run against staging without being able to decrypt production secrets. A script as the source can fetch the password from a secrets manager or a keychain, which avoids files on disk. Pitfalls: the label is a hint, not a security boundary by itself, so access control to the actual passwords is what matters, and rotating requires rekeying each ID.

Likely follow-ups

- What does the label in the header do?
- How would a script supply the password?

Vault IDs label encrypted content, so different environments use different passwords. Provide several vault-id sources at run time; Ansible matches the label. Use scripts to fetch passwords from a secrets manager and limit who holds the production one.

```
ansible-vault encrypt --vault-id prod@prompt group_vars/prod/vault.yml
```

```
ansible-playbook site.yml --vault-id dev@~/.vault_dev --vault-id
prod@scripts/get_prod_pass.sh
```

Interview tip: Say the label is not a security boundary; password access is.

📚 **Official docs**

- Ansible Vault
- HashiCorp Vault documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q41 How do you integrate Ansible with HashiCorp Vault or cloud secret managers instead of using Ansible Vault?

Vault & security

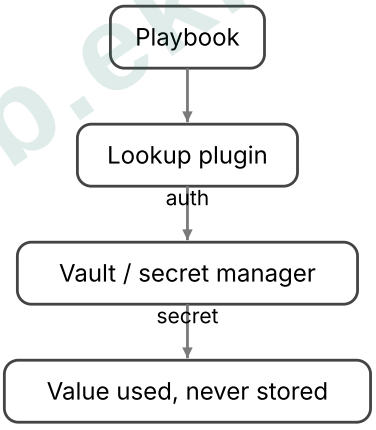
🎙 Spoken answer · ~62 s

For larger environments, a dedicated secrets manager gives central access control, audit, rotation and dynamic short-lived credentials, which file-based Vault cannot. Ansible integrates through lookup plugins, for instance the hashi_vault lookup in the community.hashi_vault collection, the aws_secret lookup in amazon.aws, or equivalents for Azure Key Vault and GCP Secret Manager. The control node, or the AWX execution environment, authenticates with a short-lived method: AppRole, Kubernetes auth, OIDC or cloud instance identity, rather than a static token. Lookups fetch values at run time, and tasks that use them set no_log true. In AWX or Ansible Automation Platform, credential types can inject secrets from an external credential source so job templates never store the secret. Considerations: the lookup runs on the controller, so network access to the secrets manager is needed there, failures should stop the run clearly, and values should not be cached in facts or written to logs.

Likely follow-ups

- Why prefer short-lived authentication methods?
- Where does a lookup run in AWX?

🗺 External secrets



Use lookup plugins for HashiCorp Vault, AWS, Azure or GCP secret managers, authenticate with short-lived identities such as AppRole, OIDC or instance identity, set no_log, and in AWX use external credential sources so secrets are not stored in job templates.

```
{{ lookup('amazon.aws.aws_secret', 'prod/db/password', region='us-east-1') }}
```

```
vault write auth/approle/login role_id=$ROLE secret_id=$SECRET
```

Interview tip: Say no static tokens and no_log on the task.

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible Vault
- Ansible: Cloud guides
- Ansible documentation

Q42 What does no_log do, and how else do you keep secrets out of logs and output?

Vault & security

🎙 **Spoken answer · ~66 s**

Setting no_log true on a task or play hides its arguments and results from output, logs and callbacks, replacing them with a notice, which prevents passwords and tokens from appearing in the console, in AWX job output or in CI logs. It also hides useful debugging information, so I enable it only on tasks that handle secrets, and keep it off in a debug build with a variable. Other protections: mark module parameters as no_log in custom modules, avoid passing secrets on command lines where they show in process lists, use environment variables or files with strict permissions, avoid the debug module on secret variables, be careful with verbose flags since they can reveal data, and exclude secrets from check and diff output with diff false on tasks that template secret files. Callback plugins and external logging should be reviewed, and CI systems should mask variables. Also remember that registered results can contain secrets, so avoid printing them.

Likely follow-ups

- How can a secret leak through diff mode?
- Why avoid secrets on command lines?

no_log hides task arguments and results from output and logs. Use it only where needed, avoid command-line secrets, disable diff on secret templates, mask CI variables and avoid printing registered results.

```
- ansible.builtin.uri:
    url: https://api/login
    body: {password: "{{ api_pass }}"}
  no_log: true
```

```
- ansible.builtin.template:
    src: secrets.j2
    dest: /etc/app/secrets
    diff: false
```

Interview tip: Say no_log hides debugging too, so scope it.

📚 **Official docs**

- Ansible Vault

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q43 How do you secure SSH access and key management for Ansible?

Vault & security

🎙 **Spoken answer · ~62 s**

Ansible's security is largely SSH security. I use a dedicated automation account on the targets, key-based authentication only, with passwords disabled, and a key per environment or per controller rather than a shared personal key. Keys are held in an agent or a secrets manager, not copied around, and protected with passphrases where a human is involved. On targets, the authorized key can be restricted with options such as from, command or no-port-forwarding when feasible, and sudo is limited through a sudoers file to what is required. Better still, use short-lived SSH certificates issued by a certificate authority such as Vault's SSH secrets engine, so there are no long-lived keys to rotate. Host key checking stays enabled, with known hosts populated by trusted means, not disabled globally, which would allow man-in-the-middle attacks. Access to private networks goes through a hardened bastion with ProxyJump or ProxyCommand and audit logging.

Likely follow-ups

- Why not disable host key checking?
- What is an SSH certificate and why use one?

Use a dedicated automation account with key-only auth, per-environment keys or short-lived SSH certificates, restricted sudo, host key checking enabled and a hardened, audited bastion for private networks.

```
ansible_ssh_common_args='-o ProxyJump=bastion'
```

```
ssh-keygen -s ca_key -I ansible-ctl -n ansible -V +1h id_ed25519.pub
```

Interview tip: Say short-lived SSH certificates beat long-lived keys.

📚 **Official docs**

- OpenSSH manual pages
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q44 How do you use Ansible for security hardening and compliance, such as CIS benchmarks?

Vault & security

🎙 **Spoken answer · ~63 s**

Ansible suits hardening because it applies and verifies a desired baseline consistently. I start from maintained content, such as the community dev-sec hardening collection or the CIS and DISA STIG roles, and adapt them rather than writing every control by hand. Controls cover SSH configuration, password and account policy, file permissions, kernel parameters, services disabled, auditd and logging, firewall rules and package updates. Each control gets a tag and a variable to enable or exempt it, with documented justifications for exceptions. I run in check mode with diff for an assessment, then in apply mode in stages, starting with staging and a canary group, since some hardening, like restricting SSH or mounting options, can lock people out or break applications. After applying, scanners such as OpenSCAP or InSpec verify compliance and produce reports for auditors. Results are scheduled to detect drift, with the playbook run from AWX on a timer.

Likely follow-ups

- How do you avoid locking yourself out?
- How do you prove compliance to auditors?

Start from maintained hardening roles, make controls tagged and toggleable with documented exceptions, assess in check mode, apply in stages with a canary group, verify with OpenSCAP or InSpec, and schedule runs to catch drift.

```
ansible-galaxy collection install devsec.hardening
```

```
ansible-playbook harden.yml --check --diff -l canary
```

Interview tip: Say staged rollout, verification scan and exceptions with reasons.

📚 **Official docs**

- kube-bench (CIS Kubernetes benchmark)
- CIS Benchmarks
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q45 How do you design privilege escalation and sudo policy for automation accounts?

Vault & security

🎙 **Spoken answer · ~65 s**

The aim is that automation can do what it needs and nothing more. I create a dedicated automation user per purpose and per environment, with key-based login, and grant it sudo through a file in the sudoers directory, validated with visudo. The broadest option is passwordless sudo for all commands, which is convenient but means that compromise of the controller equals root everywhere. A tighter policy limits sudo to specific commands, or uses separate accounts for specific roles, for example an application deployment account that may restart one service. Where full root is required, I protect the controller strongly: restricted access, audit logging, short-lived credentials and network segmentation. In playbooks I apply become only to the tasks that need it, and use become_user with an application account for file ownership. Sudo logging and session auditing record what was run. I review these policies regularly and test that playbooks still work under the restricted policy in staging.

Likely follow-ups

- What is the risk of passwordless sudo ALL?
- How do you validate a sudoers file safely?

Use dedicated automation accounts with key login and sudoers entries validated by visudo. Prefer command-limited sudo where practical, protect the controller strongly if root is needed, apply become only to needed tasks and audit sudo activity.

```
- ansible.builtin.copy:
    dest: /etc/sudoers.d/ansible
    content: "ansible ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart app\n"
    validate: 'visudo -cf %s'
    mode: '0440'
```

Interview tip: Say the controller becomes the crown jewel if sudo is broad.

📚 **Official docs**

- Kubernetes: RBAC good practices
- sudoers(5) manual

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q46 How do you handle secrets in CI/CD pipelines that run Ansible?

Vault & security

🎙 **Spoken answer · ~63 s**

The pipeline needs secrets, but they should be short-lived, scoped and never printed. I store the vault password or the cloud credentials in the CI system's secret store, expose them only to protected branches and environments, and have the job write the vault password to a temporary file with strict permissions or pass it through a script, removing it afterwards. Better, the job authenticates to a secrets manager with an OIDC identity from the CI provider, retrieving short-lived credentials at run time, which avoids stored long-lived secrets. SSH keys are loaded into an agent for the job's duration only. Logs are protected with masking and no_log, and verbosity is not enabled on production runs. For pull requests from forks, secrets are not exposed. Production runs need approvals, and the same pipeline uses a separate credential set per environment. Finally, audit logs show who triggered what, and credentials are rotated on a schedule.

Likely follow-ups

- How does OIDC avoid stored secrets in CI?
- Why not expose secrets to fork pull requests?

Keep secrets in the CI secret store scoped to protected branches, prefer OIDC to fetch short-lived credentials, write vault passwords to temporary files and remove them, load SSH keys into an agent only per job, mask logs and require approval for production.

```
echo "$VAULT_PASS" > .vp && chmod 600 .vp && ansible-playbook site.yml --vault-password-file .vp; rm -f .vp
```

```
ssh-add - <<< "$DEPLOY_KEY"
```

Interview tip: Say OIDC short-lived credentials beat stored secrets.

📚 **Official docs**

- GitHub Actions: Security hardening
- Ansible Vault
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q47 How do you audit and review Ansible code for security problems?

Vault & security

🎙 **Spoken answer · ~62 s**

I treat playbooks as code that can change production, so they get review and automated checks. ansible-lint runs in CI with a profile such as production, catching risky patterns: use of shell where a module exists, missing no_log, world-writable file modes, plain-text passwords, unpinned package versions and missing FQCNs. Secret scanning tools like gitleaks or trufflehog check commits for credentials. Code review focuses on privilege, on tasks that download and execute remote content, disabled certificate validation, disabled host key checking, and templates that embed secrets. Branch protection requires approval for production playbooks, with separate reviewers for sensitive roles. At run time, AWX or the platform provides audit logs of who ran which job template with which inputs, and I forward them to the central logging system. Dependencies are scanned too, as roles and collections from public sources can be compromised, so I pin versions and mirror trusted content.

Likely follow-ups

- Which lint rules catch security problems?
- How do you protect against malicious Galaxy content?

Run ansible-lint with a strict profile and secret scanning in CI, review privilege, downloads, disabled validation and secrets, protect production branches, keep AWX audit logs centrally, and pin and mirror third-party roles and collections.

```
ansible-lint --profile production
```

```
gitleaks detect --source .
```

Interview tip: Say pin and mirror third-party content.

📚 **Official docs**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q48 How do extra vars, vars_files and include_vars differ, and when do you use each?

Variables & Jinja2

🎙 Spoken answer · ~67 s

Extra vars, passed with the e option on the command line or from a file, have the highest precedence and override everything else, so I use them for run-specific parameters from a pipeline or an operator, such as a version to deploy or a target environment, and never for ordinary configuration. A vars_files entry in a play loads variables from listed files when the play starts, handy for a fixed set of environment data. The include_vars module loads a file or directory dynamically during execution, so the file name can depend on facts, for example an OS-specific variables file per distribution, with the loaded values taking task-level precedence. I use include_vars with first_found to select a file such as the OS family and falling back to a default. Care points: extra vars arrive as strings, so booleans and numbers need type conversion; large extra vars from untrusted users can be dangerous, so I restrict them in AWX with surveys and validation.

Likely follow-ups

- Why do extra vars arrive as strings?
- How would you load OS-specific variables?

Extra vars have the highest precedence and suit run-time parameters; vars_files loads fixed files at play start; include_vars loads dynamically, for example per OS with first_found. Convert types and restrict inputs in AWX surveys.

```
ansible-playbook deploy.yml -e version=1.4.2 -e @env/prod.yml
```

```
- ansible.builtin.include_vars: "{{ lookup('ansible.builtin.first_found', ['{{
ansible_distribution }}.yml','default.yml']) }}"
```

Interview tip: Say extra vars for parameters, not for configuration.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q49 How do you validate inputs and variables so a playbook fails early with clear messages?

Variables & Jinja2

🎙 **Spoken answer · ~63 s**

Failing early saves time and prevents half-applied changes. Within a play I use the assert module at the start with conditions and custom fail messages, for example checking that a required variable is defined, a port is in range, a version string matches a pattern, or a list is not empty. The mandatory filter and the default filter used carefully also catch missing values. For roles, the argument_specs file in the role meta directory defines types, required flags, choices and defaults for each entry point, and Ansible validates inputs before the role runs, producing a clear error and doubling as documentation. At the boundary with users, AWX surveys can restrict choices and types. I also run pre-flight checks for connectivity, disk space and package repository access in pre_tasks, so problems are found before changes start. This makes failures cheap, predictable and easy to understand for the person who ran the job.

Likely follow-ups

- What does argument_specs give you?
- What belongs in a pre-flight check?

Use assert with clear messages, the mandatory filter, role argument_specs for typed validation, AWX surveys and pre-flight tasks for connectivity and capacity, so a bad input fails before any change.

```
- ansible.builtin.assert:
    that:
      - app_version is match('^[0-9]+\\.[0-9]+\\.[0-9]+$')
      - app_port | int in range(1024, 65536)
    fail_msg: 'Invalid app_version or app_port'
```

Interview tip: Say fail early with a human-readable message.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Variables and precedence
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q50 How do you rotate secrets and the vault password with minimal disruption?

Vault & security

🎙 Spoken answer · ~65 s

Rotation should be routine, not an emergency. For Ansible Vault content, the rekey command re-encrypts files with a new password, and with vault IDs you can rotate one environment at a time. The process is: generate the new password in the secrets manager, rekey all encrypted files in a branch, update the pipeline and the controller's secret to the new value, merge, and then revoke the old one, ideally keeping a short overlap during which both work. For application secrets stored in vault files, rotation means changing the secret at the source, such as the database user, and then updating the encrypted value and redeploying, which can be automated with a playbook that updates both ends in a safe order, avoiding downtime by supporting two credentials during the transition. Moving the application secrets to an external secrets manager with dynamic credentials reduces the manual work. I record rotation dates and test the process regularly, including recovery.

Likely follow-ups

- How do you rekey many files at once?
- How do you avoid downtime during a database password change?

Rekey vault files with ansible-vault rekey, rotate per vault ID, update pipeline secrets and revoke the old password after an overlap. For application secrets, rotate at the source, support two credentials during transition and consider a secrets manager with dynamic credentials.

```
ansible-vault rekey --vault-id prod@old_pass --new-vault-id prod@new_pass
group_vars/prod/vault.yml
```

```
grep -rl '$ANSIBLE_VAULT' . | xargs ansible-vault rekey --new-vault-password-file new_pass
```

Interview tip: Say plan an overlap, then revoke the old secret.

📚 Official docs

- Vault: Dynamic database secrets
- Ansible Vault
- HashiCorp Vault documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q51 How do forks affect performance, and how do you tune them?

Scale & performance

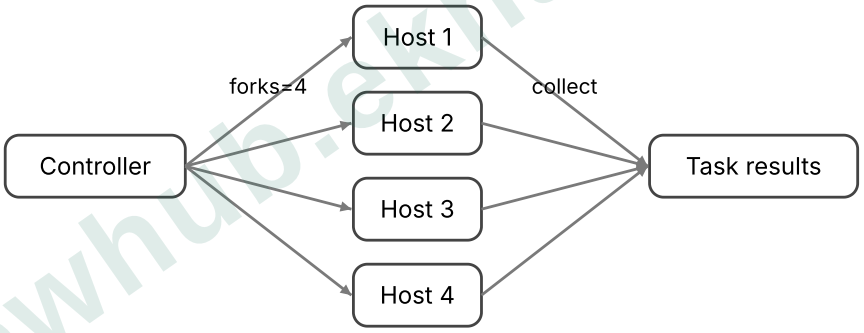
🎙 Spoken answer · ~66 s

The forks setting controls how many hosts Ansible works on in parallel for each task, defaulting to five, which is conservative. Raising it speeds up runs across large inventories, since more hosts are contacted at once, but it consumes more memory and CPU on the control node and more simultaneous SSH connections, so there is a limit set by the controller's resources and by the targets and any jump hosts or bastions, which may have connection limits. I tune by measuring: increase forks in steps, such as 20, 50 and 100, watch controller load and memory, and the task timings, and stop where improvement flattens. Forks apply per task across the current batch, so with the serial keyword the effective parallelism is limited to the batch size. Other settings interact: SSH pipelining, ControlPersist, fact gathering, and the strategy. For very large fleets, I also distribute the work across several controllers or AWX execution nodes instead of one big machine.

Likely follow-ups

- What limits forks in practice?
- How does serial interact with forks?

🗺 Forks



Forks sets parallel hosts per task, default five. Raise it stepwise while watching controller CPU, memory and bastion limits, remember serial caps batch size, and for very large fleets spread work across multiple execution nodes.

```
ansible-playbook site.yml -f 50
```

```
[defaults]
forks = 50
```

Interview tip: Say measure, increase gradually and watch the controller.

📚 Official docs

- Ansible: Strategies and performance

certs.eknathalabs.com

Q52 What are the linear, free and other strategies, and when do you use them?

Scale & performance

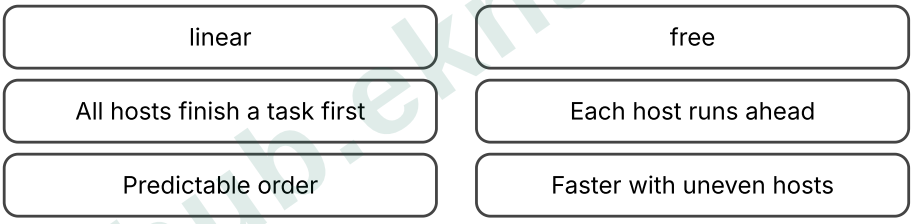
🎙 Spoken answer · ~63 s

A strategy decides how tasks are scheduled across hosts. The default linear strategy runs each task on all hosts in the batch and waits for the slowest before starting the next task, which keeps hosts in lockstep and is predictable. The free strategy lets each host proceed through the tasks as fast as it can without waiting, which shortens total time when hosts differ in speed, but output is interleaved and ordering assumptions between hosts, such as one host depending on another having finished a step, break. The host_pinned strategy is similar to free but keeps a worker on a host. The debug strategy drops into an interactive debugger on failure. I use linear for coordinated rollouts and anything with cross-host dependencies, and free for independent work such as patching or package installation across many unrelated servers. The strategy is set per play, and I combine it with serial for rolling control.

Likely follow-ups

- Why can free break dependencies between hosts?
- What does the debug strategy do?

🗺 linear vs free



Linear keeps hosts in lockstep and is predictable; free lets each host run ahead for speed when tasks are independent; host_pinned and debug are variants. Use linear for coordinated changes and free for independent bulk work.

```
- hosts: all
  strategy: free
  tasks: []
```

```
ANSIBLE_STRATEGY=linux ansible-playbook site.yml
```

Interview tip: Say free is faster but loses ordering guarantees.

📚 Start here (official docs)

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q53 How do you do rolling updates with serial, and control failures during them?

Scale & performance

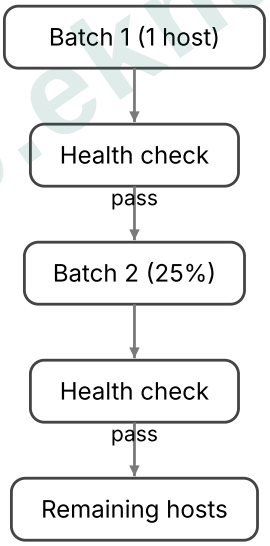
🎙 Spoken answer · ~63 s

The serial keyword limits how many hosts a play works on at a time, so the whole play, all tasks, finishes for one batch before the next batch starts. It can be a number, a percentage or a list that grows, such as 1, then 5, then 20, which gives a canary pattern: one host first, then more. Combined with max_fail_percentage and any_errors_fatal it stops the rollout if too many hosts fail, protecting the remaining healthy servers. Typical flow for a web tier: remove a host from the load balancer with a delegated task, update, run health checks, return it to service, then move to the next batch. I add a wait_for or uri check after each update, so a bad host fails the batch immediately. Note that with serial, run_once applies per batch, and that variable max_fail_percentage applies to each batch. Set batches so capacity remains sufficient while hosts are out.

Likely follow-ups

- What is a growing serial list?
- How do you pick a safe batch size?

🗺 Rolling update



serial limits a play to batches, which gives rolling updates and canary patterns with a growing list. Add max_fail_percentage or any_errors_fatal and health checks so a failing batch stops the rollout; keep enough capacity while batches are out.

```
- hosts: web
  serial: [1, 5, '25%']
  max_fail_percentage: 10
  tasks: []
```

Interview tip: Say canary first batch plus health checks and a failure threshold.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Deployments
- Ansible: Strategies and performance

Q54 How do SSH pipelining, ControlPersist and Mitogen speed up Ansible?

Scale & performance

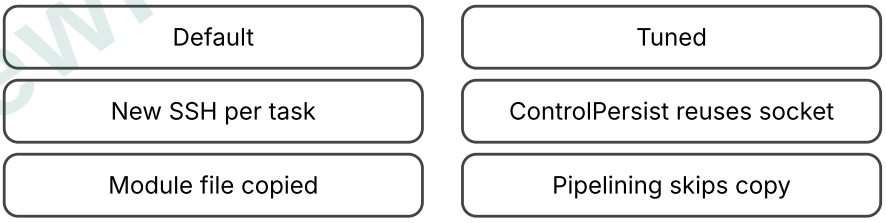
🎙 **Spoken answer · ~64 s**

By default Ansible opens a connection and runs several SSH operations per task, such as creating a temporary directory, copying the module, executing it and cleaning up. Enabling ControlPersist, which is on by default with OpenSSH, keeps a master connection open and reuses it, avoiding repeated handshakes. Enabling pipelining in ansible.cfg sends the module code over the existing SSH session rather than writing a temporary file first, cutting the number of round trips substantially, with one caveat: it requires that requiretty is not set in sudoers on targets. ControlPath length can also cause issues on long hostnames. Mitogen for Ansible is a third-party strategy plugin that replaces the connection and execution layer with a persistent, multiplexed process, which can give large speedups, but it lags behind newer ansible-core releases and is not supported in managed platforms, so it is a decision with trade-offs. I measure with the profile_tasks callback before and after changes.

Likely follow-ups

- What does pipelining require on the target?
- How do you measure which tasks are slow?

🗺 **Connection tuning**



ControlPersist reuses SSH connections and pipelining sends modules over the existing session, cutting round trips, provided requiretty is off. Mitogen can speed things further but has compatibility trade-offs. Measure with profile_tasks.

```
[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

```
ANSIBLE_CALLBACKS_ENABLED=ansible.posix.profile_tasks ansible-playbook site.yml
```

Interview tip: Say measure with profile_tasks before and after.

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

📚 **Official docs**

- Ansible: Strategies and performance
- Ansible documentation

Q55 How do fact caching and limiting fact gathering help at scale?

Scale & performance

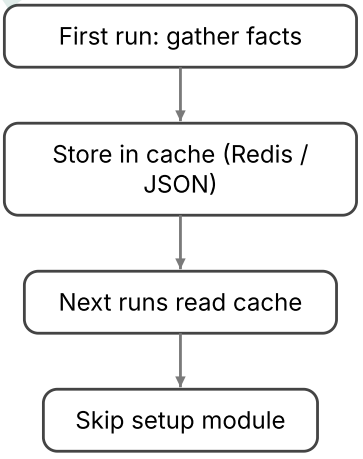
🎙 **Spoken answer · ~63 s**

Gathering facts runs the setup module on every host at the start of each play, which for thousands of hosts and several plays costs a lot of time. I reduce this in layers. First, disable gathering in plays that do not need facts. Second, use gather_subset to collect only needed categories, such as min. Third, enable a fact cache with the smart gathering policy and a backend such as Redis, memcached or JSON files, with a timeout, so facts from a recent run are reused and not recomputed. With caching, other hosts' facts remain available through hostvars even when a play targets only a subset, which helps cross-host templating. The cost is staleness: facts can be old, so the timeout should match how quickly the data changes, and tasks that need fresh values, such as disk usage, should refresh explicitly or avoid the cache. Redis gives shared caching for several controllers.

Likely follow-ups

- What does the smart gathering policy do?
- What data should not come from a cache?

🗺 **Fact caching**



Disable or subset fact gathering, and enable a fact cache such as Redis or JSON with smart gathering and a timeout so recent facts are reused. Watch staleness and refresh volatile facts explicitly.

```
[defaults]
gathering = smart
fact_caching = redis
fact_caching_timeout = 3600
fact_caching_connection = redis01:6379:0
```

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say pick a timeout matching how fast facts change.

📚 **Start here (official docs)**

- Ansible documentation

Q56 How do async and poll work, and when do you use them for long-running tasks?

Scale & performance

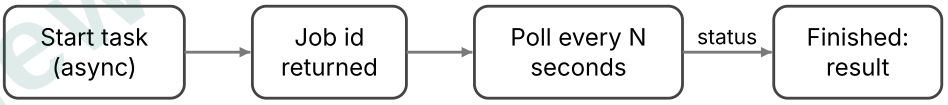
🎙 **Spoken answer · ~69 s**

By default Ansible holds an SSH connection open until a task finishes, which can time out on long operations such as large package upgrades, database dumps or reboots. With async you set a maximum runtime in seconds, and poll sets how often Ansible checks. With a poll interval greater than zero, Ansible still waits but uses a background job and polling, avoiding connection timeouts. With poll set to zero it starts the job and moves on immediately, which lets you fire a task on many hosts, run other work, and later collect the results with the async_status module and the job id from the registered variable. This is useful for parallelising slow tasks, for starting several long jobs across hosts and checking them afterwards. Caveats: you must handle failures when collecting, async tasks cannot use some features like check mode in the usual way, and cleanup of job files may be needed, so I set the retries and delay values in the status check loop.

Likely follow-ups

- How do you collect results of poll zero tasks?
- What problem does async solve for slow tasks?

🗺 **Async task**



async sets a maximum run time and poll sets how often to check, avoiding connection timeouts; poll zero fires and forgets, and async_status collects results later in a retry loop. Handle failures when collecting.

```
- ansible.builtin.command: /usr/local/bin/backup.sh
  async: 3600
  poll: 0
  register: job
```

```
- ansible.builtin.async_status:
    jid: "{{ job.ansible_job_id }}"
  register: r
  until: r.finished
  retries: 60
  delay: 30
```

certs.eknathalabs.com

InterviewHub - interviewhub.ekn

Interview tip: Say poll 0 plus async_status lets tasks overlap.

📚 **Official docs**

- Ansible: Playbook guide

Q57 How do you manage very large or fast-changing inventories?

Scale & performance

🎙 **Spoken answer · ~64 s**

For thousands of hosts, I make the inventory a derived view of a source of truth rather than a hand-edited file. Inventory plugins pull from cloud APIs, VMware, a CMDB or a database, and use caching with a sensible timeout so a run does not hammer the API each time. Keyed groups and filters create groups from tags, regions and roles, and compose maps cloud attributes to Ansible variables such as the connection address. I limit the scope with the limit option or the inventory's own filters, so a run touches only the relevant slice. Group structure is kept shallow and consistent, with environment and role as the main axes. Variables live in group_vars, not per host, to avoid thousands of files. For performance, I use fact caching, higher forks, and, if needed, partitioning the fleet across several controllers or AWX instance groups. Validation scripts check that every host belongs to the expected groups.

Likely follow-ups

- Why cache dynamic inventory results?
- How do instance groups help in AWX?

Make the inventory a view over a source of truth with plugins, caching, keyed groups and filters, keep group structure shallow, put variables in groups, limit runs to slices, and spread work over several controllers or instance groups.

```
ansible-inventory -i aws_ec2.yml --list --yaml | head
```

```
plugin: amazon.aws.aws_ec2
cache: true
cache_timeout: 300
keyed_groups:
  - key: tags.Role
    prefix: role
```

Interview tip: Say the inventory is derived from truth, never hand-maintained.

📚 **Official docs**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q58 How would you patch thousands of Linux servers safely with Ansible?

Scale & performance

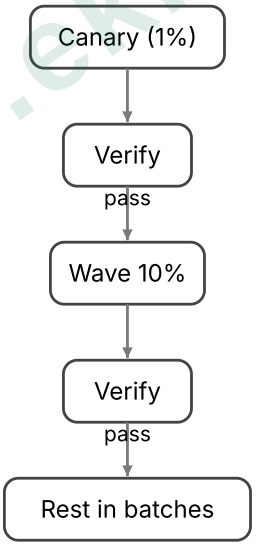
🎙 Spoken answer · ~67 s

I treat patching as a controlled rollout. First, define groups by criticality and redundancy, and a maintenance window with approval. Pre-checks confirm free disk space, repository access, backups or snapshots, and that the cluster has quorum or spare capacity. Then patch in batches with serial, starting with a canary group, and for each host: drain from the load balancer or put in maintenance mode, apply updates with the package module, reboot only if needed, detected through needs-restarting or the presence of reboot-required, wait for the host to come back with wait_for_connection, run health checks, and return it to service. Set max_fail_percentage so the rollout stops if too many hosts fail. Strategies like free can speed independent fleets, while clustered services use strict serial one at a time. Output is recorded to a report, and the process is scheduled in AWX with notifications. I also keep an easy exclusion list for packages that need coordinated upgrades, such as kernel or database versions.

Likely follow-ups

- How do you detect that a reboot is required?
- What do you do when a canary fails?

🗺 Patch waves



Group by criticality, pre-check capacity and backups, patch in serial batches starting with a canary, drain and return from the load balancer, reboot only when required, health-check each host, stop on a failure threshold and report results.

```
- ansible.builtin.reboot:
    reboot_timeout: 900
  when: reboot_required.stat.exists
```

```
ansible-playbook patch.yml -l canary --check --diff
```

interviewhub.eknathalabs.com

Interview tip: Say canary, batch, health check, stop threshold.

📚 **Official docs**

- Ansible documentation

Q59 A host shows UNREACHABLE. How do you troubleshoot it?

Troubleshooting

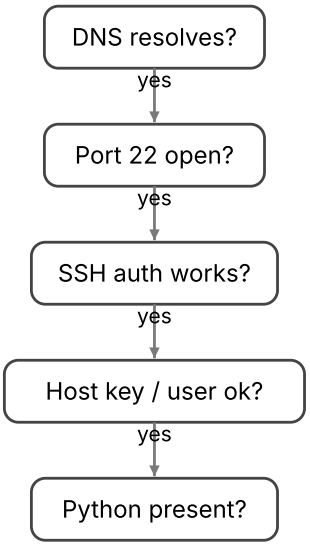
🎙 **Spoken answer · ~64 s**

UNREACHABLE means Ansible could not establish a connection, as opposed to a task failing. I move from the outside in. First, verify basic reachability with ping and the port with nc or a simple ssh attempt using the same user, key and options Ansible uses, which I see with the verbose flag: triple or quadruple v shows the exact SSH command. Common causes are DNS or the wrong address in inventory, a security group or firewall blocking port 22, the host being down or rebooting, a wrong remote user or key, host key mismatches in known hosts, too many connection attempts hitting MaxStartups, a bastion not configured, a stale ControlPersist socket, or a Windows host with WinRM disabled. I also check the SSH daemon logs on the target for authentication errors. Once SSH works manually, I rerun the ping module. For intermittent cases, I look at network quality, timeouts and the connection retries setting.

Likely follow-ups

- How do you see the exact SSH command Ansible runs?
- What does a stale ControlPersist socket look like?

🗺 **UNREACHABLE triage**



UNREACHABLE means no connection. Reproduce with the same SSH command using -vvv, then check address, DNS, firewall, user, key, host keys, sshd logs and bastion settings. Once SSH works manually, retest with the ping module.

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

```
ansible web01 -m ping -vvvv
```

```
ssh -vvv -i ~/.ssh/ansible ansible@10.0.1.12
```

Interview tip: Say replicate the exact SSH command with -vvvv first.

📚 **Official docs**

- Ansible documentation

Q60 A playbook is slow. How do you find out why and speed it up?

Troubleshooting

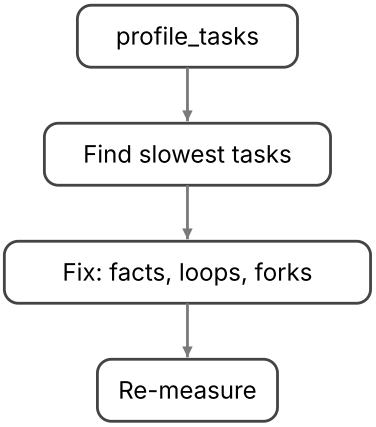
🎙 **Spoken answer · ~60 s**

I measure first. I enable the profile_tasks and timer callbacks to see the slowest tasks and total time, and optionally profile_roles. Then I look for the usual causes. Fact gathering across many hosts: disable or subset it or use caching. Low forks leaving the controller idle. SSH overhead: enable pipelining and ControlPersist. Many small tasks in loops that could be a single call, for example installing packages one by one. Package cache updates on every run, which cache_valid_time avoids. Tasks that wait on timeouts, such as unreachable hosts or slow DNS. Expensive templating or lookups evaluated repeatedly. Linear strategy waiting on a single slow host: consider free. Serial settings making batches tiny. Slow API-based modules calling cloud services without batching. After changes, I rerun and compare the timing, aiming at the top few tasks, since they usually account for most of the time.

Likely follow-ups

- What does cache_valid_time do?
- Why can a loop be much slower than a list?

🗺 **Speed up a playbook**



Profile with profile_tasks to find the slowest tasks, then fix the cause: fact gathering, low forks, SSH overhead, per-item loops, repeated package cache updates, slow lookups or the linear strategy waiting on one host. Re-measure after each change.

InterviewHub - interviewhub.eknathalabs.com

```
ANSIBLE_CALLBACKS_ENABLED=ansible.posix.profile_tasks,ansible.posix.timer ansible-playbook site.yml
```

```
- ansible.builtin.apt:
    update_cache: true
    cache_valid_time: 3600
```

Interview tip: Say measure with profile_tasks, then change one thing at a time.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

Q61 How do you debug a failing playbook: verbosity, the debug module and the debugger?

Troubleshooting

🎙 **Spoken answer · ~67 s**

I use increasing levels of detail. The v flags raise verbosity: one shows task results, two adds input, three shows connection details and four shows the SSH commands and the full communication. The debug module prints variables or messages, for example var set to a registered result, helping me see what a task returned or what a variable contains. For interactive investigation, the strategy debug or the debugger keyword on a task or play opens a prompt on failure, where I can print the task arguments, update variables, redo the task or continue. Additional tools: the step option to confirm each task, start-at-task to resume from a specific task, the syntax-check option for parse errors, list-tasks and list-hosts to confirm what will run, and ansible-inventory host to inspect variables. I also run a single task as an ad-hoc command with the same arguments to isolate module behaviour. In production runs, I avoid leaving verbose output enabled because it can reveal secrets.

Likely follow-ups

- How do you resume from a given task?
- What does the debugger let you do?

Raise verbosity with -v to -vvvv, use the debug module to inspect variables and results, use the debugger keyword or debug strategy for interactive failures, and use step, start-at-task, syntax-check, list-tasks and ansible-inventory --host to narrow down.

```
ansible-playbook site.yml -vvv --start-at-task 'Install nginx'
```

```
- hosts: web
  debugger: on_failed
```

```
ansible-playbook site.yml --syntax-check
```

certs.eknathalabs.com

InterviewHub - interviewhub.ekn

Interview tip: Say -vvvv for connection issues and debugger for logic issues.

📚 Official docs

- Ansible: Playbook guide
- Ansible documentation

Q62 A task reports changed on every run. How do you track it down?

Troubleshooting

🎙 Spoken answer · ~64 s

A task that always changes breaks idempotency and triggers handlers needlessly. The usual culprits are command, shell and raw tasks that have no changed_when, and fixes include creates, removes or a check command with a changed_when condition. Other causes: templates containing a timestamp, random value or ordering of dictionary keys that differs between runs; a file or lineinfile task with a regular expression that does not match what it writes, so it keeps adding the line; modules that compare numeric strings and quoted values differently, such as mode written as 644 instead of '0644'; services started with an unstable restart; and package versions set to latest. To investigate I run the play twice with diff mode to see the actual differences, look at the registered output to see what changed, and use the verbose flag. After fixing, I add a run-twice test in CI that fails if the second run reports any changes.

Likely follow-ups

- Why does mode as a number cause changes?
- How would you add a CI check for idempotency?

Always-changed tasks usually are command or shell without changed_when, unstable templates, regexes that do not match their own output, mode written as a number, or latest versions. Run twice with diff, fix, and add a CI check that the second run has zero changes.

```
ansible-playbook site.yml --diff | grep -A5 'changed:'
```

```
- ansible.builtin.file:
    path: /etc/app
    mode: '0755'
```

Interview tip: Say quote file modes and use changed_when on commands.

📚 Official docs

- Ansible: Playbook guide

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q63 You get an 'undefined variable' error or a templating failure. What is your approach?

Troubleshooting

🎙 **Spoken answer · ~67 s**

First I read the message carefully, since Ansible names the variable and the task. Then I confirm where the variable should come from: the inventory, group_vars, a role default, a registered result or a fact. I run the debug module with var for the name on the failing host, and ansible-inventory with the host option to see inventory-level variables. Common causes: a typo or different naming between role and playbook; the variable defined only for a different group; a registered variable from a task that was skipped or failed earlier; facts not gathered because gather_facts is off or limited; precedence causing an override with an empty value; a nested key missing from a dictionary; and quoting issues where a value starting with a curly brace is parsed as a template. Fixes include defining the variable, using default where optional, using mandatory with a message where it must exist, and adding assert checks near the top so problems are caught early.

Likely follow-ups

- Why must a YAML value starting with braces be quoted?
- How does a skipped task affect registered variables?

Read the error for the variable and task, check where it should be defined, inspect with debug and ansible-inventory --host, and look for typos, wrong groups, skipped registers, missing facts, precedence or YAML quoting problems. Then fix with defaults, mandatory and early asserts.

```
ansible web01 -m debug -a 'var=hostvars[inventory_hostname]' -i inventory
```

```
ansible-inventory -i inventory --host web01 | jq .
```

Interview tip: Say add an assert so the next failure is clearer.

📚 **Official docs**

- Ansible: Variables and precedence
- Ansible: Error handling

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q64 How do you troubleshoot privilege escalation failures, such as a missing sudo password?

Troubleshooting

🎙 Spoken answer · ~64 s

Typical errors are 'Missing sudo password', 'Authentication failure' when using become, or a timeout waiting for the privilege escalation prompt. I first test manually: ssh to the host with the same user and run sudo -n true, which shows whether passwordless sudo works. Common causes: the user is not in sudoers or the entry has a restrictive command list, so the temporary Python invocation is not allowed; requiretty is set in sudoers, which also breaks pipelining; the become method is wrong, for example su needed rather than sudo; the become password variable is missing or in the wrong scope; a different become_user lacks permission to read the temporary files in the remote directory, which needs the acl package or a world-readable temporary directory setting; or a PAM or shell profile prints text that interferes with the prompt. Fixes include correcting sudoers with visudo, setting become variables, and adjusting the allow_world_readable_tmpfiles setting only when necessary.

Likely follow-ups

- What does sudo -n true tell you?
- Why does become to an unprivileged user need acl?

Reproduce with ssh and sudo -n true. Check sudoers entries, requiretty, the become method and variables, whether become_user can read temporary files (needs acl), and noisy shell profiles. Fix sudoers with visudo and set become settings explicitly.

```
ssh ansible@host 'sudo -n true && echo ok'
```

```
ansible web01 -m ping -b -vvv
```

Interview tip: Say test sudo manually before touching Ansible.

📚 Official docs

- Kubernetes: RBAC good practices
- sudoers(5) manual

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q65 What causes 'module not found', Python interpreter and dependency errors on targets?

Troubleshooting

🎙 Spoken answer · ~70 s

Many modules need Python on the target, and some need extra Python libraries. If the interpreter is missing or at a nonstandard path, Ansible reports an interpreter discovery warning or failure. I set ansible_python_interpreter in inventory for those hosts, or use interpreter_python auto, and install Python with the raw module for bootstrapping. Errors like 'No module named boto3' or 'Failed to import the required Python library' mean the library is missing on the host that runs the module: for cloud and API modules that run on the control node via delegate or connection local, it is the controller's Python environment, not the target, which often differs from the Python that ansible-playbook uses when installed in a virtual environment. I install requirements in the right environment, ideally in an execution environment image, pin them in a requirements file, and use ansible --version to check which Python and which module paths are active. 'Module not found' for a collection module means the collection is missing or not on the collections path.

Likely follow-ups

- Where do cloud modules run and which Python do they use?
- How do you bootstrap Python on a minimal host?

Interpreter or library errors come from missing Python or packages on the host running the module, which for cloud modules is usually the controller. Set ansible_python_interpreter, install requirements in the right environment or execution environment, and install missing collections.

```
ansible --version
```

```
ansible newhost -m raw -a 'apt-get install -y python3' -b
```

```
pip install boto3 botocore
```

Interview tip: Say know which Python runs the module: controller or target.

📚 Official docs

- Ansible: Error handling
- Python Documentation
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q66 A handler did not run, or ran at the wrong time. Why?

Troubleshooting

🎙 Spoken answer · ~71 s

Handlers run only when notified by a task that actually reported a change, so if the notifying task showed ok instead of changed, nothing fires, which is expected behaviour and often the cause when someone expects a restart after a manual config change. Other reasons: a failure before the end of the play stops execution before handlers run unless force_handlers is true; the handler name does not match the notify string exactly, including case and spaces; the handler lives in a role but the notify is outside it, or vice versa, or the handler is defined in an included file that is not loaded; with serial, handlers run at the end of each batch; and with imported versus included roles, handler visibility differs. Timing problems occur when a later task needs the restarted service, which needs a flush_handlers meta task. To diagnose, I run with verbose output to see notifications, check the names, and use list-tasks or the debug module to confirm what is loaded. The listen keyword helps decouple names.

Likely follow-ups

- What does force_handlers change?
- How does the listen keyword help?

Handlers run only on a changed notify, after the play, in definition order. They can be skipped by earlier failures unless force_handlers is set, by name mismatches or by scope. Use flush_handlers for mid-play restarts and listen to decouple names.

```
- hosts: web
  force_handlers: true
  tasks:
    - ansible.builtin.meta: flush_handlers
```

Interview tip: Say changed is the trigger, and failures can skip handlers.

📚 Official docs

- Ansible: Playbook guide

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q67 A playbook fails to parse because of YAML or syntax errors. How do you find and avoid them?

Troubleshooting

🎙 **Spoken answer · ~61 s**

I start with ansible-playbook syntax-check, which reports the file and line, and then yamllint or ansible-lint for style and structural problems. Frequent causes: wrong indentation or tabs instead of spaces; an unquoted value containing a colon followed by a space, or starting with a special character like a curly brace, percent sign or asterisk; boolean surprises where yes, no, on and off are converted to booleans, so a value like no for Norway becomes false; duplicate keys, where the later one silently wins; strings that need quotes such as version numbers like 1.10 becoming 1.1; and mixing list and dictionary syntax. Jinja2 errors appear at run time, including unmatched braces or filters that do not exist. To avoid them, I use an editor with YAML and Ansible schema validation, enable lint in a pre-commit hook and CI, quote strings consistently, and prefer true and false for booleans.

Likely follow-ups

- Why does the version 1.10 become 1.1?
- What does yamllint add beyond syntax-check?

Run syntax-check, then yamllint and ansible-lint. Typical errors are indentation, unquoted colons or braces, yes and no becoming booleans, duplicate keys and numeric versions. Use editor schema validation, pre-commit hooks and quote strings.

```
ansible-playbook site.yml --syntax-check
```

```
yamllint . && ansible-lint
```

Interview tip: Say quote versions and use true or false for booleans.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Error handling
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q68 Vault decryption fails with 'Decryption failed' or a missing password. How do you resolve it?

Troubleshooting

🎙 Spoken answer · ~70 s

I first identify which file and which vault ID are involved from the error message. Common causes: the wrong password is supplied, or the right password but for a different vault ID than the file's header label; no password source is configured so Ansible cannot decrypt anything, for example when running from a different user, a cron job or AWX without the credential attached; the file was edited manually and the vault header or line wrapping was corrupted; the password file contains a trailing newline issue or different encoding, although Ansible strips newlines; or mixed encrypted content from several passwords where only one is provided. I test with ansible-vault view on the file with the intended password source to separate configuration problems from content problems. Fixes are supplying the correct vault-id, attaching the vault credential in AWX, restoring the file from version control, or rekeying. To prevent trouble, document the vault IDs, store passwords in a manager, and add a CI check that decrypts all vault files.

Likely follow-ups

- How do you tell a wrong password from a corrupted file?
- How would a CI check validate every vault file?

Identify the file and vault ID, test with ansible-vault view, and check for wrong or missing password sources, mismatched labels, corrupted headers or an unattached AWX credential. Restore or rekey as needed, and add a CI check decrypting every vault file.

```
ansible-vault view group_vars/prod/vault.yml --vault-id prod@scripts/get_pass.sh
```

```
head -1 group_vars/prod/vault.yml
```

Interview tip: Say test decryption alone with ansible-vault view.

📚 Official docs

- Ansible Vault
- HashiCorp Vault documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q69 When would you use ansible-pull instead of the usual push model?

Scale & performance

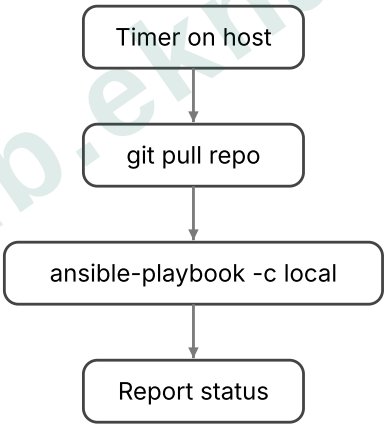
🎙 Spoken answer · ~67 s

In the push model a control node connects to each host. With ansible-pull, each managed host runs a scheduled job, typically cron or a systemd timer, that fetches a playbook repository from Git and applies it locally against itself. This inverts scaling: instead of one controller opening thousands of connections, every host does its own work, so it scales almost without limit, works for machines behind NAT, in edge locations or intermittently connected, and enforces desired state continuously like an agent-based tool. Typical uses are large fleets, laptops, kiosks, autoscaling instances that configure themselves at boot, and edge or retail sites. The costs: you lose central orchestration and ordering across hosts, it is harder to get a central view of results, each host needs Git access and Ansible installed, and credentials for secrets must be present on every machine. I use pull for baseline convergence and self-configuring nodes, and push or AWX for coordinated operations such as rolling deployments and patching.

Likely follow-ups

- How do you report results from pull-based hosts?
- Why is orchestration harder with pull?

🗺 Pull model



ansible-pull makes each host fetch a Git repo and apply it locally on a schedule. It scales widely and suits NAT, edge and autoscaled hosts, but loses central orchestration and visibility, so combine it with push for coordinated changes.

```
ansible-pull -U https://git.example.com/infra.git -i localhost, local.yml
```

```
*/15 * * * * root ansible-pull -o -U $REPO local.yml >> /var/log/ansible-pull.log
```

Interview tip: Say pull for convergence at scale, push for orchestration.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com

Q70 How do you protect rate-limited APIs and shared systems when automation runs in parallel?

Scale & performance

🎙 **Spoken answer · ~61 s**

Running tasks across many hosts at once can overwhelm a shared dependency, such as a cloud API, a package repository, a load balancer or a database. I control concurrency in layers. The throttle keyword on a task or block limits how many hosts execute that task at the same time, regardless of forks, which is ideal for a call to a rate-limited API or an artifact server. The serial keyword limits batch sizes for the whole play, and forks limits global parallelism. For retries on throttling errors, I use until, retries and delay loops with backoff, and many cloud modules have their own retry options. Delegating calls to a single host through run_once and delegate_to avoids many duplicate requests. Mirrors and caches for package and image downloads reduce external load. I also monitor the dependency during a run and keep a conservative default that can be raised.

Likely follow-ups

- What is the difference between throttle and serial?
- How do you retry on a 429 response?

Use throttle to limit concurrent executions of a task, serial for batch size and forks for global parallelism, add retries with delay on throttling errors, use run_once for single calls, and mirror downloads to reduce external load.

```
- ansible.builtin.uri:
    url: https://api.example.com/register
  throttle: 3
  register: r
  until: r.status == 200
  retries: 5
  delay: 10
```

Interview tip: Say throttle protects the dependency, not the controller.

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q71 What is the difference between AWX, Ansible Automation Platform and Ansible Tower?

AWX & AAP

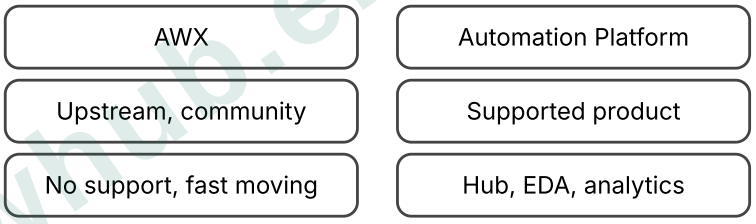
🎙 **Spoken answer · ~62 s**

Tower was the original commercial product from Ansible, offering a web UI, role-based access control, scheduling, credentials management, an API and logging around playbooks. When Red Hat open sourced the upstream project, it became AWX, a community project with frequent releases and no support guarantees. Red Hat's supported product is now Ansible Automation Platform, abbreviated AAP, which includes the automation controller, which is the successor of Tower and is derived from AWX, plus Automation Hub for certified and private collections, execution environments, Event-Driven Ansible, automation mesh and analytics, with enterprise support and security patches. In practice: AWX suits labs, learning and teams comfortable with upstream stability risks, while AAP is chosen when support, certification and long-term stability matter. Upgrading AWX across many versions can be tricky, so I plan carefully. The concepts, such as projects, inventories, credentials and job templates, are the same across them, which makes skills transferable.

Likely follow-ups

- Which components does AAP add beyond the controller?
- Why is AWX upgrading sometimes risky?

🗺 **AWX vs AAP**



Tower was the original commercial product, AWX is the open source upstream, and AAP is Red Hat's supported platform with the automation controller, Automation Hub, execution environments, Event-Driven Ansible and mesh. The concepts transfer across all three.

```
kubectl get awx -n awx
```

```
curl -s -u admin:$PASS https://awx.example.com/api/v2/ping/
```

Interview tip: Say AWX is upstream, AAP is supported, same concepts.

📚 **Official docs**

- Ansible Automation Platform / AWX
- Ansible documentation

interviewhub.eknathalabs.com

Q72 Explain projects, inventories, credentials and job templates in AWX or the controller.

AWX & AAP

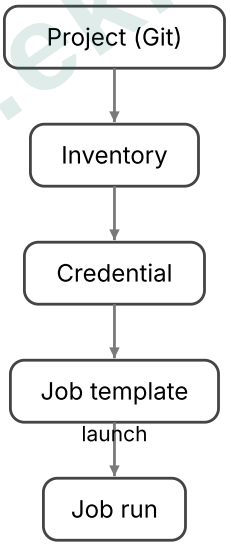
🎙 Spoken answer · ~64 s

These are the building blocks. A project points to a source of playbooks, usually a Git repository with a branch or tag, and the controller syncs it, optionally updating on launch. An inventory holds hosts and groups, either static, imported from a source, or synced dynamically from cloud and virtualisation systems, with variables. A credential stores secrets such as machine SSH keys, cloud API keys, vault passwords or source control tokens, encrypted at rest and never shown after saving, and injected into jobs at run time. A job template ties these together: it combines a project and playbook, an inventory, credentials, extra variables, limits, tags, verbosity and execution environment, plus options such as prompt on launch, surveys, concurrent runs and timeouts. Running it creates a job with a full log of results. Teams then get permission to use templates without seeing the secrets, which is the main value: delegated, audited execution of approved automation.

Likely follow-ups

- How can a user run a job without seeing the credential?
- What does prompt on launch allow?

🗺 Job template inputs



Projects pull playbooks from Git, inventories hold hosts, credentials store encrypted secrets injected at run time, and job templates combine them with variables, limits and options. Users get permission to run templates without seeing secrets.

```
awx job_templates launch 12 --extra_vars '{"version":"1.4.2"}' --monitor
```

```
awx projects update 5 --monitor
```

Interview tip: Say delegated, audited execution without exposing secrets.

certs.eknathalabs.com

📚 **Official docs**

- Ansible: Templating (Jinja2)
- Ansible Automation Platform / AWX

Q73 How does role-based access control work in AWX and the automation controller?

AWX & AAP

🎙 **Spoken answer · ~64 s**

The model has organisations at the top, containing users, teams and resources such as projects, inventories, credentials and templates. Permissions are granted through roles on specific objects: for a job template, roles include admin, execute and read; for an inventory, admin, update, use and ad hoc; for a credential, admin, use and read; and organisation-level roles such as admin, auditor and member. Roles can be given to individual users or, better, to teams, and teams can be synced from an identity provider using LDAP, SAML or OIDC mappings, so access follows group membership in the company directory. A typical pattern: developers get execute on a deployment template and use on the staging credential, but not on production ones, while a release team has production access. System auditors get read-only visibility. I follow least privilege, separate credentials per environment, review access regularly, and use the activity stream, which records who changed what, to support audits.

Likely follow-ups

- Why assign roles to teams rather than users?
- What does the use role on a credential permit?

Organisations contain teams, users and resources; roles such as admin, execute, use and read are granted on objects, best through teams mapped from LDAP, SAML or OIDC. Apply least privilege, separate credentials per environment and audit via the activity stream.

```
awx roles grant --team 3 --resource-type job_template --resource 12 --role execute
```

```
awx teams list
```

Interview tip: Say teams mapped from the identity provider.

📚 **Official docs**

- Ansible: Roles
- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q74 What are workflow templates in AWX and how do you use approvals and conditional paths?

AWX & AAP

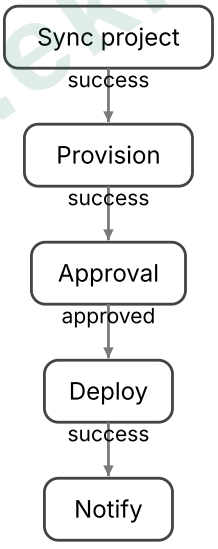
🎙 Spoken answer · ~63 s

A workflow template chains job templates, project or inventory syncs, other workflows and approval nodes into a directed graph, where each edge is conditioned on success, failure or always. This lets you model a release process: run a pre-check, provision, configure, test, and if the test fails, run a rollback and notify. Nodes can pass data forward through set_stats in a playbook, so a later node can use values produced earlier, such as a build identifier. Approval nodes pause the workflow until a user or team approves or denies, with a timeout, which gives change control for production steps. Workflows can use convergence nodes that wait for several parallel branches, and different inventories or limits per node. Benefits are reuse of small, well-tested templates, a visual representation, one audit trail, and consistent permissions. Keep workflows reasonably small and keep logic inside playbooks where possible, since complex graphs are harder to maintain.

Likely follow-ups

- How does set_stats pass data between nodes?
- How do approval timeouts behave?

🗺 Workflow with approval



Workflows chain job templates, syncs and approval nodes with success, failure and always paths. Use set_stats to pass data, approvals for production gates and convergence for parallel branches, keeping complex logic in playbooks.

```
- ansible.builtin.set_stats:
    data:
      build_id: "{{ build_id }}"
    per_host: false
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
awx workflow_job_templates launch 7 --monitor
```

Interview tip: Say approvals and rollback paths make a release workflow.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Templating (Jinja2)
- Ansible Automation Platform / AWX

Q75 What are execution environments and how do you build and manage them?

AWX & AAP

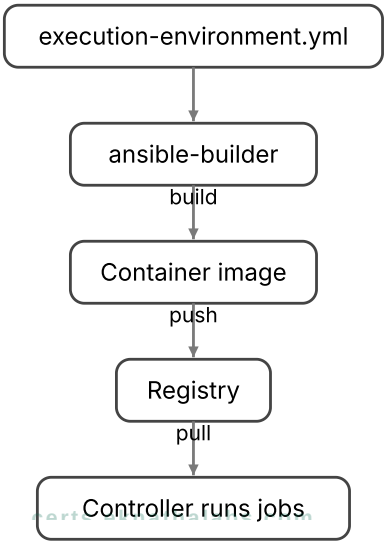
🎙 **Spoken answer · ~62 s**

An execution environment, or EE, is a container image that packages ansible-core or ansible-runner together with collections, Python libraries and system packages needed by playbooks, so jobs run in a consistent, isolated environment, instead of whatever is installed on the controller. They replace the older Python virtual environments on Tower. You define the contents in an execution-environment.yml for ansible-builder, listing base image, collections requirements, Python requirements and system packages through bindep, then build the image, push it to a registry and register it in the controller, assigning it to a project or job template. The same image can run locally with ansible-navigator, which gives developers parity with production. Benefits: reproducibility, no dependency conflicts between teams, easy version pinning and rollbacks by tag, and supply chain control by building from approved bases and scanning images. I version images semantically, rebuild regularly for security patches and test them in CI.

Likely follow-ups

- What does ansible-builder take as input?
- How do developers run an EE locally?

🗺 **Execution environment**



InterviewHub - interviewhub.eknathalabs.com

An execution environment is a container image with ansible-core, collections and dependencies. Build with ansible-builder from a definition file, push to a registry, assign it to job templates, and use ansible-navigator locally for parity. Version, scan and rebuild regularly.

```
ansible-builder build -t registry.example.com/ee-platform:1.3 -f execution-environment.yml
```

```
ansible-navigator run site.yml --eei registry.example.com/ee-platform:1.3 -m stdout
```

Interview tip: Say the EE makes laptop, CI and controller identical.

📚 **Official docs**

- Ansible Automation Platform / AWX

Q76 How do instance groups and automation mesh help scale and isolate execution?

AWX & AAP

🎙 **Spoken answer · ~65 s**

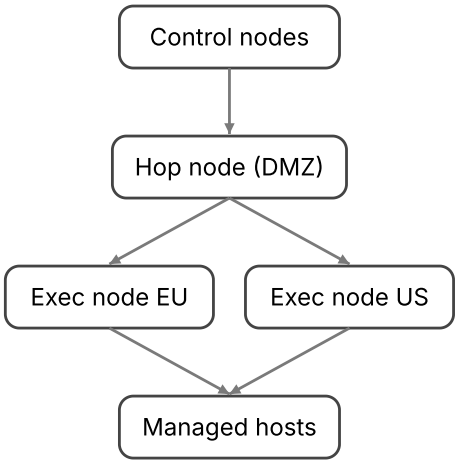
Instance groups are pools of execution capacity that jobs are assigned to, and automation mesh, built on Receptor, connects control, hybrid, execution and hop nodes into an overlay network. The control plane schedules and tracks jobs, while execution nodes actually run playbooks, and they can be placed close to the targets: in a remote data centre, a secured network segment, a cloud region or behind a firewall, where hop nodes bridge networks. This reduces latency, avoids opening wide inbound access to the central controller, and lets you scale capacity by adding nodes rather than enlarging one machine. You can bind inventories, organisations or job templates to instance groups, for example a PCI network group that only runs approved workloads. Containerised groups launch each job in its own pod on a Kubernetes cluster, giving elastic capacity and isolation. Capacity is measured by forks and memory, so I size nodes accordingly and monitor queue times and failed jobs.

Likely follow-ups

- What does a hop node do?
- How is capacity measured for a node?

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

🗺 Automation mesh



Instance groups pool execution capacity, and automation mesh via Receptor links control, execution and hop nodes across networks. Place execution nodes near targets, bind inventories or templates to groups, scale by adding nodes, and use container groups for elastic per-job pods.

```
awx instance_groups list
```

```
kubectl get pods -n awx -l app.kubernetes.io/name=awx-task
```

Interview tip: Say execution near the targets, control plane in the centre.

📚 Start here (official docs)

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q77 How do you integrate AWX with CI/CD, webhooks, surveys and notifications?

AWX & AAP

🎙 Spoken answer · ~64 s

Everything in the controller is available through a REST API, and the awx CLI and the awx collection wrap it, so a pipeline can launch a job template, wait for completion and fail if the job fails, with extra variables such as the version to deploy. Webhook integration lets GitHub or GitLab trigger a template on a push or a merge, with a secret to authenticate the call. Surveys add a form to a template where users answer questions with validated types, choices and defaults, which turns a playbook into a self-service request with safe inputs and avoids free-form extra vars. Notification templates send job results to email, Slack, Teams, PagerDuty or a webhook on start, success or failure, at template, workflow or organisation level. Schedules run templates regularly, for example nightly compliance checks. I authenticate automation using OAuth tokens scoped to a service account with only the rights needed, and rotate them.

Likely follow-ups

- Why use surveys instead of free extra vars?
- How do you authenticate a pipeline to the controller?

Use the REST API, awx CLI or collection to launch templates from CI, webhooks for repository events, surveys for validated self-service inputs, notification templates for results and schedules for recurring jobs. Use scoped OAuth tokens for service accounts.

```
awx job_templates launch deploy-app --extra_vars '{"version":"1.4.2"}' --monitor --timeout 1800
```

```
curl -H "Authorization: Bearer $TOKEN" -X POST https://awx.example.com/api/v2/job_templates/12/launch/
```

Interview tip: Say a scoped service account token and validated surveys.

📚 Official docs

- Ansible Automation Platform / AWX
- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q78 How do you deploy, size and make AWX highly available on Kubernetes?

AWX & AAP

🎙 **Spoken answer · ~66 s**

AWX is deployed with the AWX operator, which watches an AWX custom resource and creates the web, task and other deployments, services and ingress. The database is PostgreSQL, either a managed external service or one the operator creates, and for production I prefer an external, highly available database with backups, since it holds all state. Redis is a local component. High availability comes from running multiple replicas of the web and task pods across nodes and zones with anti-affinity, using an ingress or load balancer, and from separating execution capacity into container groups or execution nodes so job load does not starve the control plane. Sizing depends on forks and concurrent jobs: each job needs memory for the runner and CPU, so I set resource requests and limits and monitor queue wait times. Projects use persistent storage. I use external secrets for credentials, TLS on ingress, and configure logging aggregation to send job events to the central log system.

Likely follow-ups

- Why use an external PostgreSQL for production?
- How does the operator manage upgrades?

Deploy with the AWX operator and an AWX custom resource, use an external highly available PostgreSQL, run multiple web and task replicas with anti-affinity, separate execution into container groups, size by forks and monitor queue times.

```
kubectl apply -k github.com/ansible/awx-operator/config/default?ref=2.19.0
```

```
kubectl get awx,pods -n awx
```

Interview tip: Say external database and separated execution capacity.

📚 **Official docs**

- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q79 What is Event-Driven Ansible and where does it fit?

AWX & AAP

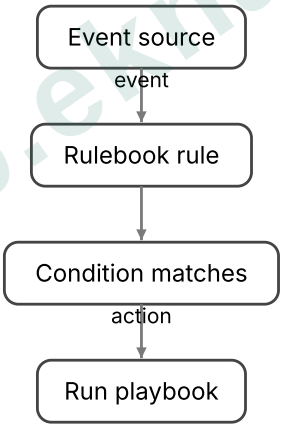
🎙 Spoken answer · ~65 s

Event-Driven Ansible, or EDA, lets automation react to events instead of being started by a person or a schedule. You write rulebooks in YAML, each with sources, rules and actions. Sources are event plugins that listen to systems such as webhooks, Kafka, Prometheus Alertmanager, cloud events or log streams. Rules match conditions on the event data, and actions run a playbook, a job template on the controller, a module or set a fact. Examples include restarting a service when an alert fires, opening a ticket and gathering diagnostics on a failure, or quarantining a host when a security tool reports a compromise. It fits self-healing and auto-remediation, with the usual need for safety: restrict actions to well-understood, idempotent remediations, add rate limits and conditions to avoid loops where an action triggers more events, require approvals for risky steps, and log every decision. EDA runs through the rulebook engine, ansible-rulebook, or the EDA controller in AAP.

Likely follow-ups

- How would you prevent remediation loops?
- What is a rulebook source?

🗺 Event-driven automation



EDA runs automation in response to events through rulebooks with sources such as webhooks, Kafka or Alertmanager, rules that match conditions and actions that run playbooks. Restrict to safe idempotent remediations with rate limits, approvals and logging.

```
ansible-rulebook --rulebook alerts.yml -i inventory.yml --verbose
```

```
- name: restart on alert
  condition: event.alert.labels.alertname == 'ServiceDown'
  action:
    run_job_template:
      name: restart-service
```

Interview tip: Say auto-remediation needs guardrails against loops.

certs.eknathalabs.com

📚 **Official docs**

- Ansible documentation

Q80 How do you back up, upgrade and recover the automation controller?

AWX & AAP

🎙 **Spoken answer · ~65 s**

State lives in the database, the projects and the configuration secrets, such as the secret key that encrypts credentials, so a recoverable backup includes all of them. For AWX on Kubernetes, the operator provides an AWXBackup custom resource that saves the database and secrets to a persistent volume, and an AWXRestore resource to restore into a new deployment. For an external PostgreSQL, I also use the database's own point-in-time recovery. Backups are copied offsite and restore is tested regularly. Upgrades: read the release notes, back up first, upgrade the operator and then the AWX version one supported step at a time, check migrations, and verify with a test job. For AAP, the installer provides backup and restore playbooks. I keep configuration as code, using the awx collection or the controller configuration collection, so job templates, credentials and inventories can be recreated from Git even in a disaster. I document the recovery procedure and its time objective.

Likely follow-ups

- Why does the secret key matter in a restore?
- What is configuration as code for the controller?

Back up the database, projects and secrets including the credential encryption key, using AWXBackup or the installer and database PITR. Upgrade stepwise after a backup and verify with a test job. Keep controller configuration as code and test restores.

```
kubectl apply -f awxbackup.yaml
```

```
kubectl get awxbackup -n awx
```

Interview tip: Say test the restore and keep config as code.

📚 **Official docs**

- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q81 How do you combine Terraform and Ansible in one workflow?

Cloud/network/K8s

🎙 Spoken answer · ~63 s

They cover different layers. Terraform creates and manages infrastructure resources, such as networks, instances, load balancers and managed services, tracking them in state, and Ansible configures what runs on the machines and operates them. A handoff is needed: Terraform outputs the addresses or identifiers, and Ansible consumes them. Options include a dynamic inventory from the cloud provider filtered by tags that Terraform applied, which is robust because inventory follows reality; the cloud.terraform collection and its inventory plugin, which reads the Terraform state directly; or generating an inventory file from outputs. Another approach is for Terraform to call Ansible from a provisioner or through a pipeline step, though provisioners are discouraged because failures leave inconsistent state. I prefer a pipeline: apply Terraform, wait for hosts to be reachable, then run Ansible, with each stage idempotent and retryable. Immutable images built with Packer and Ansible reduce the amount of runtime configuration needed.

Likely follow-ups

- Why are Terraform provisioners discouraged?
- How does the cloud.terraform inventory work?

🗺 Terraform then Ansible



Terraform creates infrastructure and Ansible configures it. Hand off through a tag-based dynamic inventory or the cloud.terraform inventory plugin, in a pipeline of Terraform apply, wait for connectivity, then Ansible. Prefer pipelines to provisioners.

```
terraform apply -auto-approve && ansible-playbook -i terraform.yml configure.yml
```

```
plugin: cloud.terraform.terraform_provider
project_path: ./infra
```

Interview tip: Say tags from Terraform drive the Ansible inventory.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q82 How do you use Ansible with AWS: dynamic inventory, SSM and common modules?

Cloud/network/K8s

🎙 Spoken answer · ~64 s

The amazon.aws collection provides modules for EC2, VPC, S3, IAM and more, along with the aws_ec2 inventory plugin, which discovers instances and builds groups from tags, regions, instance types and VPCs, with compose to choose the connection address, for example a private IP when running inside the VPC. Credentials come from the standard AWS chain, preferably an instance role or OIDC-assumed role, not static keys. For connectivity without SSH, the aws_ssm connection plugin runs tasks through Systems Manager Session Manager, which needs no inbound ports and no SSH keys, and uses an S3 bucket for module transfer, giving IAM-based access control and audit logs. Typical tasks include configuring hosts, creating resources for small setups, and operational runbooks such as snapshots or AMI updates. For infrastructure lifecycle at scale I still prefer Terraform or CloudFormation, with Ansible for configuration. Boto3 and botocore must be installed in the controller's Python environment or the execution environment.

Likely follow-ups

- What are the advantages of SSM over SSH?
- Which Python libraries do the AWS modules need?

Use amazon.aws with the aws_ec2 inventory plugin and tag-based groups, authenticate with roles or OIDC, and use the aws_ssm connection for access without inbound ports or keys. Install boto3 in the controller environment, and use Terraform for lifecycle at scale.

```
ansible-inventory -i aws_ec2.yml --graph
```

```
ansible_connection: community.aws.aws_ssm
ansible_aws_ssm_bucket_name: ansible-ssm-transfer
```

Interview tip: Say SSM removes SSH keys and inbound ports.

📚 Official docs

- Ansible: Inventory
- Ansible: Cloud guides
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q83 How do you manage Kubernetes resources with Ansible, and when is it appropriate? Cloud/network/K8s

🎙 Spoken answer · ~65 s

The kubernetes.core collection provides the k8s module to apply manifests or inline definitions, k8s_info to query resources, helm and helm_info for charts, and a kubectl-based connection and inventory plugin. Authentication uses a kubeconfig or in-cluster service account, and the controller needs the kubernetes Python library. Ansible is appropriate for bootstrapping clusters, creating namespaces, quotas and RBAC across many clusters, installing a base set of Helm charts, and for orchestrating steps that mix systems, such as updating DNS, a database and a deployment in order. It is less suited to continuous reconciliation of application state, where GitOps tools like Argo CD or Flux run controllers that detect drift constantly, and Ansible only acts when run. I therefore use Ansible to prepare clusters and platform components, and GitOps for application delivery. Practical points: use the apply option to behave like kubectl apply, wait for conditions with wait and wait_condition, and keep manifests as templates or files in Git.

Likely follow-ups

- Why does GitOps handle drift better?
- How do you wait for a rollout in a playbook?

kubernetes.core provides k8s, k8s_info and helm modules. Use Ansible for cluster bootstrap, namespaces, RBAC, base charts and cross-system orchestration, and GitOps controllers for continuous reconciliation of applications. Use wait conditions in playbooks.

```
- kubernetes.core.k8s:
    state: present
    src: manifests/namespace.yaml
    apply: true
    wait: true
```

```
pip install kubernetes
```

Interview tip: Say Ansible bootstraps clusters; GitOps reconciles apps.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q84 How is Ansible used for network device automation?

Cloud/network/K8s

🎙 Spoken answer · ~63 s

Ansible can configure switches, routers and firewalls without agents because most devices are managed through their CLI or an API. Collections exist per vendor and platform, such as cisco.ios, arista.eos, junipernetworks.junos and others, plus ansible.netcommon and ansible.utils for common functions. Connections use network_cli over SSH, netconf, httpapi or the platform's API, and tasks run on the control node, which logs in to the device. Modules come in two kinds: resource modules manage a configuration area, such as interfaces or VLANs, declaratively with states like merged, replaced, overridden, deleted, gathered and rendered, and command or config modules for raw lines. Good practice includes gathering and backing up the running config before changes, using check mode and diff where supported, templating configuration from data models in Jinja2, validating with parsed command output, rolling out in small batches, and keeping the source of truth in Git. Idempotent resource modules reduce the risk of unintended changes.

Likely follow-ups

- What do merged, replaced and overridden mean?
- How do you back up device configs with Ansible?

Vendor collections provide network modules over network_cli, netconf or httpapi from the controller. Prefer idempotent resource modules with states like merged and replaced, back up configs first, template from data models, check and roll out gradually.

```
- cisco.ios.ios_vlans:
    config:
      - vlan_id: 100
        name: app
    state: merged
```

```
- cisco.ios.ios_config:
    backup: true
```

Interview tip: Say resource modules with states, plus a config backup first.

📚 Official docs

- Ansible: Network automation
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q85 How do you manage Windows hosts with Ansible?

Cloud/network/K8s

🎙 Spoken answer · ~61 s

Windows hosts are managed from a Linux or macOS control node. The connection is WinRM by default, or SSH with the OpenSSH server on newer Windows, and the ansible.windows and community.windows collections provide modules such as win_feature, win_package, win_service, win_user, win_registry and win_shell, which run PowerShell. WinRM needs setting up the listener, usually over HTTPS with a certificate, and an authentication method: Kerberos for domain environments, which is preferred, NTLM or CredSSP when needed, and basic only over HTTPS for local accounts. Variables set the connection type, port, transport and the certificate validation behaviour. Practices include using the dedicated modules instead of win_shell so that tasks are idempotent, careful handling of reboots with the win_reboot module, using become with runas for elevated or different-user tasks, and long-running tasks with async. Common issues are WinRM firewall rules, time skew in Kerberos, double-hop authentication and execution policy settings.

Likely follow-ups

- Why prefer Kerberos for domain machines?
- What is the double-hop problem?

Use WinRM over HTTPS or SSH with the ansible.windows collections, Kerberos for domain hosts, dedicated modules for idempotency, win_reboot for restarts and become runas for elevation. Watch for firewall, time skew and double-hop issues.

```
ansible_connection: winrm
ansible_winrm_transport: kerberos
ansible_port: 5986
```

```
- ansible.windows.win_feature:
    name: Web-Server
    state: present
```

Interview tip: Say Kerberos over HTTPS and real modules, not win_shell.

📚 Official docs

- Ansible: Windows
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q86 How do you use Ansible to configure OpenStack or VMware infrastructure and virtual machines?

Cloud/network/K8s

🎙 Spoken answer · ~63 s

For OpenStack, the openstack.cloud collection provides modules for servers, networks, subnets, security groups, volumes, images and projects, authenticating through clouds.yaml, with an inventory plugin that builds groups from server metadata. For VMware, community.vmware and vmware.vmware offer modules for creating VMs from templates, cloning, snapshots, customisation and vCenter operations, with the vmware_vm_inventory plugin. These are suitable for repeatable provisioning of lab and tenant environments, standard onboarding tasks and operations like snapshots before patching. For large or long-lived cloud definitions I still prefer Terraform, which tracks state and handles dependencies and destruction better, while Ansible fills in what happens inside the machine, such as joining a domain, installing agents and configuring services. Care points: modules need the SDK libraries on the controller, API rate limits, waiting for operations to finish with the wait options, and tagging resources so that dynamic inventory groups are accurate. Kolla-Ansible itself is an example of Ansible deploying OpenStack.

Likely follow-ups

- Which tool tracks lifecycle better: Terraform or Ansible?
- How do you authenticate to OpenStack from a playbook?

Use openstack.cloud with clouds.yaml, and community.vmware for vCenter, for provisioning, snapshots and operations, with inventory plugins for dynamic groups. Prefer Terraform for long-lived resource state and Ansible for in-guest configuration. Install SDKs and handle waits and tags.

```
- openstack.cloud.server:
    cloud: prod
    name: app01
    image: ubuntu-22.04
    flavor: m1.medium
    network: tenant-net
    wait: true
```

```
ansible-galaxy collection install openstack.cloud community.vmware
```

Interview tip: Say Ansible configures the guest, Terraform holds lifecycle state.

📚 Official docs

- Ansible: Cloud guides
- Ansible documentation
- OpenStack documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q87 How do you build golden images with Packer and Ansible?

Cloud/network/K8s

🎙 Spoken answer · ~69 s

Packer builds machine images for clouds and hypervisors, and its Ansible provisioner runs a playbook inside a temporary build instance, so the image contains the baseline configuration. The flow is: Packer starts a base image, connects, runs the playbook, which installs packages, applies hardening, configures agents and removes secrets and caches, then Packer shuts down the instance and saves the image. The benefits of this immutable approach are speed at boot because nothing needs configuring, consistency across the fleet, and smaller drift, with updates done by building a new image and replacing instances rather than patching in place. I reuse the same roles for image baking and for any runtime changes, so there is one source of truth, and test the image in CI by launching it and running checks such as InSpec or Testinfra. Details to handle: clean up SSH keys and machine IDs, avoid baking secrets, version and tag images, scan them for vulnerabilities, and schedule regular rebuilds to pick up security patches.

Likely follow-ups

- What must be cleaned before saving an image?
- How do you test a baked image?

🗺 Golden image



Packer launches a build instance, runs an Ansible playbook with its provisioner, cleans secrets and caches and saves the image. Reuse roles, test with InSpec or Testinfra, version and scan images and rebuild regularly for patches.

```
provisioner "ansible" {
  playbook_file = "playbooks/baseline.yml"
}
```

```
packer build -var-file=prod.pkrvars.hcl image.pkr.hcl
```

Interview tip: Say bake once, replace instead of patching.

📚 Official docs

- Ansible documentation
- Packer documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q88 How do you deploy and configure monitoring and logging agents consistently across a fleet?

Cloud/network/K8s

🎙 Spoken answer · ~68 s

I package this as a role per agent, such as node exporter, a log shipper or a vendor agent, with defaults for versions, ports and endpoints and templates for configuration. The role installs from a trusted package repository or downloads a pinned release with a checksum, creates a dedicated system user, installs a systemd unit, opens required firewall ports, and starts and enables the service, with a handler for reload on configuration change. Inventory groups determine which configuration applies, for example different scrape labels per environment, and secrets such as API keys come from vault or a secrets manager. For Prometheus, file-based service discovery files can be generated from the inventory, so new hosts appear in monitoring automatically. After deployment, the playbook verifies the agent by hitting its local endpoint with the uri module and fails if it does not respond. Updates roll out in batches with serial. The same roles are included in image builds, so new instances come up monitored.

Likely follow-ups

- How do you verify an agent after install?
- How do you generate Prometheus targets from the inventory?

Create a role per agent with pinned, checksummed downloads, a dedicated user, systemd unit, templated config, handlers and a health check, apply by inventory group, keep secrets in vault, generate scrape targets from inventory and roll out in batches.

```
- ansible.builtin.uri:
    url: http://localhost:9100/metrics
    status_code: 200
  retries: 5
  delay: 3
```

```
- ansible.builtin.get_url:
    url: "{{ node_exporter_url }}"
    checksum: "sha256:{{ node_exporter_sha }}"
    dest: /tmp/ne.tgz
```

Interview tip: Say verify the agent after install and roll out in batches.

📚 Official docs

- Argo CD: ApplicationSet

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q89 How would you automate a zero-downtime application deployment with Ansible?

Cloud/network/K8s

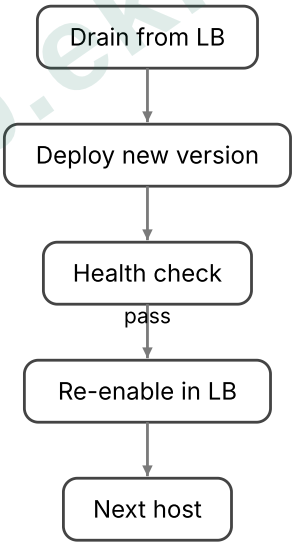
🎙 Spoken answer · ~70 s

I design it around load balancer control and health checks. The play targets the app servers with serial, so only a small batch is out at a time, and max_fail_percentage stops the rollout if failures appear. For each host: delegate a call to the load balancer or service discovery to drain the host and wait for connections to finish; stop or deregister the app; deploy the new version by unpacking a release into a versioned directory and switching a symlink, or pulling a container image and restarting it; run database migrations once with run_once, ideally backward compatible so old and new code can coexist; start the service; wait for the port and a health endpoint to respond with the uri module and retries; then add the host back to the load balancer. A rescue block rolls back to the previous release on failure. Keep several previous releases on disk for fast rollback, record the deployed version, and notify on completion. The first batch acts as a canary.

Likely follow-ups

- Why must migrations be backward compatible?
- How does a symlink-based release help rollback?

🗺 Zero downtime



Use serial batches, drain each host from the load balancer with a delegated task, deploy a versioned release and switch a symlink or image, run migrations once, wait for health checks, return the host, and use rescue for rollback with a canary first batch.

```
- ansible.builtin.file:
    src: "/opt/app/releases/{{ version }}"
    dest: /opt/app/current
    state: link
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- ansible.builtin.uri:
    url: http://localhost:8080/health
  register: h
  until: h.status == 200
  retries: 20
  delay: 3
```

Interview tip: Say drain, deploy, health check, return, with a rollback path.

📚 **Official docs**

- Ansible documentation

Q90 When would you choose Ansible over Helm, Kubernetes operators or GitOps tools?

Cloud/network/K8s

🎙 **Spoken answer · ~63 s**

I choose by what is being managed and how state should be reconciled. For workloads inside Kubernetes, Helm packages and versions applications, operators encode domain knowledge for stateful software, and GitOps controllers such as Argo CD or Flux continuously reconcile the cluster with Git, detecting and correcting drift. Ansible does not run as a continuous controller, so it is weaker at that job. Ansible is the better choice for what is outside the cluster or crosses system boundaries: configuring the operating system of nodes, bootstrapping clusters, managing network devices, databases, DNS, certificates and load balancers, orchestrating ordered multi-system procedures like failover or upgrades, and gluing together tools in runbooks. Often they combine: Ansible bootstraps the cluster and installs the GitOps controller, then Git takes over the applications. Where a team already has strong Ansible skills and few clusters, kubernetes.core can handle simple deployments, but I avoid recreating a reconciliation loop by cron.

Likely follow-ups

- What does continuous reconciliation provide that Ansible lacks?
- How would Ansible and Argo CD split responsibilities?

Use Helm, operators and GitOps controllers for continuous reconciliation of in-cluster applications. Use Ansible for node OS, cluster bootstrap, network devices, databases and ordered multi-system procedures, often handing off to GitOps after bootstrap.

```
ansible-playbook bootstrap-cluster.yml
```

```
kubectl -n argocd get applications
```

Interview tip: Say Ansible for the outside and orchestration, GitOps for in-cluster state.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- OpenGitOps principles
- Helm documentation
- Ansible documentation

Q91 How do you use ansible-lint and what does it catch?

Testing & CI/CD

🎙 **Spoken answer · ~66 s**

Ansible-lint analyses playbooks, roles and collections for correctness, style and risky patterns, and it is a core quality gate. It checks YAML structure, naming and fully qualified collection names, uses of command or shell where a module exists, missing changed_when on commands, task names missing, tasks that are not idempotent, deprecated syntax, risky file permissions, missing no_log on secret handling, unquoted octal modes, and more, with rule identifiers you can reference. Profiles such as min, basic, moderate, safety, shared and production group rules in increasing strictness, so teams can adopt progressively. Configuration in a lint config file selects the profile, skips or warns on selected rules with justification, and excludes paths. Many findings can be auto-fixed with the fix option. I run it in a pre-commit hook for fast feedback and in CI as a required check on pull requests, along with yamllint. I record exceptions in comments or the config file, so skipped rules are deliberate and reviewed.

Likely follow-ups

- What is a lint profile?
- How do you skip a rule responsibly?

ansible-lint catches style, naming, FQCN, risky command and shell use, missing changed_when, permissions, deprecated syntax and secret handling issues. Choose a profile, auto-fix where possible, run it in pre-commit and CI, and document any skipped rules.

```
ansible-lint --profile production playbooks/
```

```
ansible-lint --fix
```

Interview tip: Say adopt profiles progressively and make it a required CI check.

📚 **Official docs**

- ansible-lint
- Ansible documentation
- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q92 How does Molecule test Ansible roles, and what does a good test cover? Testing & CI/CD

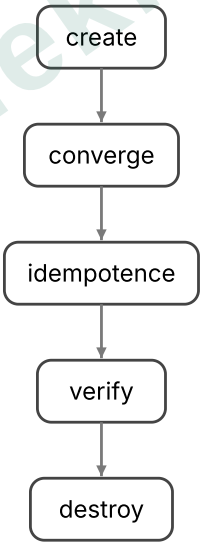
🎙 Spoken answer · ~67 s

Molecule is a framework for testing roles and collections in disposable environments. A scenario defines the platforms to create, usually containers via Podman or Docker, or VMs through Vagrant or a cloud driver, the converge playbook that applies the role, an optional prepare step, and the verify playbook that asserts results. The default sequence is create, prepare, converge, idempotence, verify and destroy. The idempotence step runs converge again and fails if anything changes, which directly enforces idempotency. Verification can be written in Ansible itself with assert tasks or with Testinfra or InSpec. A good test covers the supported operating systems through a platform matrix, default variables and a few important variable combinations, upgrades from a previous state, service state and listening ports, file contents and permissions, and failure paths. Tests should be fast, so I use lightweight containers where systemd is not required, and VMs only when kernel or init behaviour matters. Molecule runs in CI on every pull request.

Likely follow-ups

- What does the idempotence step check?
- When do containers not suffice for role testing?

🗺 Molecule sequence



Molecule runs scenarios that create disposable platforms, converge the role, check idempotence, verify with Ansible assertions, Testinfra or InSpec and destroy. Test the OS matrix, variable combinations, upgrades and failure paths, and run it in CI.

```
molecule init scenario default -r nginx -d podman
```

```
molecule test
```

Interview tip: Say the idempotence step makes the check automatic.

certs.eknathalabs.com
InterviewHub · interviewhub.eknathalabs.com

📚 **Official docs**

- Ansible: Roles
- Ansible documentation
- Molecule documentation

Q93 What does a CI/CD pipeline for Ansible code look like?

Testing & CI/CD

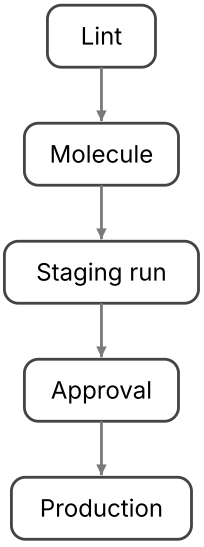
🎙 **Spoken answer · ~62 s**

The pipeline moves changes through increasing confidence. On every pull request: install pinned dependencies from requirements files or the execution environment image, run yamllint and ansible-lint, syntax-check playbooks, run Molecule for changed roles, scan for secrets, and run a check-mode and diff run against a staging inventory, posting the output to the pull request for reviewers. After merge, the pipeline runs against staging for real, executes smoke tests, and then promotes to production behind an approval, ideally by launching a job template in AWX or the controller using the same code revision, so that credentials stay in the controller. Versioning uses tags or release branches, and collections and roles are published to a private hub. The pipeline records artifacts such as lint reports and run logs. Failures notify the team. Protective measures include branch protection, required reviews for production roles, separate credentials per environment, and avoiding secrets in logs.

Likely follow-ups

- Why run check mode against staging in a pull request?
- How do you promote the same revision to production?

🗺 **Ansible pipeline**



On pull requests run lint, syntax checks, Molecule, secret scans and check mode with diff against staging. After merge deploy to staging, smoke test, then promote the same revision to production behind approval via the controller, with branch protection and separate credentials.

interviewhub.eknathalabs.com

```
ansible-lint && ansible-playbook site.yml --syntax-check
```

```
ansible-playbook -i inventories/staging site.yml --check --diff
```

Interview tip: Say promote the same revision through environments.

📚 **Official docs**

- Ansible documentation
- Molecule documentation

Q94 How do you structure an Ansible repository for multiple teams and environments?

Testing & CI/CD

🎙 **Spoken answer · ~63 s**

I aim for clarity and separation of concerns. A common layout has a top-level ansible.cfg and requirements.yml, an inventories directory with one subdirectory per environment holding hosts or dynamic sources and group_vars, a playbooks directory with small entry-point playbooks per purpose, a roles directory for local roles or, better, roles packaged into versioned collections, and tests and documentation. Playbooks stay thin, listing roles and variables, while roles contain logic. Environment differences live in inventory variables only, never in branches, so the same code runs everywhere. Shared roles used by several teams are published as collections in a private hub, with semantic versions, so consumers upgrade deliberately. A monorepo works for a small team and eases changes across layers, while separate repositories per collection suit larger organisations with independent release cycles. I add CODEOWNERS to route reviews, a README with run instructions, and a consistent naming convention for roles, variables and tags.

Likely follow-ups

- Why avoid branches per environment?
- When do you prefer separate repos per collection?

Use a top-level config and requirements, an inventory directory per environment with group_vars, thin playbooks, logic in roles or versioned collections, and environment differences only in variables. Use CODEOWNERS and conventions, and split into repos when teams need independent releases.

```
tree -L 2
.
├── ansible.cfg
├── requirements.yml
├── inventories/{prod,staging}
├── playbooks/
└── collections/
```

```
git log --oneline -- roles/nginx
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say one code base, environment differences in inventory only.

📚 **Official docs**

- Ansible documentation

Q95 How do you build and use ansible-navigator and a local development workflow that matches production?

Testing & CI/CD

🎙 **Spoken answer · ~66 s**

The goal is that what works on my laptop works in the pipeline and on the controller. I containerise the toolchain: define an execution environment with ansible-builder containing the exact ansible-core, collections, Python libraries and system packages, then run playbooks locally with ansible-navigator, which launches the playbook inside that image, with options to mount the project and credentials. Navigator offers an interactive text UI to explore plays, tasks and results, a stdout mode for scripts, and replay of previous runs from saved artifacts, which helps debugging. The same image is used in CI and registered in the controller, so version drift disappears. Locally I add pre-commit hooks for lint, run Molecule for roles, and use a staging inventory or containers for quick trials. For editors, the Ansible extension provides schema validation and completion. Developers pull the latest image regularly, and the image tag is recorded in the repository, so a change in dependencies is reviewed like any other change.

Likely follow-ups

- What does ansible-navigator replay do?
- How does a shared image prevent works-on-my-machine issues?

Build an execution environment, run locally with ansible-navigator against it, reuse the same image in CI and the controller, add pre-commit lint and Molecule, and record the image tag in the repository.

```
ansible-navigator run site.yml -i inventories/staging --eei registry.example.com/ee-platform:1.3 --mode stdout
```

```
ansible-navigator replay artifact.json
```

Interview tip: Say the same image everywhere removes drift.

📚 **Official docs**

- Ansible: Developing modules
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q96 How do you promote changes across environments and roll back safely? Testing & CI/CD

🎙 Spoken answer · ~69 s

Promotion uses the same code revision with different inventory, so the only difference between environments is data. A change moves dev, staging, then production, each with automated checks, and production with approvals, change windows and a limited first batch. For rollback, I prepare in advance because Ansible does not have a built-in undo. Options: redeploy the previous tag of the playbooks and variables, which works when tasks are idempotent and converge to the old desired state; keep previous artefact versions on the host, such as release directories with a symlink, so rollback is switching back; use block with rescue to automatically revert on a failed deployment; and take snapshots or backups before risky changes such as database schema or kernel updates. Changes should be backward compatible where possible. I rehearse rollback in staging and keep it as a documented, tested playbook with its own tag, since a rollback that has never been run tends to fail when needed. All runs are logged with the revision identifier.

Likely follow-ups

- Why does Ansible not have a built-in undo?
- What makes a change safe to roll back by redeploying?

Promote one revision through environments by changing only the inventory, with approvals in production. Prepare rollback with previous tags, release directories and symlinks, rescue blocks and snapshots, rehearse it in staging and record the revision per run.

```
git checkout v1.4.1 && ansible-playbook -i inventories/prod deploy.yml -e version=1.4.1
```

```
ansible-playbook rollback.yml --tags rollback
```

Interview tip: Say rehearse the rollback; untested rollbacks fail.

📚 Official docs

- Ansible: Playbook guide
- Kubernetes: Deployments

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q97 How do you detect and handle configuration drift with Ansible?

Testing & CI/CD

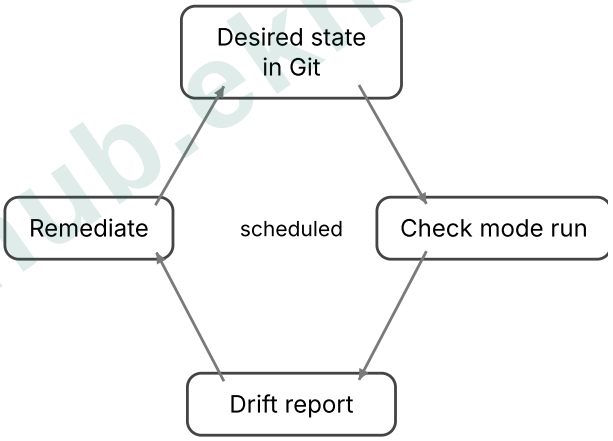
🎙 Spoken answer · ~65 s

Drift is when real systems diverge from the desired state, through manual changes, failed runs or other tools. Ansible detects it by running the same playbooks in check mode with diff and reporting changes: any task that would change something indicates drift. I schedule this regularly from the controller, for example nightly, publish the results to a dashboard or ticket, and alert on unexpected changes in critical files or services. Because it relies on tasks that support check mode and are idempotent, I keep the roles clean and avoid always-changed tasks, which would make the report noisy. The response depends on policy: for low-risk settings, enforce automatically by running in apply mode on a schedule, perhaps with ansible-pull; for sensitive systems, require review so unexpected changes, which might be incidents, are investigated first. The most effective remedy is preventing manual changes through access control and audit, so drift becomes the exception and each case is understood.

Likely follow-ups

- Why must roles be idempotent for drift reports?
- When would you not auto-correct drift?

🗺 Drift loop



Schedule check-mode and diff runs from the controller and report any would-change tasks as drift. Keep roles idempotent to avoid noise, auto-enforce low-risk settings, review drift on sensitive systems, and reduce manual changes through access control.

```
ansible-playbook site.yml --check --diff -i inventories/prod | tee drift-$(date +%F).log
```

```
awx job_templates launch drift-check --monitor
```

Interview tip: Say drift reports need idempotent roles to be trustworthy.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Infrastructure drift (Terraform: plan -refresh-only)
- Ansible documentation

Q98 How do you migrate shell scripts and manual runbooks into Ansible?

Testing & CI/CD

🎙 **Spoken answer · ~68 s**

I migrate incrementally and keep the behaviour the same at first. Start by listing what the script does and what state it assumes. Then, step by step, replace commands with modules that express the desired state: package, user, file, template, service, lineinfile or blockinfile and others, which gives idempotency and check mode. Where a command has no module, wrap it with creates, removes, changed_when and failed_when. Move hard-coded values to variables, and secrets to vault or a secrets manager. Break the result into roles when logic is reusable, and add handlers for restarts. Test equivalence by running the old script and the new playbook against test machines and comparing resulting state, then in staging. Add Molecule tests or smoke checks, put the code under version control with review, and document how to run it, preferably through the controller with RBAC and logging. Finally, retire the script to avoid two sources of truth. The result is repeatable, visible and safer than ad-hoc commands.

Likely follow-ups

- How do you verify the playbook matches the script's result?
- When is a command task acceptable?

Inventory what the script does, replace commands with state-based modules, wrap leftovers with creates and changed_when, move values to variables and secrets to vault, test equivalence in staging, add tests and review, run through the controller and retire the script.

```
ansible-playbook migrate.yml --check --diff -l test01
```

```
diff <(ssh old 'rpm -qa | sort') <(ssh new 'rpm -qa | sort')
```

Interview tip: Say compare end states, then retire the script.

📚 **Official docs**

- GNU Bash reference manual
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q99 How do you test playbooks end to end, beyond role-level Molecule tests? Testing & CI/CD

🎙 Spoken answer · ~69 s

Roles tested in isolation do not prove that the whole system works, so I add integration tests that exercise playbooks the way production does. The pipeline creates an ephemeral environment that resembles production, using containers, VMs or a short-lived cloud account provisioned by Terraform, applies the full site playbook against an inventory that mirrors real groups, and then runs verification. Verification includes assertion tasks within the playbook, such as checking service status, listening ports and HTTP responses with the uri module, and external tools like Testinfra, InSpec or Goss for state checks, plus a functional smoke test of the application. The idempotence check runs the playbook a second time and expects zero changes. For risky operations, such as upgrades or failover, I test the procedure on a copy and measure how long it takes. After tests, the environment is destroyed to control cost. Results and logs are kept as pipeline artifacts, and flaky tests are fixed quickly, because ignored failures destroy trust in the pipeline.

Likely follow-ups

- What do you verify besides service status?
- How do you keep ephemeral test environments cheap?

Create an ephemeral production-like environment, apply the full playbook against a mirrored inventory, verify with assertions, Testinfra, InSpec or Goss and a smoke test, re-run to check idempotence, test risky procedures, destroy the environment and keep artifacts.

```
ansible-playbook -i inventories/ci site.yml && ansible-playbook -i inventories/ci site.yml
| grep -q 'changed=0'
```

```
goss -g goss.yaml validate
```

Interview tip: Say test the whole playbook, not just the roles.

📚 Official docs

- Ansible: Playbook guide
- Ansible: Roles
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q100 How do you version, release and document roles and collections for other teams?

Testing & CI/CD

🎙 **Spoken answer · ~65 s**

Treat shared automation like a product. Each collection or role has a semantic version: patch for fixes, minor for backward compatible features and major for breaking changes such as renamed variables, and a changelog generated from fragments, for example with antsibull-changelog, so consumers know what changed. Releases are cut from tags by the pipeline, which builds the artefact with ansible-galaxy collection build and publishes it to the private Automation Hub or Galaxy server after tests pass. Consumers pin version ranges in requirements.yml and upgrade deliberately, with a deprecation policy that announces removals one or two releases ahead. Documentation includes a README with purpose, supported platforms, variables and examples, module and plugin docs written in the documentation strings so ansible-doc works, and sample playbooks. I add argument specs for role inputs, which both validate and document them. Ownership is clear through CODEOWNERS, and a channel or issue tracker handles questions. Regular security updates keep dependencies current.

Likely follow-ups

- What counts as a breaking change in a role?
- How do you announce deprecations?

Use semantic versions, changelog fragments, tagged releases built and published to a private hub by the pipeline, version-range pinning by consumers, a deprecation policy, documentation in READMEs and argument specs, and clear ownership.

```
antsibull-changelog release --version 1.3.0
```

```
ansible-galaxy collection build && ansible-galaxy collection publish *.tar.gz --server
private_hub
```

Interview tip: Say semver, changelog, pinned consumers, deprecation notice.

📚 **Official docs**

- Ansible: Roles

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q101 Which Ansible command-line tools do you use besides ansible-playbook, and what does each do?

Core concepts

🎙 **Spoken answer · ~60 s**

Beyond ansible-playbook, several tools make daily work faster. Ansible-doc shows module and plugin documentation offline, including examples and parameters, which is quicker than searching the web. Ansible-inventory renders the parsed inventory as a list, graph or JSON, which is the best way to check that groups, host variables and dynamic sources resolve the way I expect. Ansible-config dump with the changed-only flag shows which settings differ from defaults and where they came from. Ansible-galaxy installs roles and collections from requirements files and scaffolds new role skeletons. Ansible-vault encrypts, views and edits secrets. Ansible-console gives an interactive shell against a host pattern, and plain ansible runs ad-hoc modules. Ansible-lint and ansible-navigator are separate but part of the same workflow. Together they let me inspect, document, validate and execute without leaving the terminal, and I use inventory and config output first whenever behaviour surprises me.

Likely follow-ups

- How do you see where a setting came from?
- How do you check that an inventory resolves correctly?

Use ansible-doc for offline module help, ansible-inventory to verify parsed inventory, ansible-config dump for effective settings, ansible-galaxy for dependencies, ansible-vault for secrets, ansible-console for interactive runs, and ansible-lint before committing.

```
ansible-doc -s ansible.builtin.copy
```

```
ansible-inventory -i inventory --graph --vars
```

```
ansible-config dump --only-changed
```

Interview tip: Say inspect first with inventory and config, then run.

📚 **Official docs**

- Python: argparse
- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q102 What YAML pitfalls commonly break Ansible playbooks?

Core concepts

🎙 Spoken answer · ~69 s

Playbooks are YAML, so a few syntax traps cause most parse problems and strange behaviour. Indentation must be consistent spaces, never tabs, and list items under a key must line up correctly. Unquoted values can be reinterpreted: yes, no, on and off become booleans, a version such as 1.10 becomes the float 1.1, a leading zero in a file mode may be read as octal, and a value containing a colon followed by a space or starting with a curly brace needs quotes, because braces begin a Jinja expression or a YAML mapping. A string that begins with a Jinja expression must be quoted as a whole. Multi-line strings use the pipe style to preserve newlines or the greater-than style to fold them. Duplicate keys silently override earlier ones in some parsers. Anchors and aliases reduce repetition but can confuse readers. I quote file modes and versions, use the yamllint and ansible-lint tools in the editor and CI, and run a syntax check before every commit.

Likely follow-ups

- Why must a file mode be quoted?
- How do you write a multi-line string?

Use spaces only, quote booleans-like words, versions, file modes and anything starting with a brace or containing a colon-space, use pipe or greater-than for multi-line text, avoid duplicate keys, and run yamllint, ansible-lint and syntax-check.

```
mode: "0644"
```

```
version: "1.10"
```

```
ansible-playbook site.yml --syntax-check && yamllint .
```

Interview tip: Say quote modes, versions and anything brace-leading.

📚 Official docs

- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q103 How does Ansible pick and use the Python interpreter on managed hosts? Core concepts

🎙 Spoken answer · ~70 s

Most modules are Python programs that Ansible copies to the target, runs and removes, so the target needs a usable interpreter, with the exception of raw and a few modules that do not need one. Ansible discovers the interpreter automatically on first contact by probing a list of common paths, and the discovery setting controls whether it prefers an auto, a legacy or a silent mode; a warning appears if it falls back. Problems arise on minimal images, containers and older systems where only Python 2 or a differently named binary exists, or where the system Python lacks libraries a module needs, such as the apt bindings. The fix is to set ansible_python_interpreter per host or group to an explicit path, often a virtual environment for modules with extra dependencies, and to install Python first using the raw module on bare hosts. Pinning the interpreter in inventory makes runs reproducible and silences discovery warnings, and I record it with the group variables so it is reviewed like code.

Likely follow-ups

- What is the raw module for in bootstrap?
- Where do you set the interpreter path?

Ansible auto-discovers the target Python; for minimal images or special libraries set ansible_python_interpreter explicitly in inventory, bootstrap Python with the raw module on bare hosts, and use a virtualenv when modules need extra packages.

```
ansible_python_interpreter=/usr/bin/python3
```

```
ansible bare -m raw -a 'apt-get install -y python3' -b
```

Interview tip: Say pin the interpreter in group vars.

📚 Official docs

- Ansible documentation
- Python Documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q104 What happens under the hood when Ansible runs a task on a remote host? Core concepts

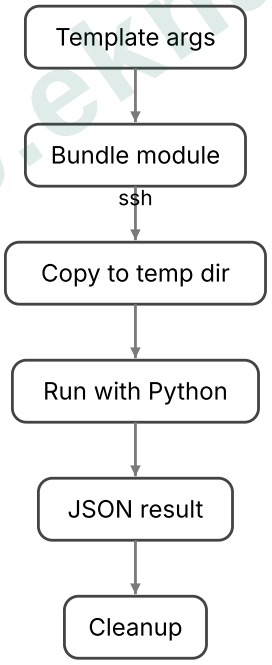
🎙 Spoken answer · ~68 s

For a typical module task, the controller first evaluates variables and templates the task arguments for each host. It then builds the module payload, a Python file bundled with its helper code and the arguments, and connects over the configured transport, usually SSH. Ansible creates a temporary directory on the target, copies the payload there, makes it executable, runs it with the chosen interpreter, and the module prints a JSON result containing fields like changed, failed and any returned data. Ansible reads that JSON, registers it if asked, fires handlers, deletes the temporary files and moves to the next task. With pipelining enabled, it streams the module over the existing connection instead of copying a file, saving round trips. Tasks run in lockstep across the hosts in each batch by default. Understanding this explains why each task costs a connection round trip, why the target needs Python, why temporary directory permissions matter with become, and why the controller does all the logic.

Likely follow-ups

- What does pipelining change?
- What does the module return to the controller?

🗺 Task execution



The controller templates arguments, bundles the module, copies it to a temp directory over SSH, runs it with Python, reads the JSON result, then cleans up; pipelining streams the module to avoid the copy. The controller does all the logic.

```
ansible all -m ping -vvv
```

```
ANSIBLE_KEEP_REMOTE_FILES=1 ansible web -m setup
```

certs.eknathalabs.com

Interview tip: Say module copy, execute, JSON back, cleanup.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

Q105 What conventions make an Ansible playbook readable and maintainable?

Core concepts

🎙 **Spoken answer · ~62 s**

Readable automation follows a few consistent habits. Every play and task has a descriptive name that states the intent, since names appear in output and logs. I use fully qualified collection names so modules are unambiguous, prefer dedicated modules over command or shell, and keep each task doing one thing. Variables are prefixed with the role name, such as nginx_port, to avoid collisions, and defaults are documented. Secrets are never inline. Tasks use state parameters declaratively, tags sparingly, and handlers for restarts. Complex conditions are simplified with named variables, and templates stay logic-light. Repositories keep inventories, group variables, roles and playbooks in predictable directories, with a README and requirements file. Playbooks stay short and call roles, while ansible-lint and yamllint enforce style automatically in CI. Idempotency and check mode support are treated as requirements. Code review looks for these points, and consistent formatting makes diffs small and reviewable.

Likely follow-ups

- Why prefix role variables?
- Why prefer dedicated modules over shell?

Name every task, use fully qualified module names, prefer dedicated modules, keep tasks single-purpose, prefix role variables, keep secrets out, put logic in roles and templates, keep playbooks thin, and enforce style with ansible-lint and yamllint.

```
- name: Ensure nginx is installed
  ansible.builtin.package:
    name: nginx
    state: present
```

```
ansible-lint --profile production
```

Interview tip: Say names, FQCN, one thing per task, lint in CI.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q106 How do you use until, retries and wait_for to handle tasks that need time to succeed?

Playbooks & modules

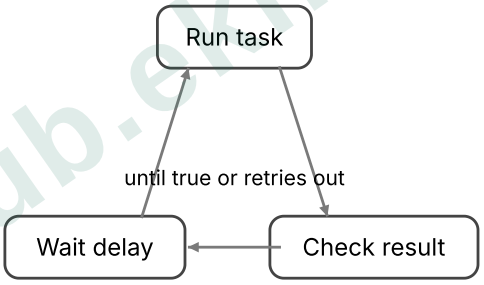
🎙 Spoken answer · ~68 s

Operations often depend on something becoming ready: a service starting, a port opening or an API responding. Wait_for blocks until a port is open or closed, a file appears or a string shows up in a file, with a timeout and a delay, and is ideal after restarts. The until keyword turns any task into a retry loop: it re-runs the task until a condition on the registered result is true, with retries and delay controlling how many attempts and how long to pause between them. A common pattern is polling a health endpoint with the uri module until the status is 200. Timeouts should be realistic, and failure after the final attempt is a real failure that should stop the play or trigger a rescue block. I avoid fixed pause tasks, because they waste time on fast systems and are too short on slow ones. Combined with serial rollouts, these waits ensure the next batch only starts after the previous one is healthy.

Likely follow-ups

- Why avoid a fixed pause?
- What happens when retries run out?

🗺 Retry until ready



Use wait_for for ports and files, and until with retries and delay for polling APIs; set realistic timeouts, let the final failure stop the play, and avoid fixed pauses. Combine with serial so each batch must be healthy first.

```
- ansible.builtin.uri:
    url: http://localhost:8080/health
  register: r
  until: r.status == 200
  retries: 20
  delay: 5
```

```
- ansible.builtin.wait_for:
    port: 8080
    timeout: 120
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say poll for readiness, never sleep blindly.

📚 **Official docs**

- Microsoft: Cloud design pattern – Retry
- Ansible: Playbook guide
- Ansible: Error handling

Q107 How do you control loop output and iterate over complex data structures?

Playbooks & modules

🎙 **Spoken answer · ~67 s**

The loop keyword accepts lists, and with loop_control I can tune behaviour. The label option limits what is printed for each item, which keeps output readable and avoids leaking secrets from large dictionaries. Loop_var renames the default item variable, essential in included tasks and nested loops so inner and outer variables do not collide. The pause option slows iterations for rate-limited systems, and index_var gives the position. For dictionaries, the dict2items filter converts a mapping into key and value items. For nested data, the subelements lookup or the product filter iterate over lists of lists, such as users and their keys. Loops can combine with when conditions, but the condition is evaluated per item. Large loops over modules are slow because each item is a separate execution, so where a module accepts a list, such as the package module, I pass the list once instead. Registered results from loops contain a results list, which I process with filters when needed.

Likely follow-ups

- Why rename loop_var?
- How do you loop over a dictionary?

Use loop_control with label for quiet output, loop_var to avoid collisions, pause for rate limits, dict2items for mappings and subelements for nested lists; pass whole lists to modules that accept them instead of looping.

```
loop: "{{ users }}"
loop_control:
  label: "{{ item.name }}"
  loop_var: user
```

```
loop: "{{ app_config | dict2items }}"
```

Interview tip: Say label your loops and rename loop_var in includes.

📚 **Official docs**

- Ansible: Playbook guide

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q108 How do any_errors_fatal and max_fail_percentage protect a rolling operation?

Playbooks & modules

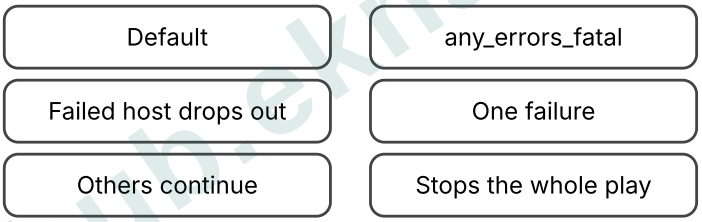
🎙 Spoken answer · ~68 s

By default, if a task fails on one host, Ansible removes that host from the rest of the play and continues with the others. For simple independent servers that is fine, but for coordinated changes it can leave a half-updated fleet. Setting any_errors_fatal on a play makes a failure on any host stop the whole play for all hosts, which suits cluster-wide steps where partial success is dangerous. The max_fail_percentage setting is for batched rollouts: combined with serial, the play aborts if more than the stated percentage of hosts in the current batch fail, so a bad release stops after the first small batch instead of spreading. Both interact with serial, rescue blocks and handlers, so I test the failure path deliberately. I also pair them with pre-checks and post-deploy health checks, so failures are detected quickly rather than silently accepted, and run first against a canary batch of one host. The goal is limiting the blast radius when something goes wrong.

Likely follow-ups

- What is the default behaviour on host failure?
- How do serial and max_fail_percentage work together?

🗺 Failure handling



By default failed hosts drop out while others continue. any_errors_fatal stops the whole play on any failure; max_fail_percentage with serial aborts a rollout when too many hosts in a batch fail. Use a canary batch and health checks.

```
- hosts: web
  serial: [1, 5, 25%]
  max_fail_percentage: 10
  any_errors_fatal: false
```

Interview tip: Say bad releases should stop after the first batch.

📚 Official docs

- Ansible: Playbook guide
- Ansible: Error handling
- Ansible: Strategies and performance

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q109 What are meta tasks and when do you use flush_handlers, end_play, end_host and refresh_inventory?

Playbooks & modules

🎙 **Spoken answer · ~66 s**

Meta tasks control the playbook run itself rather than the target system. Flush_handlers runs any pending handlers immediately instead of waiting until the end of the play, which is needed when a later task depends on a restart or reload that an earlier change triggered, for example restarting a service before a health check. End_play stops the play for all hosts cleanly and is useful for early exit when a precondition shows there is nothing to do. End_host ends the play for the current host only, which suits skipping hosts that fail a guard condition without marking them failed. Refresh_inventory reloads the inventory mid-run, essential after creating new instances with a cloud module so that later plays can target them. Clear_facts and clear_host_errors reset cached state. Meta tasks do not take most task keywords and cannot be looped, and conditions apply per host. I use them sparingly, because they hide flow control, and I document why each is there.

Likely follow-ups

- When must handlers be flushed early?
- Why refresh the inventory after creating servers?

Meta tasks steer the run: flush_handlers applies pending handlers now, end_play and end_host exit early, refresh_inventory reloads hosts created mid-run. Use sparingly and explain why.

```
- ansible.builtin.meta: flush_handlers
```

```
- ansible.builtin.meta: refresh_inventory
```

Interview tip: Say flush handlers before health checks.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Inventory
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q110 How do you develop and debug playbooks efficiently with start-at-task, step, list-tasks and limit?

Playbooks & modules

🎙 Spoken answer · ~65 s

Fast iteration matters when playbooks are long. The list-tasks and list-hosts flags show what would run and where without executing anything, which is a safe review step. The limit option restricts the run to a host pattern, so I can test on one server first. The tags and skip-tags options select subsets of tasks. The start-at-task flag resumes from a named task after fixing a failure, saving time, though skipped earlier tasks may leave variables unset. The step flag asks for confirmation before each task, handy for careful live changes. Check and diff modes preview changes. Verbosity levels reveal connection and variable details, and the retry file or limit with the failed hosts re-runs only hosts that failed. For repeated debugging, a small test inventory and a lab VM or container keep feedback quick. I combine these with ansible-lint and syntax-check before ever touching production, and I avoid relying on start-at-task in pipelines because it hides dependencies.

Likely follow-ups

- What can go wrong with start-at-task?
- How do you re-run only failed hosts?

Use --list-tasks and --list-hosts to preview, --limit for a canary host, --tags to select tasks, --start-at-task and --step during development, and check and diff for dry runs. Re-run failed hosts with a limit; avoid start-at-task in pipelines.

```
ansible-playbook site.yml --list-tasks --limit web01
```

```
ansible-playbook site.yml --limit web01 --step --diff
```

Interview tip: Say test on one host before the fleet.

📚 Official docs

- Ansible: Playbook guide
- Ansible: Developing modules
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q111 What are argument_specs and how do they validate role inputs?

Roles & collections

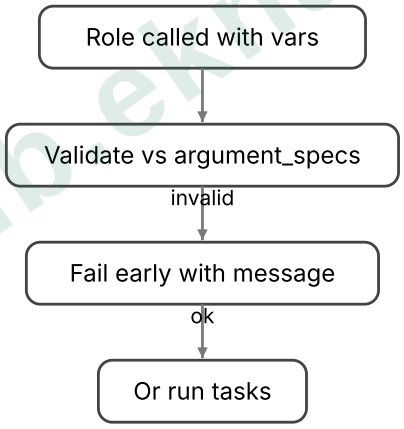
🎙 Spoken answer · ~68 s

Argument specs let a role declare its interface in a meta file: the supported entry points, and for each, the variables it accepts with their types, defaults, allowed choices, whether they are required and descriptions. When the role runs, Ansible validates the supplied variables against the spec before executing tasks, so a missing required value or a wrong type fails immediately with a clear message, instead of failing halfway through with a confusing templating error. The same file serves as machine-readable documentation, and tools can generate role documentation from it. Specs support nested options for dictionaries and lists, and choices for enumerations. I use them for any role shared across teams, because they catch typos and incorrect value types early, and they define a contract that can be versioned. They complement, not replace, assert tasks for cross-field rules such as two options being mutually exclusive, which the spec cannot express. Keeping the spec in sync with defaults is part of code review.

Likely follow-ups

- Where is the spec defined?
- What can a spec not validate?

🗺 Role input validation



Argument specs in meta/argument_specs.yml declare variable types, defaults, choices and required flags per entry point; Ansible validates inputs before tasks run and the file doubles as documentation. Use assert for cross-field rules.

```
argument_specs:
  main:
    options:
      nginx_port:
        type: int
        default: 80
      nginx_state:
        choices: [present, absent]
        required: true
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say fail early with a clear message.

📚 **Official docs**

- Ansible: Roles

Q112 How do you design a role that works across several operating systems?

Roles & collections

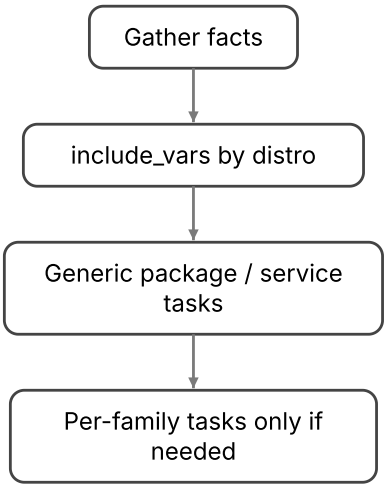
🎙 **Spoken answer · ~68 s**

The aim is one role interface with OS differences hidden inside. I gather facts and load a variables file chosen by distribution or OS family, using include_vars with a first-found lookup so a specific file such as the distribution and major version wins, falling back to the family, then to a default. Those files hold package names, service names, paths and repository settings. Tasks then use the generic package and service modules with variables, so most tasks are identical everywhere. Where behaviour truly differs, I put it in separate task files included by OS family, instead of scattering conditions through the role. Defaults describe the common case and the top-level variables stay OS-neutral. Molecule scenarios test each supported platform with container or VM images, and the role metadata lists supported platforms. I avoid assuming systemd, a package manager or a path unless declared. This keeps the role small, testable and easy to extend with a new platform by adding a vars file.

Likely follow-ups

- How does first-found help?
- How do you test each platform?

🗺 **Multi-OS role**



Load per-distribution or family variable files with a first-found lookup for packages, services and paths, keep tasks generic with package and service modules, isolate truly different logic in per-family task files, and test each platform with Molecule.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- ansible.builtin.include_vars: "{{ lookup('ansible.builtin.first_found', params) }}"
  vars:
    params:
      files:
        - "{{ ansible_distribution }}-{{ ansible_distribution_major_version }}.yml"
        - "{{ ansible_os_family }}.yml"
        - default.yml
      paths: [vars]
```

Interview tip: Say variables per OS, tasks generic.

📚 **Official docs**

- Ansible: Roles

Q113 When do you use template, copy, lineinfile, blockinfile and replace for managing files?

Roles & collections

🎙 **Spoken answer · ~71 s**

Each module fits a different ownership model. Template renders a Jinja file and gives full control of the whole file, so it is the best choice for files that automation owns entirely, and it is idempotent because it compares the result. Copy puts static content or a file in place, and can validate before replacing. Lineinfile ensures a single line exists or is absent, matching with a regular expression, which is suitable for small edits to files owned by someone else, such as a sysctl setting or an SSH option. Blockinfile manages a marked multi-line block, leaving the rest alone. Replace performs regex substitutions across a file. The more of a file you edit in place, the more fragile it becomes, because patterns can match unexpectedly and different runs may produce different results. Wherever possible I prefer a drop-in directory such as a conf.d file written by template, and the validate parameter to check syntax before activating, for example with nginx -t or visudo, so a bad file never goes live.

Likely follow-ups

- Why prefer drop-in files over editing in place?
- What does the validate option do?

Use template for files you fully own, copy for static content, lineinfile and blockinfile for small edits to shared files, and replace for regex substitutions. Prefer drop-in conf.d files and always use validate so bad configs are never activated.

```
- ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/conf.d/10-app.conf
    validate: '/usr/sbin/app --check %s'
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- ansible.builtin.lineinfile:
    path: /etc/sysctl.d/99-tune.conf
    line: 'vm.swappiness=10'
    create: true
```

Interview tip: Say own the whole file when you can; validate always.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Templating (Jinja2)
- Ansible: Error handling

Q114 How do you manage users, groups, SSH keys and sudoers with Ansible safely?

Roles & collections

🎙 **Spoken answer · ~66 s**

Account management is sensitive, so I make it declarative and reviewable. A variable describes users: name, groups, shell, state, SSH public keys and sudo rights. The user and group modules create accounts with explicit state, and the authorized_key module manages keys with the exclusive option when I want the file to contain only approved keys. Sudo rules are written as drop-in files under sudoers.d using the template or copy module with validate set to visudo, so a syntax error never locks everyone out. Removing a user sets state absent with remove, and departing staff are handled by their data being removed from the variable, with exclusive keys or cleanup tasks ensuring they disappear. Passwords, if used at all, are hashed with the password_hash filter and stored in vault. Where possible, central identity such as LDAP or SSO replaces local accounts, and Ansible configures the client. Break-glass accounts are documented and monitored, and changes pass code review before reaching production.

Likely follow-ups

- Why validate sudoers with visudo?
- What does exclusive do for authorized keys?

Describe users in variables, use user, group and authorized_key with exclusive, write sudo drop-ins validated with visudo, hash any passwords and keep them in vault, remove accounts explicitly, and prefer central identity with reviewed changes.

```
- ansible.builtin.copy:
    dest: /etc/sudoers.d/ops
    content: '%ops ALL=(ALL) NOPASSWD: /usr/bin/systemctl'
    mode: '0440'
    validate: 'visudo -cf %s'
```

Interview tip: Say always validate sudoers before activating.

certs.eknathalabs.com

📚 **Official docs**

- OpenSSH manual pages
- sudoers(5) manual
- Ansible documentation

Q115 How do you make roles idempotent and safe in check mode?

Roles & collections

🎙 **Spoken answer · ~68 s**

A role is idempotent when a second run changes nothing. I achieve that by using declarative modules with a state parameter instead of command and shell, and where a command is unavoidable, I guard it with creates or removes, or a prior check task, and set changed_when accurately. Templates and copies are naturally idempotent. I avoid tasks that always report changed, such as timestamps in rendered files, appended lines without matching and unconditional restarts, using handlers instead. For check mode, modules that support it predict changes without applying them. Tasks that must run to produce data for later tasks, like a read-only command, need check_mode set to false so downstream conditions work, and tasks that cannot be simulated are skipped with a condition on the check mode variable. Molecule's idempotence step runs the role twice and fails if the second run reports changes, which makes the rule enforceable in CI. I also test with diff mode, which shows exactly what would change in files.

Likely follow-ups

- How does Molecule check idempotence?
- When do you set check_mode false?

Use declarative modules, guard command and shell with creates, removes and accurate changed_when, avoid always-changing templates, restart through handlers, set check_mode false on read-only discovery tasks, and enforce a no-change second run with Molecule.

```
- ansible.builtin.command: /opt/app/init.sh
  args:
    creates: /var/lib/app/.initialised
```

```
- ansible.builtin.command: app status
  register: st
  changed_when: false
  check_mode: false
```

Interview tip: Say second run must change nothing.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Ansible: Roles

certs.eknathalabs.com

InterviewHub - interviewhub.ekn

Q116 How do you use hostvars, groups and inventory_hostname to build cluster configuration?

Variables & Jinja2

🎙 Spoken answer · ~73 s

Many services need each node to know about its peers, such as a database cluster, a load balancer pool or a monitoring target list. Magic variables make this data-driven. Inventory_hostname is the name of the current host, groups is a dictionary of group names to host lists, and hostvars gives access to any other host's variables and gathered facts, provided facts were collected for them in this run. In a template, I loop over a group and read each peer's address from hostvars, for example to render an upstream block or a cluster member list. The ordering of the group can matter, so I sort it, and I take the first member as a simple leader choice where needed. If facts from other hosts are required, an earlier play must gather them, or I use delegated fact gathering. Relying on hostvars of hosts outside the current play limit can fail with undefined errors, so limits need care. Clear variable naming, such as an explicit service address variable, is better than depending on discovered facts.

Likely follow-ups

- Why can hostvars be undefined?
- How do you pick a leader node deterministically?

Use groups to list peers, hostvars to read each peer's variables and facts, and inventory_hostname for the current node, sorted for stable output. Ensure facts are gathered for those hosts and beware of --limit hiding them.

```
{% for h in groups['db'] | sort %}
server {{ hostvars[h].ansible_host }}:5432;
{% endfor %}
```

Interview tip: Say sort groups and gather facts for peers.

📚 Official docs

- Ansible: Inventory
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q117 How do combine, json_query, selectattr and map help transform data in Ansible?

Variables & Jinja2

🎙 Spoken answer · ~66 s

Real inventories and API responses are nested data, and filters reshape them without custom code. The combine filter merges dictionaries, with recursive and list merge options, which layers defaults and overrides. Selectattr and rejectattr filter a list of dictionaries by an attribute test, for example only enabled services, and map extracts an attribute or applies a filter to each element, so a list of users becomes a list of names. The json_query filter uses JMESPath for more complex queries on JSON, such as pulling instance identifiers from a cloud describe result, and requires the jmespath library on the controller. Other helpers include dict2items and items2dict, zip, flatten, unique, difference and regex_replace. Chains of filters can become unreadable, so I break them into set_fact steps with names, or move heavy transformation into a custom filter plugin. Output is always inspected with the debug module while developing, and I test edge cases such as empty lists, because errors there are common.

Likely follow-ups

- What library does json_query need?
- When do you write a custom filter?

Use combine to merge dicts, selectattr and rejectattr to filter lists, map to extract attributes, json_query with JMESPath for nested JSON, and dict2items for mappings. Split long chains into named set_fact steps and write a custom filter if it gets heavy.

```
- ansible.builtin.set_fact:
    names: "{{ users | selectattr('active') | map(attribute='name') | list }}"
```

```
{{ r.json | json_query('items[?status==`ready`].id') }}
```

Interview tip: Say small named steps beat long filter chains.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q118 How do add_host and group_by create groups dynamically during a run?

Variables & Jinja2

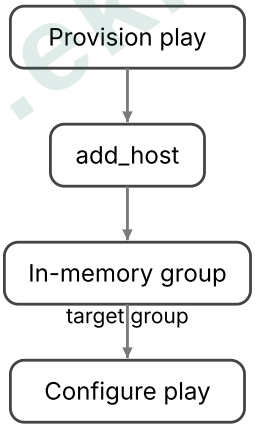
🎙 Spoken answer · ~70 s

Sometimes the targets are not known until the playbook runs. The add_host module adds a host to the in-memory inventory, optionally placing it into groups and setting variables, so a later play can target it. It is typical after creating cloud instances in the first play: register the results, loop over them with add_host, and then configure them in the next play. The host exists only for this run. The group_by module creates groups based on facts or any expression, such as the operating system family or a custom variable, so later plays can target those groups without maintaining them in the inventory. Both run once per batch of hosts and have subtle behaviour with delegation and serial, since add_host is evaluated per host unless run_once is used carefully. For regular needs, a dynamic inventory plugin or inventory refresh through the meta task is often cleaner. Dynamic groups keep playbooks flexible, but they should be documented, because the groups do not appear in the static inventory files.

Likely follow-ups

- When does the added host disappear?
- What does group_by use to decide membership?

🗺 Dynamic groups



add_host injects hosts and groups into the in-memory inventory for later plays, typically after provisioning; group_by creates groups from facts or expressions. Both last only for the run, so prefer dynamic inventory plugins for recurring needs.

```
- ansible.builtin.add_host:
    name: "{{ item.public_ip }}"
    groups: new_web
  loop: "{{ created.instances }}"
```

```
- ansible.builtin.group_by:
    key: "os_{{ ansible_os_family }}"
```

certs.eknathalabs.com

Interview tip: Say add_host is run-scoped; inventory plugins persist.

📚 **Official docs**

- Ansible: Inventory

Q119 What naming conventions and structure do you use for variables to keep large automation manageable?

Variables & Jinja2

🎙 **Spoken answer · ~65 s**

Consistent naming prevents collisions and makes code searchable. Role variables get the role name as a prefix, such as nginx_worker_processes, so two roles never clash, and internal variables used only by the role start with a double underscore or an additional marker. Variables describe meaning, not mechanism, for example app_listen_port instead of port. Booleans read naturally, like app_tls_enabled. Defaults live in the role's defaults directory so they can be overridden easily, while constants that should not change go in the vars directory. Environment-specific values live in group variables, host-specific ones in host variables, and secrets in vault files with a vault prefix, referenced from a normal variable so the encrypted names are discoverable. Complex structures use lists of dictionaries with a stable schema documented in the readme or argument specs. I avoid deep nesting and avoid set_fact for configuration, because it has high precedence and surprises. A short style guide and linting rules keep contributors aligned.

Likely follow-ups

- Why prefix role variables?
- Why a vault_ prefix for secrets?

Prefix role variables with the role name, name by meaning, keep defaults in defaults/, constants in vars/, environment data in group_vars, secrets in vault files referenced by plain variables, avoid deep nesting and document the schema.

```
# group_vars/prod/vars.yml
db_password: "{{ vault_db_password }}"
```

```
# group_vars/prod/vault.yml (encrypted)
vault_db_password: s3cret
```

Interview tip: Say prefix, name by meaning, reference secrets indirectly.

📚 **Official docs**

- Ansible: Variables and precedence

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q120 How do custom facts, facts.d and set_stats work?

Variables & Jinja2

🎙 **Spoken answer · ~68 s**

Custom facts let a host describe itself to Ansible. Files in the local facts directory on the target, usually facts.d under etc/ansible, ending in .fact, can be INI or JSON files or executables that print JSON. When facts are gathered, their content appears under ansible_local, so playbooks can branch on a host's role, version or state recorded there. A common use is storing the deployed application version, which later plays read to decide whether an upgrade is needed. Facts can also be set during a run with set_fact, optionally cached for later runs through the fact caching plugin. Set_stats is different: it records custom statistics for the playbook as a whole, which callback plugins and workflow nodes in AWX can pass along as artifacts to later job templates, so the output of one job becomes input to the next. Gathering must be enabled for local facts to appear, and the setup module with a filter can refresh them mid-play after the file changes.

Likely follow-ups

- Where do local facts appear?
- How do workflow jobs pass data forward?

Drop .fact files or executables in facts.d to expose data under ansible_local, refresh with the setup module, cache facts if needed, and use set_stats to pass artifacts to later jobs in AWX workflows.

```
- ansible.builtin.copy:
    dest: /etc/ansible/facts.d/app.fact
    content: '{"version": "{{ app_version }}"}'
```

```
- ansible.builtin.set_stats:
    data:
      deployed_version: "{{ app_version }}"
```

Interview tip: Say facts.d records host state; set_stats chains jobs.

📚 **Official docs**

- Ansible: Variables and precedence

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q121 What is encrypt_string and when do you encrypt single values instead of whole files?

Vault & security

🎙 **Spoken answer · ~68 s**

The encrypt_string subcommand of ansible-vault encrypts one value and prints a YAML snippet tagged as vault-encrypted, which can be pasted into any variables file next to ordinary plain variables. The advantage is that diffs and reviews still show the variable name and surrounding structure, and merge conflicts are limited to a single value, whereas a fully encrypted file is opaque in review. The disadvantages are that long multi-line values like certificates are awkward, a larger number of encrypted items complicates rekeying, and it is easy to leave a plaintext value in history. Whole-file encryption is simpler to rotate and protects names too. My usual approach is a separate vault file per environment for secrets, holding prefixed variables, referenced from plain files, and encrypt_string only for occasional single values passed on the command line. Whichever approach I use, I scan commits for plaintext secrets with a pre-commit hook, use vault IDs to separate environments and keep the vault password out of the repository.

Likely follow-ups

- What is the trade-off in reviews?
- How do you avoid committing plaintext by mistake?

encrypt_string encrypts a single value for inline use, keeping diffs readable but making rekeying and multi-line values harder. Whole-file vaults rotate easier; many teams use per-environment vault files and scan commits for plaintext.

```
ansible-vault encrypt_string --vault-id prod@prompt 's3cret' --name 'db_password'
```

Interview tip: Say per-environment vault files plus secret scanning.

📚 **Official docs**

- Ansible Vault

interviewhub.eknathalabs.com
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q122 How do you use Ansible to scan and remediate compliance with OpenSCAP or CIS profiles?

Vault & security

🎙 Spoken answer · ~69 s

Compliance work splits into assessment and remediation. For assessment, I install the OpenSCAP scanner on hosts or the controller, run oscap with the chosen profile, such as a CIS or STIG profile for the OS, and fetch the XML and HTML reports back with the fetch module for archiving and review. Ansible also generates remediation playbooks from the same profiles, and the rules can be applied directly, but blind application is risky, so I review the generated tasks, remove those that break the service, and convert accepted ones into a role with variables for tunable values. Where mature roles exist, such as community hardening collections, I use and pin them rather than rewrite them. Changes roll out in stages, with check mode, a canary group and a rollback plan, since hardening can lock out SSH or break applications. Findings and exceptions are tracked with owners and expiry dates. Scans run on a schedule, results feed a dashboard, and drift is corrected by rerunning the role.

Likely follow-ups

- Why not apply generated remediation blindly?
- How do you keep exceptions controlled?

Run oscap with a CIS or STIG profile, fetch the reports, review generated remediation, turn accepted rules into a parameterised role, roll out in stages with check mode and a canary, track exceptions with owners and expiry, and rescan on a schedule.

```
oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis --results r.xml --
report r.html /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml
```

```
ansible-playbook harden.yml --check --diff --limit canary
```

Interview tip: Say review before applying; stage the rollout.

📚 Official docs

- Ansible documentation
- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q123 How do you manage TLS certificates with Ansible, including generation and renewal?

Vault & security

🎙 Spoken answer · ~68 s

The community.crypto collection provides modules to create private keys, certificate signing requests, self-signed certificates and to interact with ACME servers such as Let's Encrypt, so certificate lifecycles can be automated. For public certificates, the acme_certificate module performs the challenge and retrieves the certificate, with DNS challenges suiting hosts that are not publicly reachable, and the result is written to disk with strict permissions, followed by a handler that reloads the service. For internal certificates, a CSR is generated on the host, signed by an internal certificate authority through its API or by delegating to the CA host, and the certificate is distributed. Private keys should be generated on the target and never travel through the controller if possible; if a key must be distributed, it is vaulted. Renewal runs on a schedule, with a check that renews when the remaining validity is below a threshold, using the x509_certificate_info module. Monitoring alerts on expiry as a backstop, and cert-manager is preferable for Kubernetes workloads.

Likely follow-ups

- Why generate keys on the target?
- How does the renewal threshold work?

Use community.crypto to create keys and CSRs, acme_certificate for ACME, or sign internally; keep keys on the target with strict modes, reload services by handler, renew when remaining validity is below a threshold and back it with expiry monitoring.

```
- community.crypto.openssl_privatekey:
    path: /etc/ssl/private/app.key
    size: 4096
    mode: '0600'
```

```
- community.crypto.x509_certificate_info:
    path: /etc/ssl/app.crt
  register: cert
```

Interview tip: Say keys stay on the host; renew before expiry.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q124 How do you reduce SSH risk with bastion hosts, certificates and ProxyJump in Ansible?

Vault & security

🎙 Spoken answer · ~65 s

Production networks usually forbid direct SSH from the controller, so access goes through a bastion or jump host. Ansible supports this by setting ansible_ssh_common_args with ProxyJump, or the ssh configuration file referenced from ansible.cfg, so all connections route through the bastion with no per-host duplication. To reduce credential risk, I prefer short-lived SSH certificates issued by a certificate authority, such as Vault's SSH secrets engine or an SSH CA, over long-lived static keys, so access expires automatically and revocation is simple. Agent forwarding is avoided because a compromised bastion could use it, and ProxyJump is safer because the key never reaches the jump host. Host key checking stays enabled, with known host data distributed, rather than disabling verification, which exposes automation to man-in-the-middle attacks. Dedicated automation accounts have limited sudo, keys are rotated, and sessions are logged on the bastion. Connection settings live in group variables for each network zone so the routing is explicit and reviewed.

Likely follow-ups

- Why prefer ProxyJump over agent forwarding?
- What are the benefits of SSH certificates?

🗺 Bastion access



Route through a bastion with ProxyJump in ansible_ssh_common_args or ssh config, use short-lived SSH certificates instead of static keys, avoid agent forwarding, keep host key checking on, use dedicated limited accounts and log sessions.

```
ansible_ssh_common_args: '-o ProxyJump=ops@bastion.example.com'
```

```
[ssh_connection]
ssh_args = -F ./ssh.cfg
```

Interview tip: Say short-lived certs and ProxyJump, never disable host key checks.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q125 How do you audit what Ansible changed and who ran it?

Vault & security

🎙 Spoken answer · ~70 s

Auditability needs records at three levels. First, who and when: run automation through a controller such as AWX or a CI pipeline where every job has a user, a commit, a start time and stored output, instead of laptops. Second, what changed: use callback plugins that log task results to files, syslog, or a log platform, and keep diff mode output for files that matter, with no_log protecting secrets. The JSON or log-oriented callbacks produce structured events that can be indexed, and the profile callbacks add timing. Third, the change itself: Git history shows the exact playbook and inventory version used, and pull request approvals provide review evidence. On the managed hosts, auditd rules and the sudo logs show commands executed by the automation account, correlating with job times. I include the job identifier or commit hash in a marker on the host, such as a facts file or a motd line, so state can be traced back. Retention periods follow compliance requirements, and logs are protected from tampering.

Likely follow-ups

- Which callbacks help with structured logs?
- How do you tie host changes back to a job?

Run through AWX or CI for user and job records, log task results with callback plugins, keep Git history and approvals, use no_log for secrets, correlate with auditd and sudo logs on hosts, and stamp the commit or job id on the host.

```
[defaults]
stdout_callback = yaml
callbacks_enabled = ansible.posix.profile_tasks, ansible.builtin.log_plays
```

```
auditctl -w /etc/ssh/sshd_config -p wa -k ansible-change
```

Interview tip: Say controller records, Git history, host audit logs.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q126 How do host patterns and the limit option select targets precisely?

Scale & performance

🎙 Spoken answer · ~68 s

A host pattern chooses which hosts or groups a play or ad-hoc command targets. Names can be combined: a colon unions groups, an ampersand intersects, and an exclamation mark excludes, so web:&prod:!maintenance means web servers in production that are not in maintenance. Wildcards and regular expressions, prefixed with a tilde, match names, and subscripts select by position, such as the first host of a group, which helps for canary runs. The limit command-line option applies a further restriction on top of the play's own hosts line, and can read from a file prefixed with an at sign, which supports re-running only failed hosts. Patterns are evaluated against the inventory, so ansible-inventory or the list-hosts flag verifies the result before running anything risky. A play with hosts all combined with a careless limit can accidentally widen scope, so I keep production playbooks targeted by explicit variables and safety checks. Using limit for a canary host first and then the rest is a standard rollout pattern.

Likely follow-ups

- What does an ampersand do in a pattern?
- How do you re-run only the failed hosts?

Combine groups with colon, intersect with &, exclude with !, match with wildcards or ~regex, pick by subscript, and narrow further with --limit or @file. Verify with --list-hosts and use a canary host first.

```
ansible-playbook site.yml --limit 'web:&prod:!maintenance' --list-hosts
```

```
ansible-playbook site.yml --limit @site.retry
```

Interview tip: Say verify the target list before running.

📚 Official docs

- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q127 How do throttle, run_once and serial differ for controlling concurrency?

Scale & performance

🎙 Spoken answer · ~66 s

They limit concurrency at different levels. Serial is set on a play and splits the host list into batches, so the whole play, every task, completes on one batch before the next starts, which suits rolling updates. Forks, set globally, limits how many hosts Ansible works on in parallel for any single task. Throttle is set on a task or block and caps the number of hosts running that specific task concurrently, even when forks is higher, which protects a shared dependency such as a database, an API with rate limits or a package mirror, without slowing the rest of the play. Run_once executes a task on a single host of the current batch, usually for cluster-wide actions like running a schema migration, and its result is shared with the others, though with serial it runs once per batch. Delegate_to combined with run_once targets a specific server. Using the right one avoids both overloading systems and needlessly slow runs.

Likely follow-ups

- When would you use throttle over serial?
- What does run_once do with serial?

Serial batches whole plays for rolling updates, forks caps overall parallelism, throttle caps concurrency for one task or block to protect shared services, and run_once executes a task on a single host, once per batch when serial is used.

```
- name: Pull image
  community.docker.docker_image:
    name: app:1.2
    source: pull
  throttle: 5
```

```
- ansible.builtin.command: /opt/app/migrate
  run_once: true
```

Interview tip: Say throttle protects dependencies; serial protects availability.

📚 Official docs

- Microsoft: Cloud design pattern – Throttling
- Ansible: Strategies and performance

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q128 How do you profile Ansible runs with callbacks such as profile_tasks and timer?

Scale & performance

🎙 **Spoken answer · ~70 s**

To speed up a slow playbook, I measure first. The profile_tasks callback prints the duration of each task and a sorted list of the slowest ones at the end, and the timer callback shows the total run time. Profile_roles does the same per role. They are enabled in the configuration file through the callbacks setting, or by an environment variable for one run. The output usually shows that a few tasks dominate: fact gathering on many hosts, package cache updates, large loops that run a module per item, synchronise or copy of many small files, and tasks waiting on slow services. With that evidence I apply targeted fixes: disable or limit fact gathering, increase forks, enable pipelining and ControlPersist, replace loops with modules that accept lists, use archives or synchronise instead of many copies, cache expensive lookups and use async for long operations. I re-measure after each change to confirm the improvement, since guessing often optimises the wrong thing, and keep the callbacks enabled in CI for trend tracking.

Likely follow-ups

- Which callbacks give task timings?
- What often dominates run time?

Enable profile_tasks, profile_roles and timer callbacks to find the slowest tasks, then fix the real causes: fact gathering, per-item loops, many small file copies, low forks and missing pipelining. Re-measure after every change.

```
ANSIBLE_CALLBACKS_ENABLED=ansible.posix.profile_tasks,ansible.posix.timer ansible-playbook site.yml
```

Interview tip: Say measure before tuning.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q129 How do you run Ansible against thousands of hosts across many networks and regions?

Scale & performance

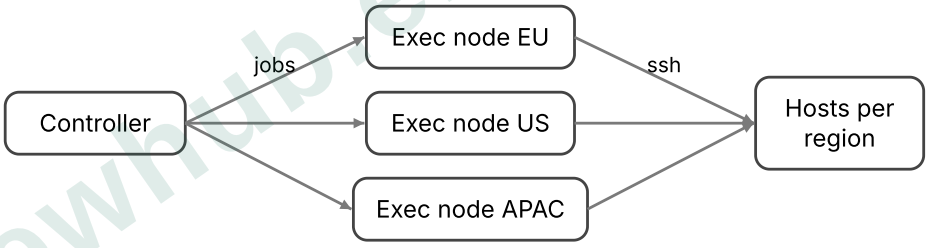
🎙 Spoken answer · ~69 s

At that scale, a single controller and one flat run stop working, so I split by topology. Execution is distributed: AWX or the automation platform with instance groups and execution nodes close to each region or network zone, so connections are local and firewalls are simpler, with the control plane coordinating jobs through a mesh. Inventories are dynamic and segmented by region, environment and role, and jobs slice the inventory into parts that run in parallel. Within a run, I tune forks, enable pipelining and persistent SSH connections, limit fact gathering with caching, and use free or batched strategies where ordering permits. Changes roll out in waves, with canary groups, health gates and automatic stop conditions, rather than all at once. Rate limits on shared services use throttle. Failures are handled by retrying subsets rather than the entire fleet. Observability collects job results and timing into dashboards. For configuration that must converge continuously across huge fleets, pull mode or an agent-based approach can complement push.

Likely follow-ups

- Why place execution nodes near targets?
- What is job slicing?

🗺 Distributed execution



Distribute execution with AWX instance groups or execution nodes per region, segment dynamic inventories, slice jobs, tune forks, pipelining and facts, roll out in canary waves with health gates, throttle shared services and consider pull mode for continuous convergence.

```
ansible-playbook site.yml -f 100 --limit 'region_eu' -e 'serial_pct=10'
```

Interview tip: Say distribute execution, slice inventory, roll out in waves.

📚 Official docs

- Ansible: Network automation
- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q130 How do you handle hosts that are slow, flaky or temporarily offline during a large run?

Scale & performance

🎙 **Spoken answer · ~66 s**

Large fleets always contain machines that are down, rebooting or unreachable, so the run must be resilient. Connection and command timeouts are set to sensible values so a dead host fails quickly instead of blocking a batch. The ignore_unreachable keyword lets a play continue past hosts that cannot be contacted, and the failed or unreachable hosts are listed in the recap, from which I build a limit file to retry later. For long-running tasks on slow hosts, async with polling avoids connection drops. Retries with until absorb transient errors. Strategy choices matter too: the free strategy stops fast hosts waiting for slow ones. Batching with max_fail_percentage prevents a widespread failure from being masked as a few offline machines. For machines that are legitimately offline, a scheduled re-run or ansible-pull applies changes when they return. I also report on persistently unreachable hosts so owners can fix or decommission them, instead of repeatedly retrying forever, and keep the inventory clean.

Likely follow-ups

- What does ignore_unreachable do?
- Why use the free strategy here?

Set short connect timeouts, use ignore_unreachable, retries and async for slow tasks, collect failed hosts into a limit file for a re-run, use the free strategy and failure thresholds, rely on ansible-pull for returning machines and clean up the persistently dead.

```
- hosts: all
  ignore_unreachable: true
  strategy: free
```

```
ansible-playbook site.yml --limit @site.retry
```

Interview tip: Say fail fast, retry later, clean up dead hosts.

📚 **Start here (official docs)**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q131 A playbook fails with SSH host key verification errors. How do you resolve them without disabling security?

Troubleshooting

🎙 **Spoken answer · ~71 s**

The message means the controller's known hosts file does not contain a matching key for the target, either because the host is new, was rebuilt, or something is intercepting traffic. Disabling host key checking removes protection against man-in-the-middle attacks, so I avoid it in production. For new hosts, I distribute the correct host keys deliberately: collect them with ssh-keyscan during provisioning from a trusted network, or better, bake them in or publish them via SSH certificates and a known-hosts file that trusts the certificate authority, so rebuilt machines are accepted automatically. For rebuilt servers whose key legitimately changed, I remove the old entry with ssh-keygen and verify the new fingerprint through an independent channel such as the cloud console before accepting it. In ephemeral environments like CI against disposable test hosts, relaxing the check for that scoped inventory is acceptable. The known_hosts module can manage the file declaratively. Repeated changes in production hosts deserve investigation, because they might indicate a rebuild, a load balancer in the path or a compromise.

Likely follow-ups

- How can SSH certificates avoid this problem?
- When is disabling the check acceptable?

Do not disable host key checking in production. Distribute verified host keys via provisioning, ssh-keyscan from a trusted network or an SSH CA, remove stale entries with ssh-keygen after verifying new fingerprints, and relax checks only for disposable test hosts.

```
ssh-keygen -R web01.example.com
```

```
ssh-keyscan -t ed25519 web01.example.com >> ~/.ssh/known_hosts
```

Interview tip: Say verify the new fingerprint out of band.

📚 **Official docs**

- OpenSSH manual pages
- Ansible: Playbook guide
- Ansible: Error handling

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q132 A task hangs forever with no output. How do you diagnose it?

Troubleshooting

🎙 Spoken answer · ~76 s

A hung task usually waits on something: a prompt, a lock, a network call or a process that never exits. First I rerun with high verbosity to see which task and connection are active, and on the target I look at the process tree to see what the module launched. Common causes are commands waiting for interactive input such as a confirmation or a password prompt, which need a non-interactive flag or the expect module; package managers blocked by an apt or dpkg lock held by another process; a service start that never detaches, so the command never returns, which needs a proper service module or async; a network call without a timeout, like a download from an unreachable URL; or sudo prompting for a password with no tty. SSH connections can also stall from broken network paths, which timeouts and ServerAlive settings detect. I add timeouts on tasks, use async with polling for long commands, set DEBIAN_FRONTEND to noninteractive, and use the debugger or strace on the stuck process if needed. Afterwards I fix the root cause rather than just raising timeouts.

Likely follow-ups

- Which flag helps with apt lock waits?
- How do you stop commands waiting for input?

Rerun with -vvv, inspect the process tree on the target, and look for prompts, package locks, non-detaching services, network calls without timeouts or sudo prompts. Add task timeouts, async, non-interactive flags and fix the cause.

```
ansible-playbook site.yml -vvv --limit web01
```

```
ps -ef --forest | grep -A3 ansible
```

```
environment:
  DEBIAN_FRONTEND: noninteractive
```

Interview tip: Say look at the process on the host, not just the controller.

📚 Official docs

- Ansible: Playbook guide

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q133 A boolean or string comparison behaves unexpectedly, for example 'yes', 'true' and '0' in conditions. What is going on?

Troubleshooting

🎙 **Spoken answer · ~72 s**

Ansible uses YAML and Jinja, and types are easily confused. The word yes in YAML may be a boolean in some contexts, while values read from environment variables, command output, extra vars on the command line and registered results are strings. A string such as false is truthy if used directly in a when condition without conversion in some situations, and the string 0 is not the number zero. Comparisons between a string and an integer silently fail, so a port read from a file will not equal a number in a variable. Facts like memory sizes are integers while values parsed from stdout are strings. Extra variables passed with key equals value syntax are always strings, while JSON-style extra vars keep types. The fix is explicit conversion: the bool filter for truthiness, int and float for numbers, and string for comparisons, and quoting values deliberately in YAML. I also use the debug module with the type_debug filter to see what a variable actually is, and validate inputs early with assert tasks.

Likely follow-ups

- How do you see a variable's type?
- Why are command-line extra vars strings?

Types get mixed between YAML, Jinja, command output and extra vars. Convert explicitly with bool, int and string filters, quote values, use JSON extra vars for typed input, inspect with type_debug and validate with assert.

```
- ansible.builtin.debug:
    msg: "{{ port | type_debug }}"
```

```
when: enable_feature | bool
```

```
ansible-playbook site.yml -e '{"port": 8080}'
```

Interview tip: Say convert explicitly; don't trust implicit truthiness.

📚 **Start here (official docs)**

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q134 Variables named item or other loop variables collide or are undefined inside included tasks. How do you fix this?

Troubleshooting

🎙 **Spoken answer · ~75 s**

The default loop variable is item, and nested loops or loops that include task files containing their own loops overwrite each other, giving wrong values or an error that item is undefined or has no attribute. Included tasks inherit the caller's variables, so an inner loop using the same name replaces the outer one for the rest of that iteration. The fix is to rename the variable using loop_var in loop_control on one or both loops, with a descriptive name such as user or vhost, so the scopes are distinct. When a task file is included in a loop, I document the expected variable in the file's header and validate it with assert. Registered variables inside loops produce a results list, and if the loop did not run because it was empty or skipped, the variable may exist without a results key, so downstream code should handle skipped and empty cases. Using clear variable names, avoiding set_fact in loops without care, and testing with small data sets avoid most of these problems. Ansible warns about loop variable reuse in newer versions.

Likely follow-ups

- How does loop_var solve the problem?
- What if the loop list is empty?

Nested loops and included task files overwrite the default item variable. Rename with loop_control loop_var, validate expected variables with assert, handle empty and skipped loops when reading registered results, and test with small data.

```
loop: "{{ vhosts }}"
loop_control:
  loop_var: vhost
vars:
  vhost_name: "{{ vhost.name }}"
```

Interview tip: Say give every loop its own variable name.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Variables and precedence

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q135 A command or shell task is not idempotent and always shows changed. How do you make it behave?

Troubleshooting

🎙 Spoken answer · ~67 s

Command and shell cannot know whether they changed anything, so they always report changed. To fix this, first ask whether a proper module exists, because replacing the command with a module solves idempotency, check mode and error handling together. If a command is truly needed, I give Ansible a way to decide. The creates and removes arguments skip the command when a file exists or is absent, which suits one-time initialisation. For read-only commands, changed_when false stops them from reporting a change. When the command outputs something meaningful, I register the result and set changed_when based on that output, such as a text indicating an update was applied, and failed_when for known acceptable non-zero exit codes. Where state must be checked first, a separate task queries the current state and a when condition runs the change only if needed. Atomic scripts should themselves be idempotent. I also test with Molecule's idempotence check, which flags tasks reporting changes on the second run.

Likely follow-ups

- What do creates and removes do?
- When use failed_when with a return code?

Prefer a real module. Otherwise use creates or removes, changed_when false for read-only commands, changed_when and failed_when based on registered output and return codes, or a prior check task, and enforce with Molecule idempotence.

```
- ansible.builtin.command: /opt/app/setup.sh
  args:
    creates: /opt/app/.setup_done
```

```
- ansible.builtin.command: grep -q enabled /etc/app.conf
  register: r
  changed_when: false
  failed_when: r.rc > 1
```

Interview tip: Say tell Ansible how to decide what changed.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- Ansible: Playbook guide

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q136 How do custom credential types work in AWX and the automation controller?

AWX & AAP

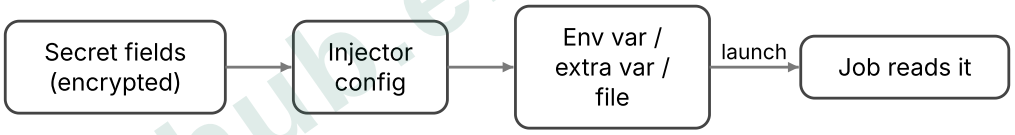
🎙 Spoken answer · ~73 s

Built-in credential types cover machines, cloud accounts, vaults and source control, but organisations often need to inject other secrets, such as an API token for an internal service. A custom credential type defines an input schema, the fields that users fill in and which are marked secret, and an injector configuration that says how those values reach the job. Injectors can set environment variables, add extra variables, or write files at runtime, such as a temporary configuration or key file whose path is exposed through a variable. Secret fields are encrypted in the database and never shown again. When a job template has a credential of that type attached, the controller renders the injectors into the execution environment just before the job runs, and removes them after. This lets playbooks read secrets as ordinary variables without knowing where they come from, and permissions govern who may use the credential without seeing it. Credentials can also be looked up from external secret managers through credential plugins, so the controller stores only a reference, and rotation happens centrally.

Likely follow-ups

- How do injectors deliver secrets?
- What are external credential lookups?

🗺 Credential injection



A custom credential type defines input fields, secret flags and injectors that set environment variables, extra vars or temporary files at job time. Users can use the credential without seeing it, and plugins can fetch values from external secret managers.

```
inputs:
  fields:
    - id: api_token
      label: API Token
      secret: true
injectors:
  env:
    INTERNAL_API_TOKEN: "{{ api_token }}"
```

Interview tip: Say users use credentials without seeing them.

📚 Official docs

- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q137 How do you manage AWX or the automation controller configuration as code?

AWX & AAP

🎙 Spoken answer · ~68 s

Clicking through the web interface does not scale and is hard to audit, so the controller's own objects are described in code. The ansible.controller collection, or the awx.awx collection for the community version, provides modules to manage organisations, projects, inventories, credentials, job templates, workflows, schedules and role assignments. A repository holds these definitions as variables, a playbook loops over them, and a pipeline applies them whenever they change, making the controller itself reproducible and recoverable. The awxkit command line tool and the REST API serve scripts and integrations, and the export and import tools can dump existing configuration into a structured file to bootstrap the repository. Secrets are not stored in the code, only references, with values supplied by the pipeline or external lookups. Changes go through pull requests like any other code. Doing so supports multiple controllers for different environments with the same configuration, disaster recovery by reapplying everything to a new instance, and drift detection by comparing the declared and actual state.

Likely follow-ups

- Which collection manages controller objects?
- How does this help disaster recovery?

Use the ansible.controller or awx.awx collection modules, awxkit or the API to define organisations, projects, inventories, templates and workflows in Git, apply them from a pipeline, keep secrets as references, and reuse the definitions across controllers and for recovery.

```
- awx.awx.job_template:
    name: Deploy app
    organization: Platform
    project: app-playbooks
    playbook: deploy.yml
    inventory: prod
    state: present
```

Interview tip: Say treat the controller itself as code.

📚 Official docs

- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q138 How do job slicing and concurrency settings work in AWX?

AWX & AAP

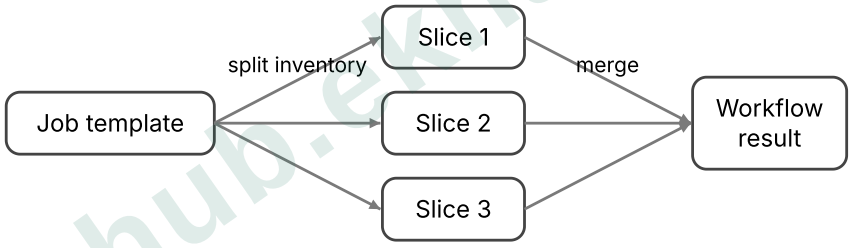
🎙 Spoken answer · ~71 s

Job slicing divides a large inventory into several smaller jobs that run in parallel across the cluster, packaged into a workflow job that groups them. For example, a template with slice count five splits the hosts into five roughly equal sets, each executed as a separate job on available instances, which reduces total run time for large fleets beyond what forks on one node can achieve. Slices work best when hosts are independent, since each slice is isolated and does not see the others, so coordinated tasks that need all hosts at once, such as cluster-wide steps, do not fit. Concurrency is controlled elsewhere: the allow simultaneous runs option on a template decides whether multiple runs of the same template may happen together, instance groups cap capacity, and forks per job limit parallelism within a job. Capacity is measured in forks, and the scheduler places jobs where capacity exists. I match slice count to available capacity and watch queues, because too many slices on limited nodes only create waiting jobs.

Likely follow-ups

- Which playbooks do not suit slicing?
- How is capacity measured?

🗺 Job slicing



Job slicing splits the inventory into parallel jobs inside a workflow, reducing run time for independent hosts; it does not suit cluster-wide coordination. Concurrency uses allow-simultaneous-runs, instance group capacity measured in forks, and per-job forks.

```
job_slice_count: 5
allow_simultaneous: false
forks: 50
```

Interview tip: Say slice independent hosts only; match capacity.

📚 Official docs

- Ansible Automation Platform / AWX
- Molecule documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q139 How do you troubleshoot AWX jobs that are stuck pending, fail to start or lose their output?

AWX & AAP

🎙 Spoken answer · ~72 s

I look at where the job is in its lifecycle. A job stuck in pending usually has no capacity: instance groups lack free forks, instances are disabled or offline, the template restricts to an unavailable group, or the project update or inventory sync it depends on is waiting or failed. The instances page and the dispatcher logs show capacity and errors. A job that fails to start often fails while launching the execution environment container: the image cannot be pulled because of registry credentials, a missing pull secret or a network restriction, or the container runtime on the node is unhealthy. On Kubernetes deployments, I check the task and web pods, the receptor and the job pod's events. A failed project sync points to Git credentials or an invalid playbook path. Truncated or missing output is related to receptor connections, result stream problems or disk and database pressure. I check controller logs, the supervisor processes, database health, and the node's disk space. Cleanup jobs keep tables small, and monitoring alerts on queue depth.

Likely follow-ups

- Why would a job stay pending?
- Where do you see container start failures?

Pending usually means no capacity, offline instances or a blocked project or inventory update; start failures are often execution environment image pulls or runtime issues; lost output points to receptor or resource pressure. Check instance capacity, dispatcher and receptor logs and pod events.

```
kubectl -n awx get pods
```

```
kubectl -n awx logs deploy/awx-task -c awx-task --tail=100
```

```
awx-manage list_instances
```

Interview tip: Say check capacity first, then the job pod.

📚 Official docs

- Kubernetes: Debug Pods
- Ansible Automation Platform / AWX

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q140 What do Event-Driven Ansible rulebooks look like and what are practical use cases?

AWX & AAP

🎙 Spoken answer · ~68 s

Event-Driven Ansible reacts to events instead of waiting for a human or a schedule. A rulebook is a YAML file with sources, rules and actions. Sources are plugins that listen to event streams, such as a webhook, a message queue like Kafka, a monitoring system's alerts or cloud events. Rules contain conditions that match event data, and actions say what to do when a rule matches, such as running a playbook, running a job template on the controller, setting a fact or posting a message. Practical uses include self-healing: when an alert says a disk is nearly full or a service is down, a rulebook triggers a remediation playbook; automatic ticket enrichment; responding to security events by isolating a host; and scaling or configuration changes after a deployment event. Safeguards matter: conditions must be precise to avoid loops, actions rate-limited, risky remediation gated by approval, and everything logged. Rulebooks are tested with sample events before production, and teams start with low-risk, well-understood fixes.

Likely follow-ups

- What are the three parts of a rulebook?
- What safeguards prevent runaway automation?

A rulebook has sources that listen for events, rules with conditions on event data, and actions such as running a playbook or job template. Use it for self-healing and event responses, with precise conditions, rate limits, approvals for risky actions and full logging.

```
- name: Disk alerts
  hosts: all
  sources:
    - ansible.eda.alertmanager:
        host: 0.0.0.0
        port: 5000
  rules:
    - name: Clean disk
      condition: event.alert.labels.alertname == "DiskFull"
      action:
        run_job_template:
          name: Cleanup disk
          organization: Platform
```

Interview tip: Say start with safe, well-understood remediation.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q141 How do you automate VMware vSphere with Ansible?

Cloud/network/K8s

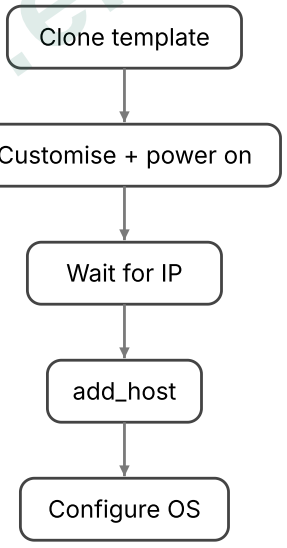
🎙 Spoken answer · ~73 s

The community.vmware collection provides modules to clone and create virtual machines from templates, set CPU and memory, attach networks and disks, manage snapshots, power state, tags, folders, resource pools and datastores, and to gather information. A dynamic inventory plugin reads vCenter and groups machines by folder, tags, cluster or custom attributes so configuration plays can target them. Authentication uses a service account with a role limited to the required operations, stored in vault or fetched from an external secret manager, and TLS validation stays enabled with a trusted certificate. Typical workflow: a play running on the controller or a delegated host creates the VM from a template with customisation specifications for hostname and network, waits for VMware tools to report an address, adds the host to inventory, then a second play configures the operating system. Idempotency relies on VM names and consistent parameters. Large operations are throttled because vCenter can become a bottleneck, and snapshot cleanups are scheduled to avoid performance impact. Terraform may handle provisioning where lifecycle and state are important, with Ansible doing configuration.

Likely follow-ups

- How do you wait for a new VM to be ready?
- Why throttle vCenter operations?

🗺 vSphere provisioning



Use community.vmware modules and the vCenter inventory plugin with a least-privilege service account in vault, clone from templates, customise, wait for tools and IP, then configure the OS in a second play. Throttle vCenter and keep TLS validation on.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- community.vmware.vmware_guest:
    hostname: "{{ vcenter }}"
    name: app01
    template: rhel9-template
    state: poweredon
    wait_for_ip_address: true
  delegate_to: localhost
```

Interview tip: Say provision in one play, configure in the next.

📚 **Official docs**

- Ansible documentation

Q142 How do you use dynamic inventory plugins for Azure and GCP?

Cloud/network/K8s

🎙 **Spoken answer · ~69 s**

Cloud inventory plugins build the inventory from the provider's API at run time, so the inventory always reflects reality. For Azure, the azure_rm plugin reads subscriptions and resource groups, using a service principal or managed identity, and can create groups and variables from tags, locations and resource groups. For Google Cloud, the gcp_compute plugin lists instances by project and zone, with filters and keyed groups based on labels. A YAML configuration file, named with the plugin's required suffix, sets credentials, regions, filters, hostnames and compose expressions that map cloud attributes to variables such as ansible_host. Keyed groups turn tags or labels into groups like env_prod or role_web, which feed group variables. Caching reduces API calls for large estates, with a timeout. Credentials come from workload identity or environment variables rather than files in the repository. Filters limit the result to relevant resources, avoiding enormous inventories, and I validate output with ansible-inventory before running. Hostnames should be stable identifiers, preferably private addresses when running inside the network.

Likely follow-ups

- What do keyed groups do?
- Why cache dynamic inventory?

Use azure_rm and gcp_compute plugins configured in YAML with identity-based credentials, filters, compose for ansible_host and keyed_groups to turn tags or labels into groups. Enable caching for large estates and validate with ansible-inventory.

```
plugin: azure.azcollection.azure_rm
auth_source: auto
keyed_groups:
  - prefix: env
    key: tags.environment
```

```
ansible-inventory -i azure_rm.yml --graph
```

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Interview tip: Say tags become groups; verify with ansible-inventory.

📚 **Official docs**

- Ansible: Inventory
- Ansible: Developing modules
- Ansible: Cloud guides

Q143 How do you automate database administration tasks with Ansible, such as users, schemas and backups?

Cloud/network/K8s

🎙 **Spoken answer · ~67 s**

Database collections, such as community.postgresql, community.mysql and community.mongodb, provide modules to create databases, roles and users, grant privileges, manage extensions, run queries and configure replication. Declarative tasks let me define the desired roles and grants as variables, so permissions are reviewable and repeatable across environments. Passwords come from vault or an external secret manager, and tasks use no_log so credentials do not appear in output. Backups can be orchestrated by running native dump tools through command tasks with proper locking, writing to protected storage, and verifying with a periodic restore test, scheduled from the controller or a timer. Schema migrations are better handled by application migration tools run as a step in the deployment flow, not by ad-hoc SQL in playbooks. Ansible is good for provisioning installs, parameter files, replication topology and user management, while a managed database service or operator may be preferable for complex failover. Risky operations are gated by approvals, check mode where supported, and backups taken first.

Likely follow-ups

- Why use no_log with database tasks?
- What belongs to migration tools instead?

Use community.postgresql, mysql or mongodb modules to manage roles, grants and settings declaratively, keep credentials in vault with no_log, orchestrate backups with restore tests, leave schema migrations to migration tools, and gate risky changes.

```
- community.postgresql.postgresql_user:
    name: appuser
    password: "{{ vault_appuser_password }}"
    db: appdb
    priv: ALL
  no_log: true
```

Interview tip: Say declare users and grants; migrations belong to the app pipeline.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible: Playbook guide
- Ansible documentation

Q144 How do you manage Docker or Podman containers and images with Ansible, and when is it appropriate?

Cloud/network/K8s

🎙 **Spoken answer · ~67 s**

The community.docker collection manages images, containers, networks, volumes and compose projects, and containers.podman covers Podman and quadlet-based systemd units. Tasks pull an image by tag or digest, create a container with ports, environment, volumes, restart policy and health checks, and keep it in the desired state, recreating it when parameters change. This is appropriate for single hosts or small groups where Kubernetes would be excessive, for edge devices, for build agents, and for bootstrapping a container runtime. Good practice is using pinned image digests, storing secrets in vault and passing them securely, defining resource limits, and using systemd units generated by Podman so containers start on boot and are managed with the service module. Monitoring and log collection need explicit configuration. Compared with Kubernetes, there is no scheduling, self-healing across nodes or rolling updates, so I add rolling logic with serial and health checks. For larger fleets, an orchestrator is preferable, while Ansible still installs and configures the runtime and nodes.

Likely follow-ups

- Why pin images by digest?
- What do you lose compared with Kubernetes?

Use community.docker or containers.podman to pull pinned images and run containers declaratively with limits, restart policy and health checks, preferably as systemd units; appropriate for single hosts and edge cases, while larger fleets need an orchestrator.

```
- community.docker.docker_container:
    name: web
    image: "nginx@sha256:{{ nginx_digest }}"
    restart_policy: unless-stopped
    published_ports: ['80:80']
    state: started
```

Interview tip: Say Ansible for small container fleets, Kubernetes beyond that.

📚 **Official docs**

- Ansible documentation
- Docker documentation
- Podman documentation

carts.eknathalabs.com
InterviewHub - interviewhub.ekn

Q145 How do you roll out load balancer and reverse proxy configuration changes with Ansible without dropping traffic?

Cloud/network/K8s

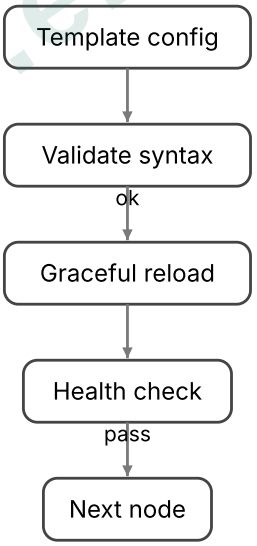
🎙 Spoken answer · ~73 s

Configuration for HAProxy, Nginx or cloud load balancers should change in a controlled sequence. With a pair of balancers, I update one at a time using serial, taking each out of rotation if there is a front virtual IP, applying a templated configuration, validating the syntax with the tool's check command before replacing the live file, and reloading gracefully instead of restarting so existing connections complete. A post-change health check confirms that the listener is up and returns expected responses before moving to the next node. For backend pool changes, such as adding or draining application servers, I mark servers as draining through the admin socket or cloud API, wait for connections to finish, perform the deployment, and re-enable them after a health check passes. The template validate option prevents a broken file going live, and the previous configuration is kept for quick rollback. All changes run first in a staging environment and in check and diff mode, and the work is wrapped in a block with rescue to restore the old configuration on failure.

Likely follow-ups

- Why reload rather than restart?
- How does draining a backend work?

🗺 LB config rollout



Roll out one balancer at a time with serial, validate templates before activating, reload gracefully, health-check before continuing, drain backends before deployments and keep the previous config for rollback in a rescue block.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
- ansible.builtin.template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
    validate: 'haproxy -c -f %s'
  notify: reload haproxy
```

```
- community.general.haproxy:
    state: disabled
    host: "{{ inventory_hostname }}"
    drain: true
  delegate_to: lb01
```

Interview tip: Say validate, reload, health-check, then next node.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Cloudflare Learning: What is a reverse proxy?
- NGINX documentation

Q146 **How do you verify the result of a role or playbook with Testinfra, Goss or assert tasks?** Testing & CI/CD

🎙 **Spoken answer · ~71 s**

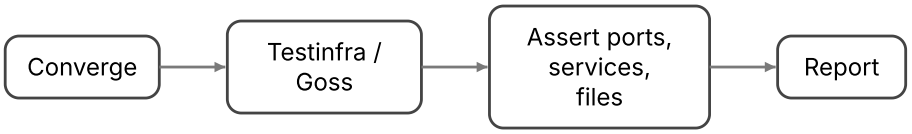
Running a playbook successfully only proves the tasks executed, not that the system is correct, so I add verification. Within Ansible, assert tasks check facts and command results against expectations, and the service_facts, package_facts and uri modules gather state to assert on, such as a service running, a port listening or an endpoint returning the right status. For independent checks, Testinfra, a pytest plugin, connects to the host and asserts on packages, files, services, ports and users using Python, and integrates with Molecule as its verifier. Goss offers a YAML-based alternative that is fast and language neutral, and serverspec is another option. Tests should check outcomes rather than repeat the implementation, for example that the web server answers on the right port and the configuration file has the right permissions, not that a template task ran. They run after convergence in CI against containers or VMs, and a subset can run in production as smoke tests after a deploy. Failing tests block promotion, and test results are published alongside pipeline logs.

Likely follow-ups

- What does Testinfra use under the hood?
- Why test outcomes rather than tasks?

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

🗺 **Independent verification**



Use assert with service_facts, package_facts and uri for in-play checks, and Testinfra or Goss as independent verifiers, ideally through Molecule. Test outcomes such as ports, services and permissions, run in CI and as production smoke tests.

```
def test_nginx(host):
    assert host.package('nginx').is_installed
    assert host.service('nginx').is_running
    assert host.socket('tcp://0.0.0.0:80').is_listening
```

```
molecule verify
```

Interview tip: Say verify outcomes independently of the tasks.

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Roles
- Ansible documentation

terviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q147 What does a GitHub Actions or GitLab CI pipeline for Ansible look like in practice?

Testing & CI/CD

🎙 **Spoken answer · ~72 s**

A practical pipeline has stages that increase in cost and risk. On every merge request: install pinned tool versions in a clean environment, often a container image holding ansible-core, collections and linters; run yamllint and ansible-lint, a syntax check on playbooks, and Molecule tests for changed roles using a container or VM driver. A job can render templates and run check mode against a disposable test inventory. Merging to the main branch deploys to a staging environment through the controller or a direct ansible-playbook call with credentials supplied by short-lived federation rather than stored long-lived keys, followed by smoke tests. Production runs require manual approval, use a protected environment, run serially with health gates, and record the commit and job logs. Collections and roles come from requirements files with pinned versions, cached between runs. Secrets arrive from the CI secret store or vault, masked in logs. Concurrency groups prevent two deployments to the same environment, and notifications report results. The pipeline definition lives in the same repository and is reviewed like other code.

Likely follow-ups

- How are credentials supplied safely?
- Why pin tool versions in a container image?

Merge requests run yamllint, ansible-lint, syntax-check and Molecule in a pinned tool image; main deploys to staging with smoke tests; production needs approval and serial rollout. Use short-lived credentials, pinned requirements, masked secrets and concurrency groups.

```
ansible-galaxy install -r requirements.yml
```

```
ansible-lint && ansible-playbook site.yml --syntax-check
```

```
molecule test
```

Interview tip: Say lint, test, stage, approve, roll out.

📚 **Official docs**

- Ansible documentation
- GitHub Actions documentation
- GitLab CI/CD documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q148 What do you look for when reviewing Ansible pull requests?

Testing & CI/CD

🎙 Spoken answer · ~69 s

A review checks correctness, safety and maintainability. For correctness, I look for idempotent tasks, proper use of modules instead of shell, correct conditions and handlers, variables defined and named consistently, templates that render valid configuration, and check mode support. For safety, I examine blast radius: which hosts are targeted, whether limits, serial and failure thresholds are set, whether destructive operations are guarded, whether privilege escalation is minimal, and whether secrets are vaulted, kept out of logs with no_log, and never inline. I check that changes to defaults do not silently alter other environments, that dependencies and collection versions are pinned, and that a rollback path exists. For maintainability, I want clear names, small tasks, documented variables, argument specs for roles, passing lint and tests, and updated documentation or changelog. I also read the diff of rendered output where possible and ask for evidence such as a Molecule result or a staging run. Automated checks handle style so that human review focuses on risk and design.

Likely follow-ups

- Which red flags do you check for first?
- How do automated checks help reviewers?

Review idempotency, module choice, conditions, handlers and variables; blast radius with limits, serial and guards; secrets and privilege; pinned dependencies and rollback; plus names, docs and passing lint and tests. Automation handles style so people focus on risk.

```
ansible-playbook site.yml --check --diff --limit staging
```

```
git diff main -- roles/ group_vars/
```

Interview tip: Say review blast radius and secrets first.

📚 Official docs

- Ansible documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q149 How do you upgrade ansible-core and collections safely when versions change and features are deprecated?

Testing & CI/CD

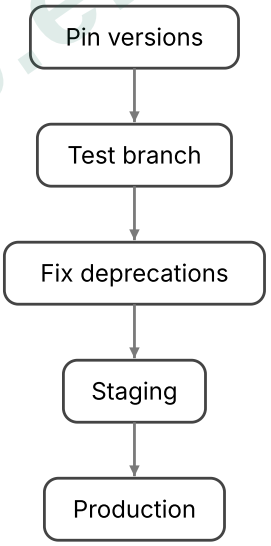
🎙 Spoken answer · ~68 s

Upgrades should be planned and tested, not accidental. I pin ansible-core and every collection in requirements files and in the execution environment image, so all environments run identical versions. When a new core release arrives, I read the porting guide and changelog, then run the existing test suite on a branch with the new version, watching for deprecation warnings, which I enable and treat as work items, since removed features break in the next major version. Collection upgrades are handled one at a time, reading release notes for changed module parameters and return values. Playbooks that rely on removed behaviour, such as old include directives, bare variables in conditions or short module names, are updated. Controller and managed-node Python requirements are checked, because newer core versions drop support for old target interpreters. Rollout goes from a development environment to staging and then production, with a rollback by reverting the pinned versions. Dependabot or Renovate propose version bumps, making upgrades small and frequent.

Likely follow-ups

- Why pin collections as well as core?
- What do deprecation warnings tell you?

🗺 Safe upgrade



Pin ansible-core and collections, read porting guides, run the test suite on a branch with the new version, fix deprecation warnings, check Python requirements for targets and promote through environments with rollback by reverting pins. Bots propose small frequent bumps.

```
ansible --version && ansible-galaxy collection list
```

```
ANSIBLE_DEPRECATION_WARNINGS=true molecule test
```

certs.eknathalabs.com

Interview tip: Say pin everything and upgrade in small steps.

📚 **Official docs**

- Kubernetes: Version skew policy
- Ansible: Roles
- Ansible documentation

Q150 How do you document Ansible roles, playbooks and runbooks so other teams can use them?

Testing & CI/CD

🎙 **Spoken answer · ~68 s**

Documentation should live with the code and be partly generated so it stays accurate. Each role has a readme describing purpose, supported platforms, requirements, an example playbook, and a table of variables with defaults and meanings, ideally generated from the argument specs file so it cannot drift. Collections include documentation for modules and plugins using the standard docstring format, viewable with ansible-doc and publishable as a documentation site. Repositories explain the layout, how to set up the tooling, how to run tests, and the release process. Playbooks have comments for non-obvious decisions, and tasks have meaningful names. Operational runbooks describe how to run common procedures, with the exact commands or job templates, expected output, rollback steps and who to contact, and link to the controller templates. Changelogs follow a standard fragment-based process so releases are traceable. Diagrams of inventory structure and rollout flow help newcomers. Documentation updates are part of the definition of done in pull requests, and a periodic review removes outdated parts.

Likely follow-ups

- How can documentation be generated?
- What belongs in a runbook?

Keep readmes with purpose, platforms, example and a variables table generated from argument specs; document modules with ansible-doc docstrings; explain repo layout and testing; keep runbooks with commands and rollback; use changelogs and require doc updates in pull requests.

```
ansible-doc -t role -l
```

```
ansible-doc my.collection.my_module
```

Interview tip: Say docs live in the repo and are generated where possible.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Ansible: Playbook guide
- Ansible: Roles
- Ansible documentation

terviewhub.eknathalabs.co

certs.eknathalabs.com
InterviewHub - interviewhub.ekn